



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 wide notification number F-9-12/2001-U-3 of Government of India)

॥ वसुधैव कुटुम्बकम् ॥ Re-Accredited by NAAC with 'A++' Grade



Course: Digital Electronics and Logic Design

Course Code: 0705210307

Syllabus of the subject [Credits: 3, Max. Marks: 75 (CA: 30 + ESE: 45), Contact Hours: 2]



Sr. No.	Topic	Actual Teaching Hours	Contact Hours Equivalence
1	Data and number systems Binary, Octal, and Hexadecimal representation and their conversions; BCD, ASCII, Gray codes, and their conversions; Signed binary number representation with 1"s and 2"s complement methods, Binary arithmetic	7	7



Unit 1- Number System:

Digital means??



a signal expressed as series of the digits 0 and 1

Today, the digital technology is used not only in computers but also in many other applications such as TV, Radar, Military systems, Medical equipment, Communication systems, Industrial process control etc.



Identify digital device and analog device





SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR



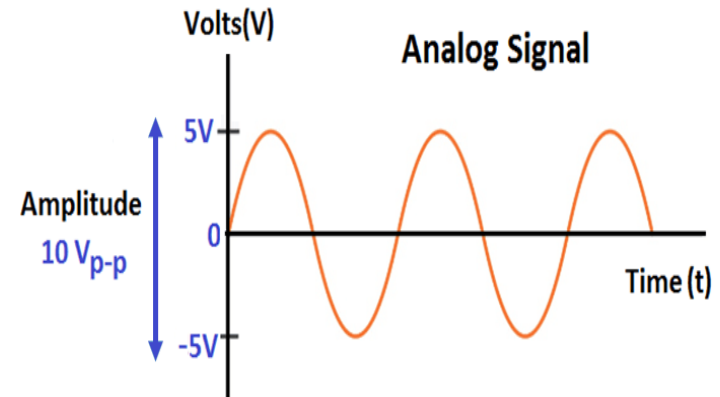
SIGNALS-

It is a physical quantity which contains some information

- The signals are classified into two categories.

1. Analog signals:-

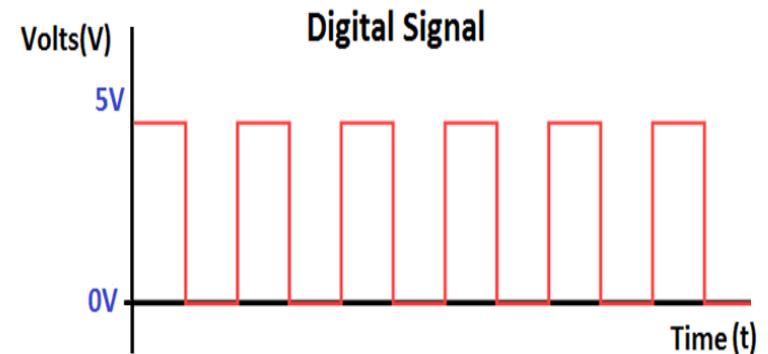
They vary continuously with time.



2. Digital signals.

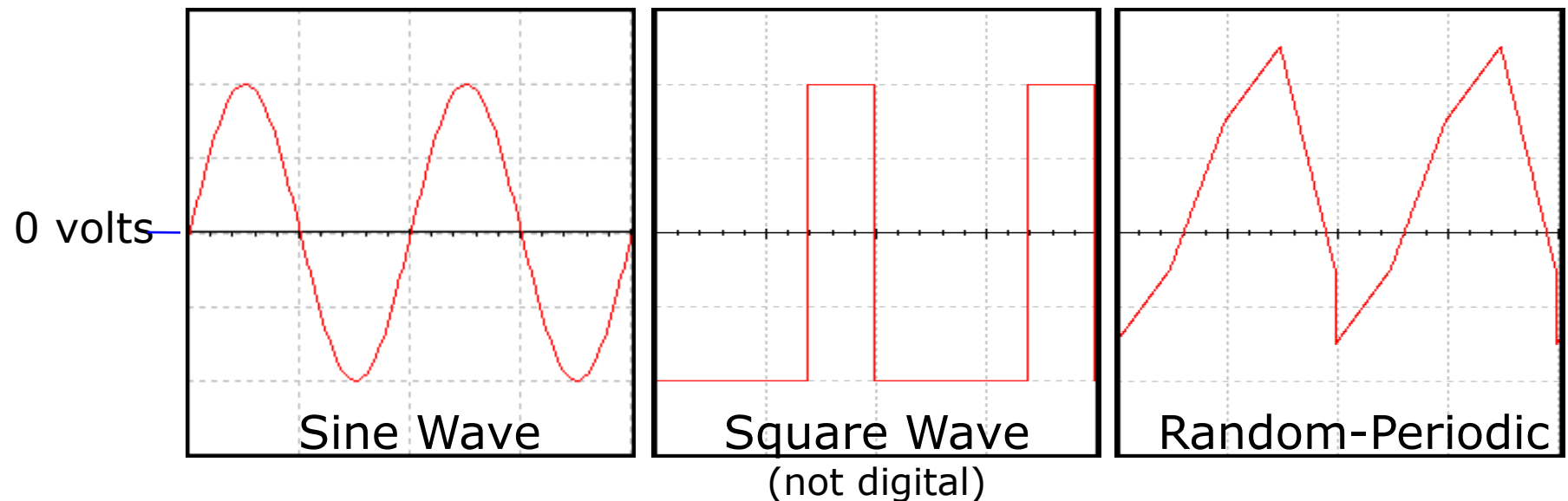
They do not vary continuously with time.

They are discrete with time.



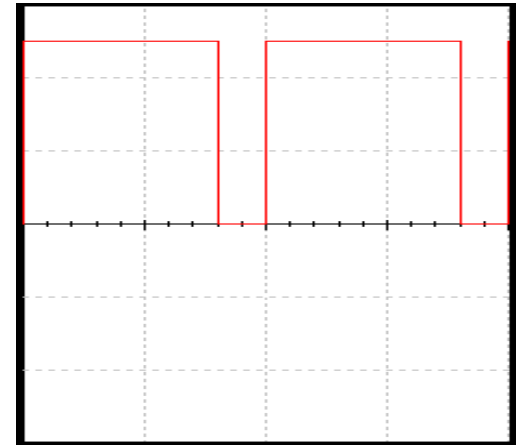
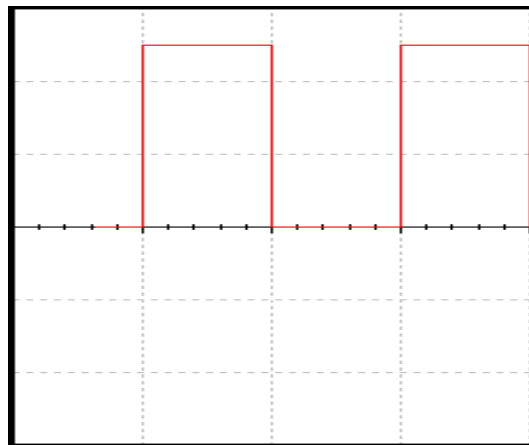
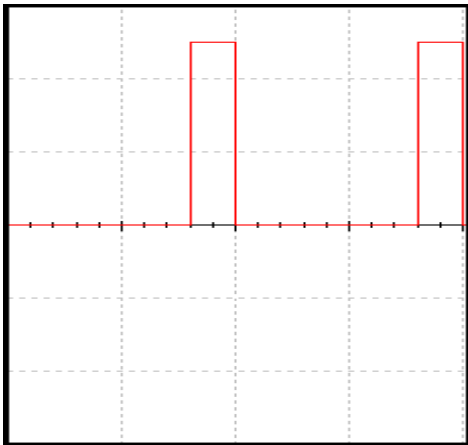
Example of Analog Signals:

- An analog signal can be any time-varying signal.
- Minimum and maximum values can be either positive or negative.
- They can be periodic (repeating) or non-periodic.
- Sine waves and square waves are two common analog signals.
- Note that this square wave is not a digital signal because its minimum value is negative.
- Video and Audio



Example of Digital Signals:

- Digital signals are commonly referred to as square waves or clock signals.
- Their minimum value must be 0 volts, and their maximum value must be 5 volts.
- They can be periodic (repeating) or non-periodic.
- The time the signal is high (t_H) can vary anywhere from 1% of the period to 99% of the period.
- Text and Integers.



Advantages of Digital Signal

- Digital signal can be processed and transmitted more efficiently and reliably than analog signals.
- It is possible to store the data
- The effect of noise(unwanted voltage fluctuations) is less. So digital data does not get corrupt.



Comparison of Analog and Digital Signal

Sr.No	Parameter	Analog Signal	Digital Signal
1	Number of values	Infinite	Finite
2	Nature	Continuous	Discrete
3	Sources	Transducers, Signal generator	Computers, A to D Converters
4	Examples	Sinewave, triangular wave	Binary signal
5	Applications	Operational Amplifiers, telephone	Television, Military systems, Microprocessors



Systems or Circuits

- Analog Systems:
The system which required **analog** signals.
Ex- filters, amplifiers.
- Digital Systems:
The system which required **Digital** signals.
Ex- flip flop, shift register, timer, counter.

Sources of Digital Signal:

Computer-All the data used by the computers is digital

A to D Converter



Analog system

- **Disadvantages:-**

1. Less accurate
2. Less reliability
3. Storage & processing data is not possible
4. Their parameter change with time.
5. System affected due to noise.



Digital system

- **Advantages**

1. They have more accuracy.
2. Highly reliable system
3. Less affected by noise.
4. Digital systems have memory hence Information and data can be stored and processed.
5. Design is easier.
6. They are less affected by variation of temperature.
7. They are cost efficient
8. Communication between the digital systems is much easier



Comparison

Sr no	Parameter	Analog systems	Digital systems
1.	Type of signals processed	Analog signals	Digital signals
2.	Type of display	Analog meters	Digital displays using LED and LCD
3.	Accuracy	Less	More
4.	Design complexity	Difficult to design	Easier to design
5.	Memory	No memory	They have Memory
6.	Storage of information	Not Possible	Possible
7.	Effect of noise	More	Less
8.	Versatility	Less	More
9.	Distortion	More	Less
10.	Effect of temperature and ageing on performance	More	Less
11.	Communication between systems	Not easy	Easy
12.	Examples	Filters, amplifiers, power supplies, signal generators	Counters, resisters, microprocessors, Computers



Questions

- 1) What is signal? Define analog and digital signal.
- 2) State advantages of digital signal.
- 3) State disadvantages of analog signal
- 4) State advantages of digital system.
- 5) Compare analog and digital signal.
- 6) Compare analog and digital system.



Binary Logic and Logic levels

Statement which is true if some condition is satisfied and false if not satisfied-----Logic

Logic Denoted by two voltage levels—

- 1) High level (logic 1)
- 2) Low level (logic 0)

- **Positive logic** is defined as a high voltage level representing a logic 1 and a low voltage level representing a logic 0.

+ve logic	-ve logic
high voltage correspond to logic '1'	high voltage correspond to logic '0'
'+5V' = LOGIC "1"	'+5V' = LOGIC "0"
'0V' = LOGIC "0"	'0V' = LOGIC "1"

- **Negative logic** is the reverse, i.e., a low voltage level represents a logic 1 and a high voltage level represents a logic 0



1.1 Terms- Bit, Byte, Nibble

A **bit** is a single **0** or **1** on its own.

1

A group of 4 bits together is called a **nibble**

1010

A group of 8 bits together is called a **byte** and this is one of the main units of measurement in all of computing

11011010



Important Definition

- Base or Radix:

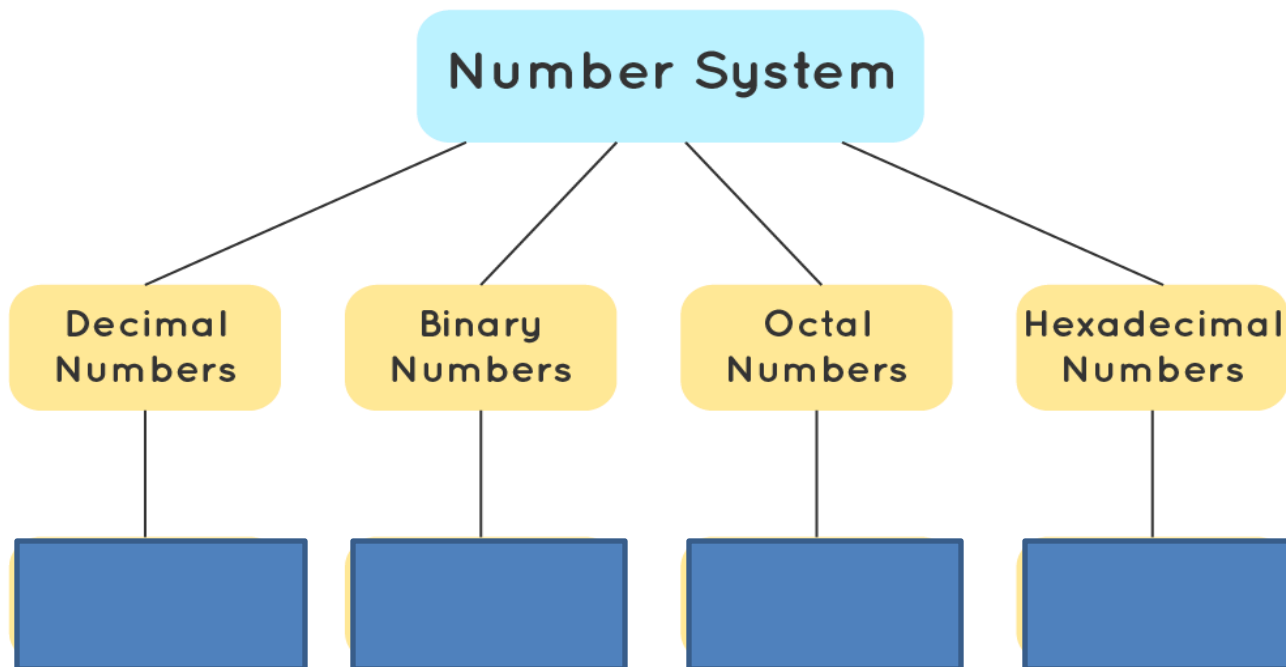
The radix or base is **the number of unique digits, including the digit zero, used to represent numbers.**

For example, for the decimal/denary system (the most common system in use today) the radix (base number) is ten, because it uses the ten digits from 0 through 9.



1.2 Number System

- A numeral system (or system of numeration) is a **writing system for expressing numbers.**





Different types of number systems

Numbering Systems		
System	Base	Digits
Binary	2	0 1
Octal	8	0 1 2 3 4 5 6 7
Decimal	10	0 1 2 3 4 5 6 7 8 9
Hexadecimal	16	0 1 2 3 4 5 6 7 8 9 A B C D E F



Counting

Decimal	Binary	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12

Common Number Systems

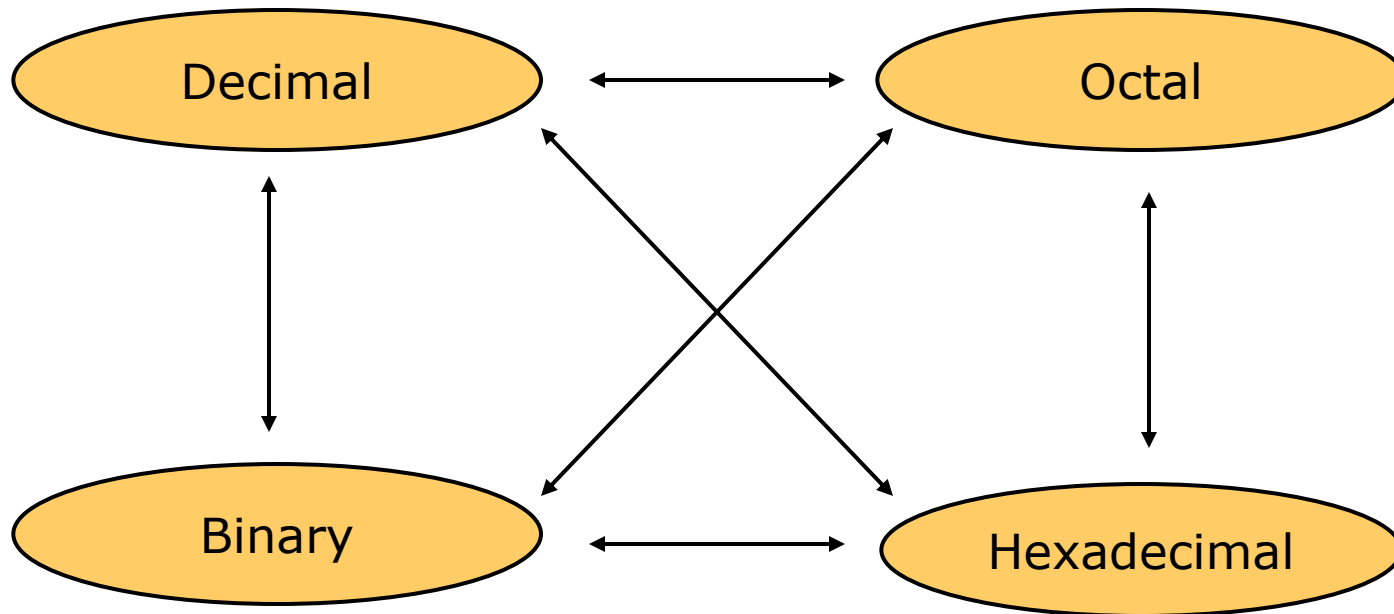
System	Base	Symbols	Used by humans?	Used in computers?
Decimal	10	0, 1, ... 9	Yes	No
Binary	2	0, 1	No	Yes
Octal	8	0, 1, ... 7	No	No
Hexa-decimal	16	0, 1, ... 9, A, B, ... F	No	No

Then why Octal and Hexadecimal systems used?????



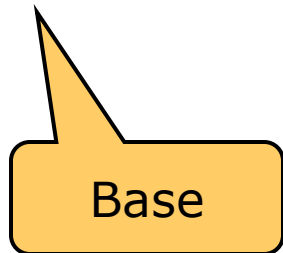
Conversion Among Bases

- The possibilities:

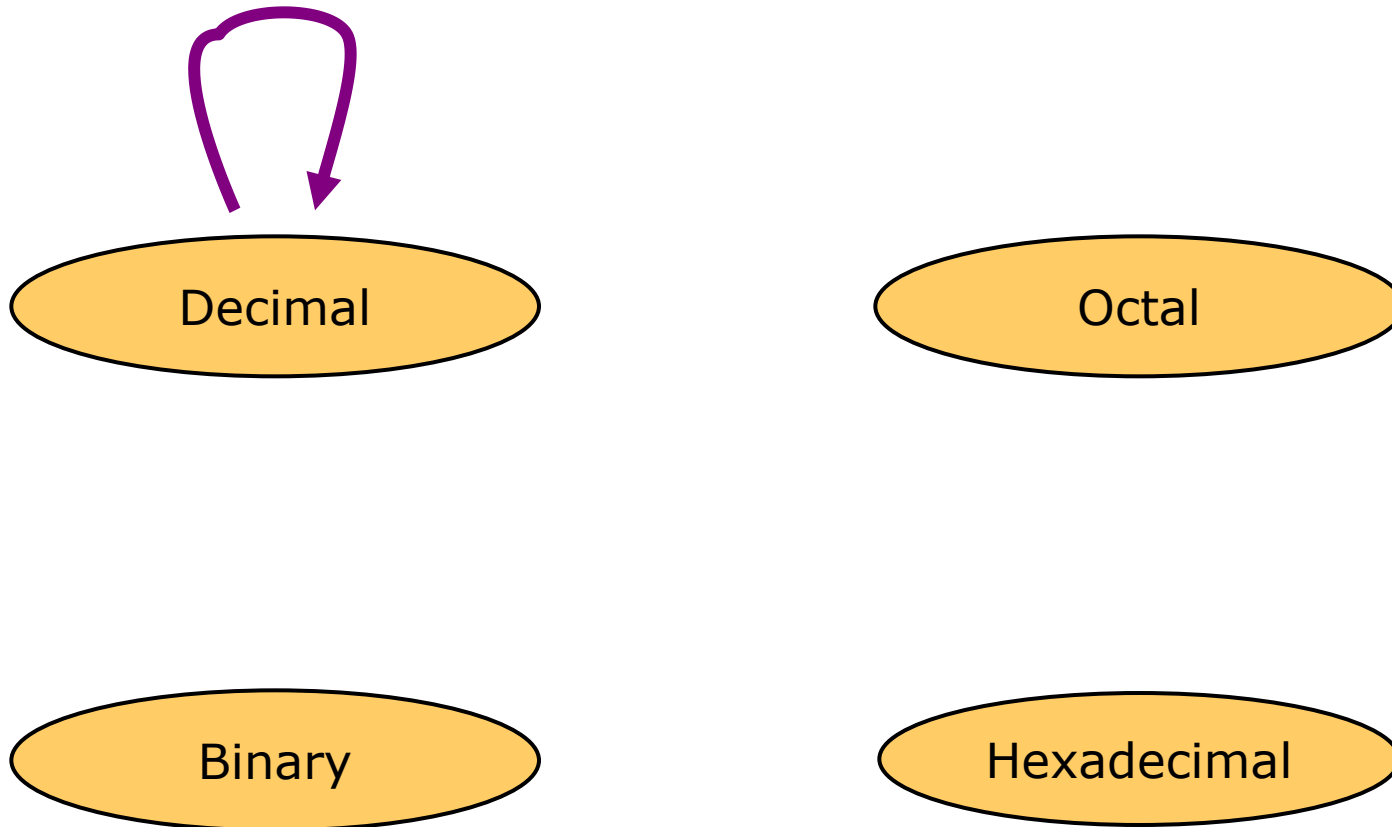


Quick Example

$$25_{10} = 11001_2 = 31_8 = 19_{16}$$



Decimal to Decimal



Next slide...



Conversion Decimal to Decimal

Decimal number	3	4	9	.	2	5
	↓	↓	↓		↓	↓
Position of Digits	10^2	10^1	10^0		10^{-1}	10^{-2}
	3×10^2	4×10^1	9×10^0		2×10^{-1}	5×10^{-2}

$$(3 \times 10^2) + (4 \times 10^1) + (9 \times 10^0) + (2 \times 10^{-1}) + (5 \times 10^{-2})$$

$$(300) + (40) + (9) + (0.2) + (0.05)$$

349.25





Other System into Decimal

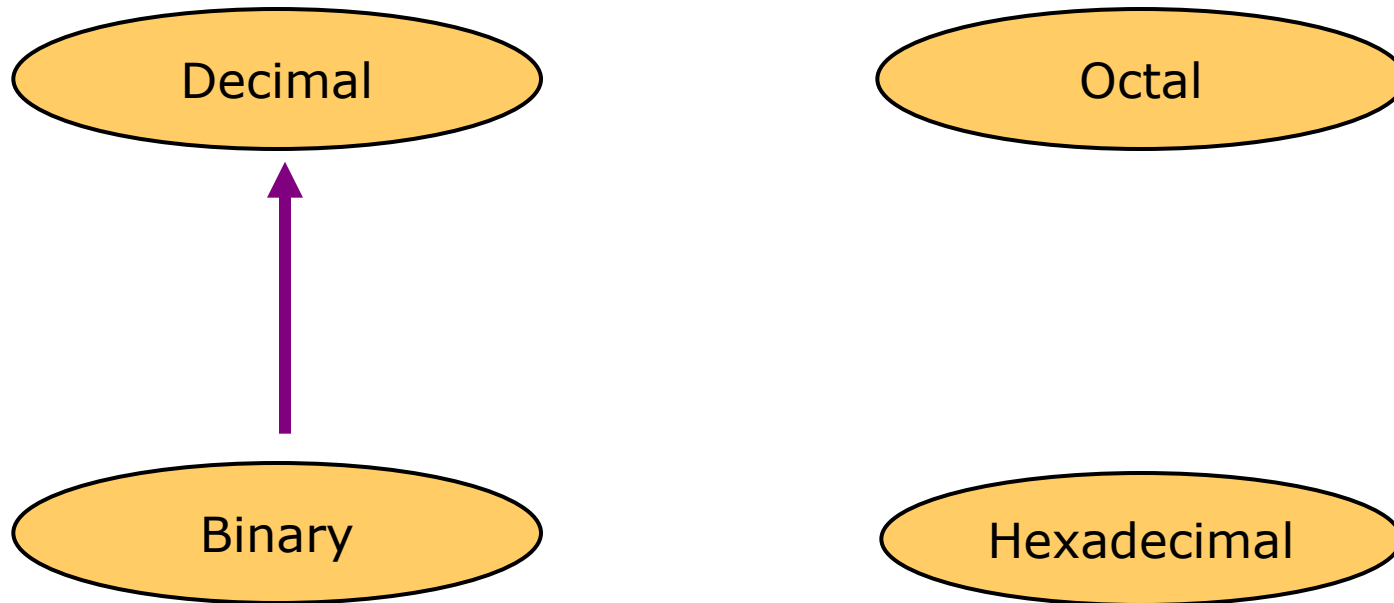
(into means Multiplication)

Steps to be followed:

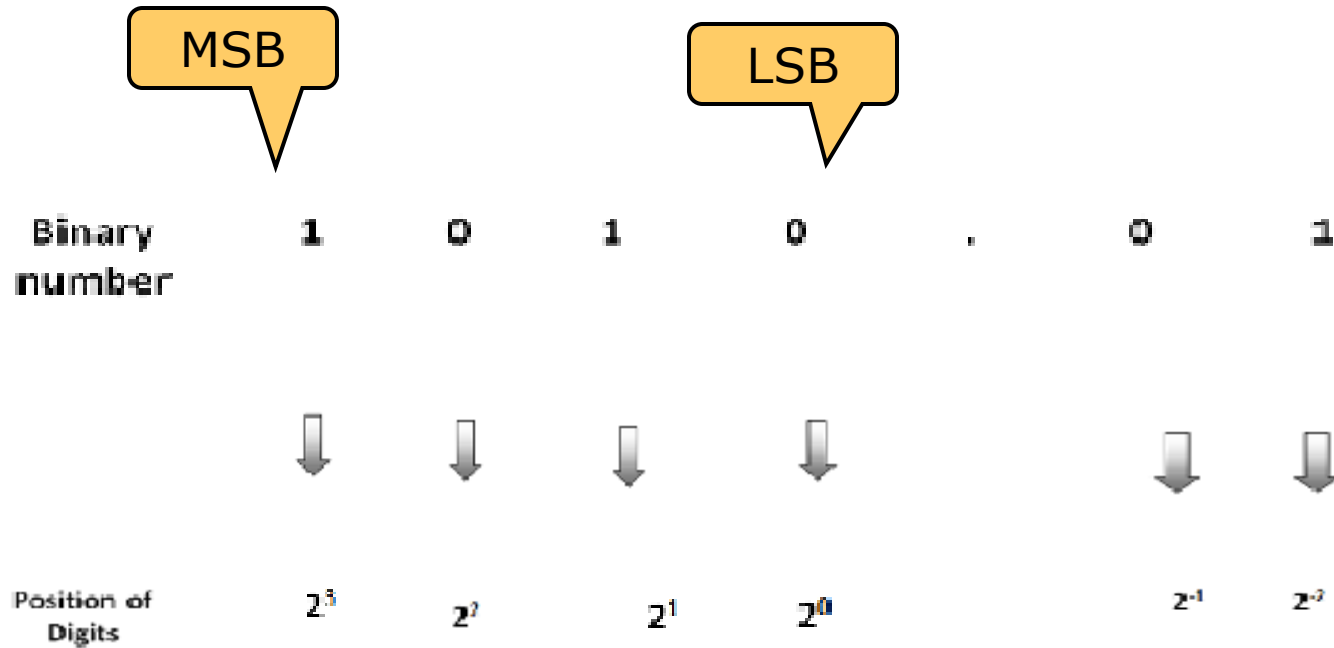
1. Note down the given number
2. Write down the weights corresponding to different positions
3. **Multiply** each digit in the given number with the corresponding weight
4. Obtain product number
5. Add all the product number to get the decimal equivalent.



Binary to Decimal



Conversion Binary to Decimal



$$(1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2})$$

$$(8) + (0) + (2) + (0) + (0) + (0.25)$$

$$(10.25)_{10}$$





Solve

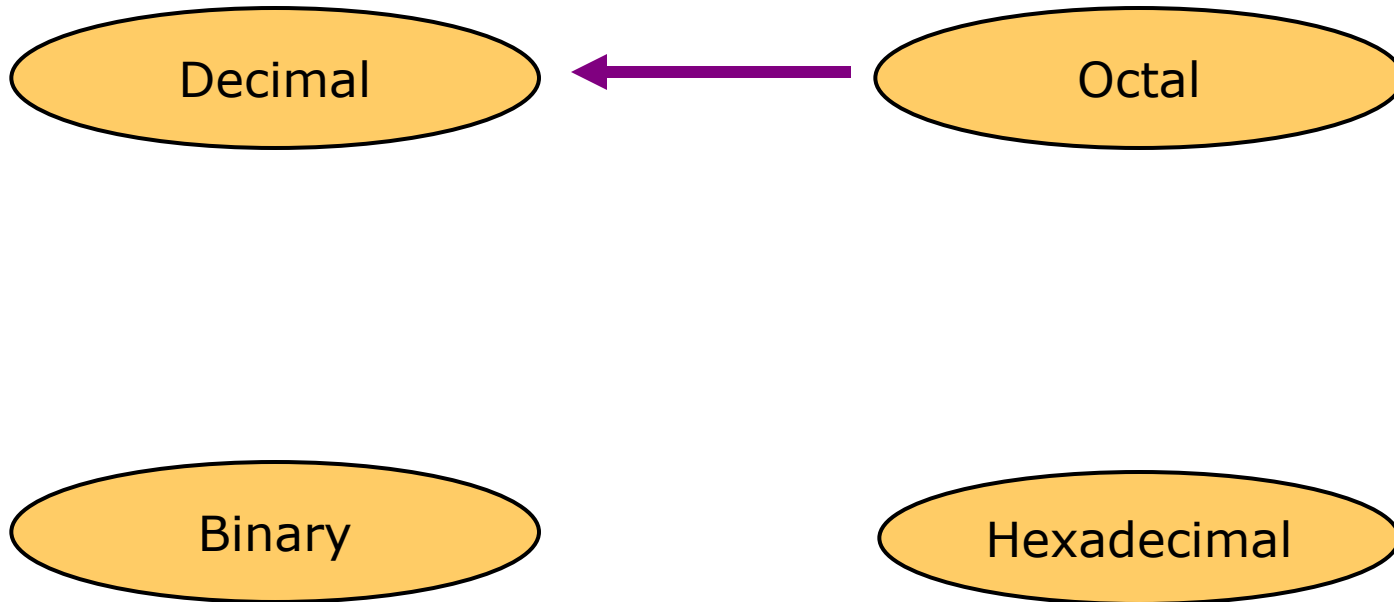
- Convert following Binary numbers into its Decimal equivalent.

1. $(1100010.0101)_2$

2. $(1011.01)_2$



Octal to Decimal





Conversion Octal to Decimal

1. Given Octal Number



Octal number

7

2

3

1

.

4

5



Position of Digits

8^3

8^2

8^1

8^0

8^{-1}

8^{-2}

2. Multiply each digit by its positional weight



$$(7 \times 8^3) + (2 \times 8^2) + (3 \times 8^1) + (1 \times 8^0) + (4 \times 8^{-1}) + (5 \times 8^{-2})$$

$$(7 \times 512) + (2 \times 64) + (3 \times 8) + (1 \times 1) + (4 \times 0.125) + (5 \times 0.015625)$$

3. Final answer



$$(7231.45)_8 = (3737.5782625)_{10}$$





Solve

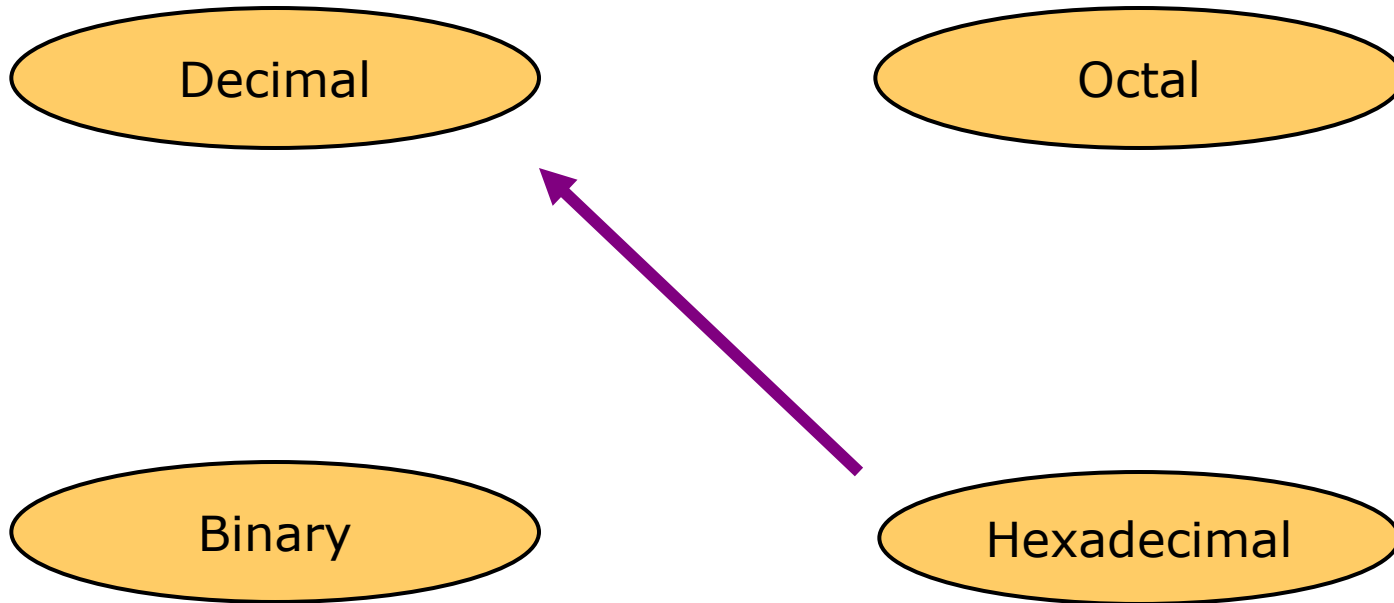
- Convert following Octal numbers into its Decimal equivalent

1. $(753.41)_8$

2. $(123.76)_8$



Hexadecimal to Decimal





Conversion Hexadecimal to Decimal

Hexadecimal number	7	A	3	D	.	4	C
	↓	↓	↓	↓		↓	↓
Position of Digits	16^3	16^2	16^1	16^0		16^{-1}	16^{-2}

$$(7 \times 16^3) + (A \times 16^2) + (3 \times 16^1) + (D \times 16^0) + (4 \times 16^{-1}) + (C \times 16^{-2})$$

$$(7 \times 4096) + (A \times 256) + (3 \times 16) + (D \times 1) + (4 \times 0.0625) + (C \times 0.0039)$$

$$(31293.25)_{10}$$

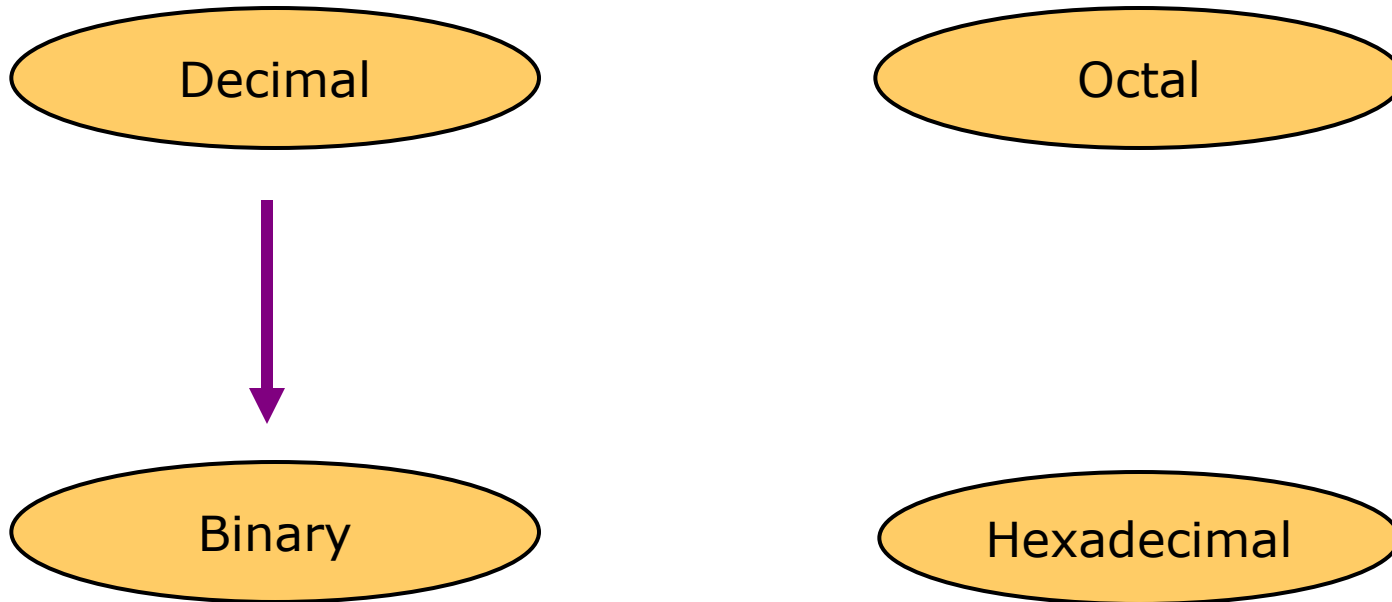


Solve

- Convert following Hexadecimal numbers to Decimal
 1. $(892.41)_{16}$
 2. $(A234.CD)_{16}$



Decimal to Binary



Conversion from Decimal into other system

D=Divide

- Technique
 - 1) Divide the decimal number by the base of other system
 - 2) Continue to divide the quotient by the base until there is nothing left
 - 3) List the remainder values in reverse order from bottom to top to find the equivalent.

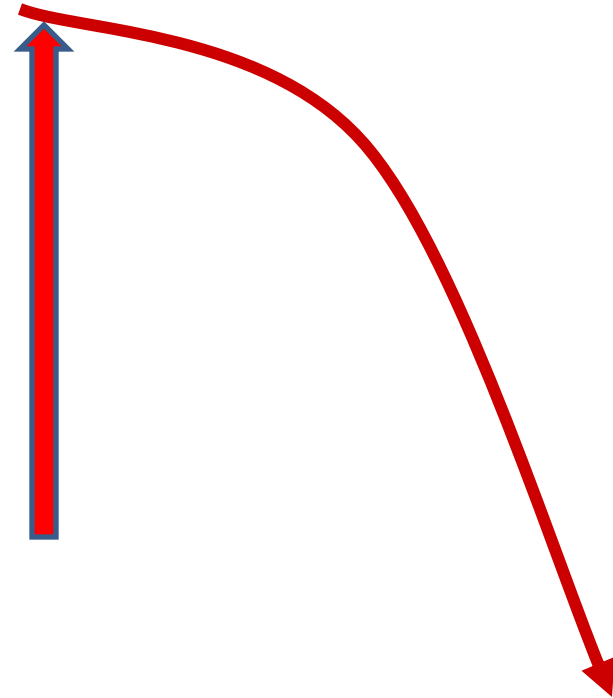
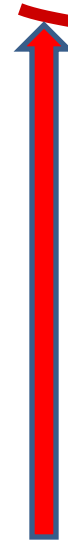


Decimal to Binary

Divide by two, keep track of the remainder

$$125_{10} = ?_2$$

2	125	
	62	1
2	31	0
	15	1
2	7	1
	3	1
2	1	1
	0	1



$$125_{10} = 1111101_2$$





Fractional decimal to binary

$$(0.42)_{10} = (?)_2$$

$$0.42 \times 2 = 0.84$$

$$0.84 \times 2 = 1.68$$

$$0.68 \times 2 = 1.36$$

$$0.36 \times 2 = 0.72$$

$$0.72 \times 2 = 1.44$$

0

1

1

0

1



$$(0.42)_{10} = (0.01101)_2$$



We could have continued further. But the conversion is generally carried out only upto 5 digits.





Solve

- Convert following decimal numbers to binary

1. $(526.42)_{10} = (?)_2$

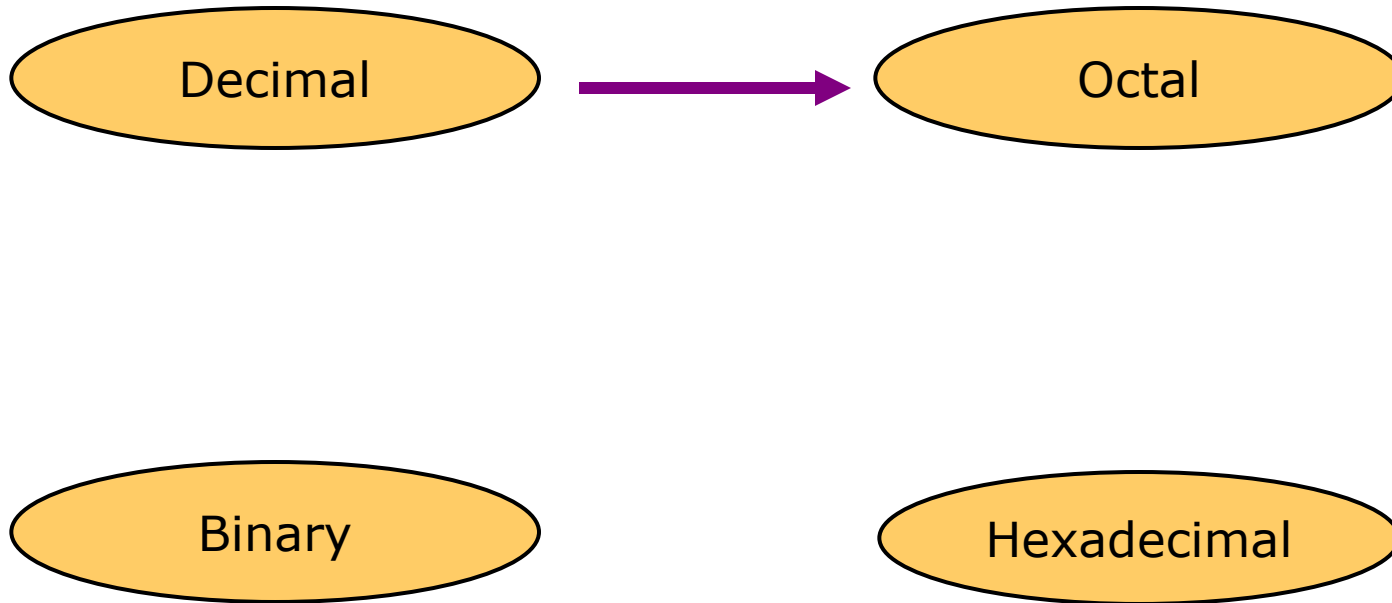
2. $(1234.78)_{10} = (?)_2$

3. $(9999.99)_{10} = (?)_2$

4. $(4926.25)_{10} = (?)_2$



Decimal to Octal



Technique

Divide by 8

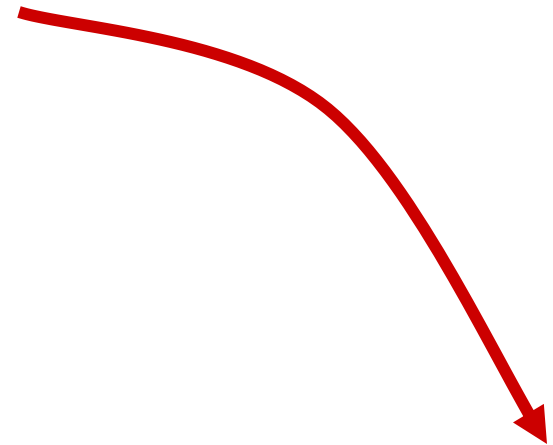
Keep track of the remainder



Example

$$1234_{10} = ?_8$$

8		1234	
8		154	2
8		19	2
8		2	3
8		0	2



$$1234_{10} = 2322_8$$





Fractional decimal to octal

$$(0.6234)_{10} = (?)_8$$

$$0.6234 \times 8 = 4.9872$$

$$0.9872 \times 8 = 7.8976$$

$$0.8976 \times 8 = 7.1808$$

$$0.1808 \times 8 = 1.4464$$

$$0.4464 \times 8 = 3.5712$$

4

7

7

1

3



$$(0.6234)_{10} = (0.47713)_8$$



Solve

- Convert following decimal numbers to octal.

1. $(753.41)_{10} = (?)_8$

2. $(123.76)_{10} = (?)_8$

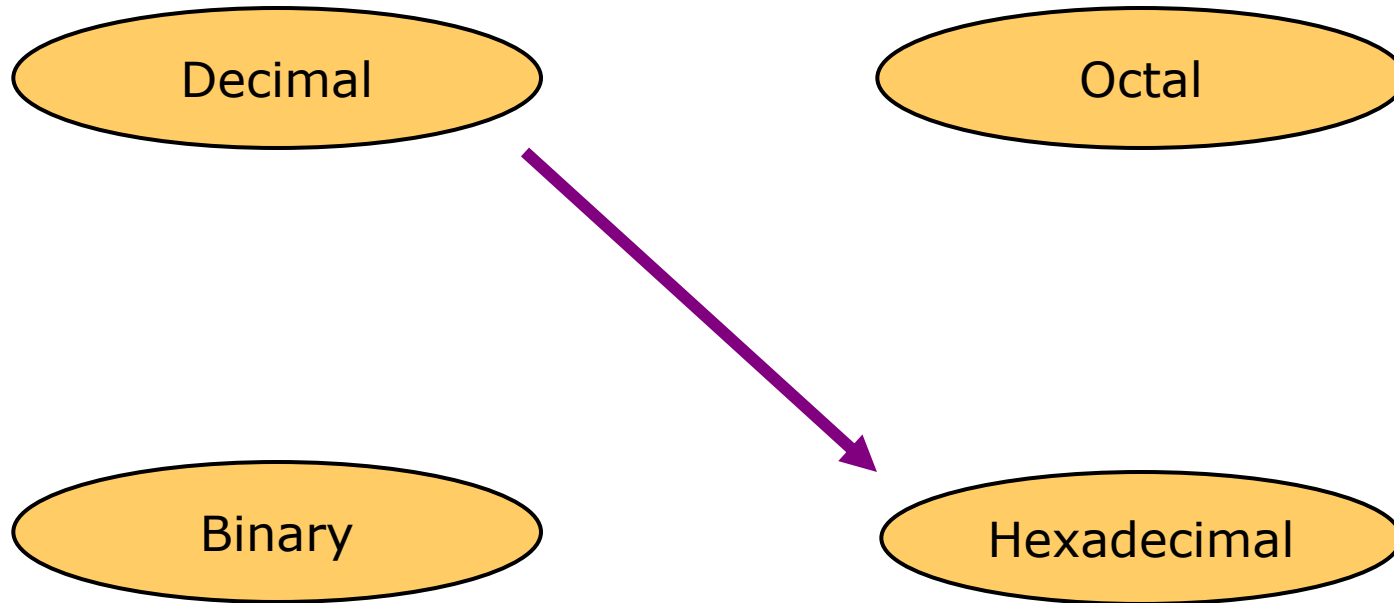
3. $(1234.78)_{10} = (?)_8$

4. $(9999.99)_{10} = (?)_8$

5. $(4926.25)_{10} = (?)_8$



Decimal to Hexadecimal



Technique

Divide by 16

Keep track of the remainder



Example

$$1234_{10} = ?_{16}$$

16		1234	
		77	2
16		4	13 = D
16		0	4

$$1234_{10} = 4D2_{16}$$



Fractional decimal to hexadecimal

$$(0.31)_{10} = (?)_2$$

$$0.31 \times 16 = 4.96$$

$$0.96 \times 16 = 15.36$$

$$0.36 \times 16 = 5.76$$

$$0.76 \times 16 = 12.16$$

$$0.16 \times 16 = 2.56$$

4

F

5

C

2



$$(0.31)_{10} = (0.4F5C2)_2$$



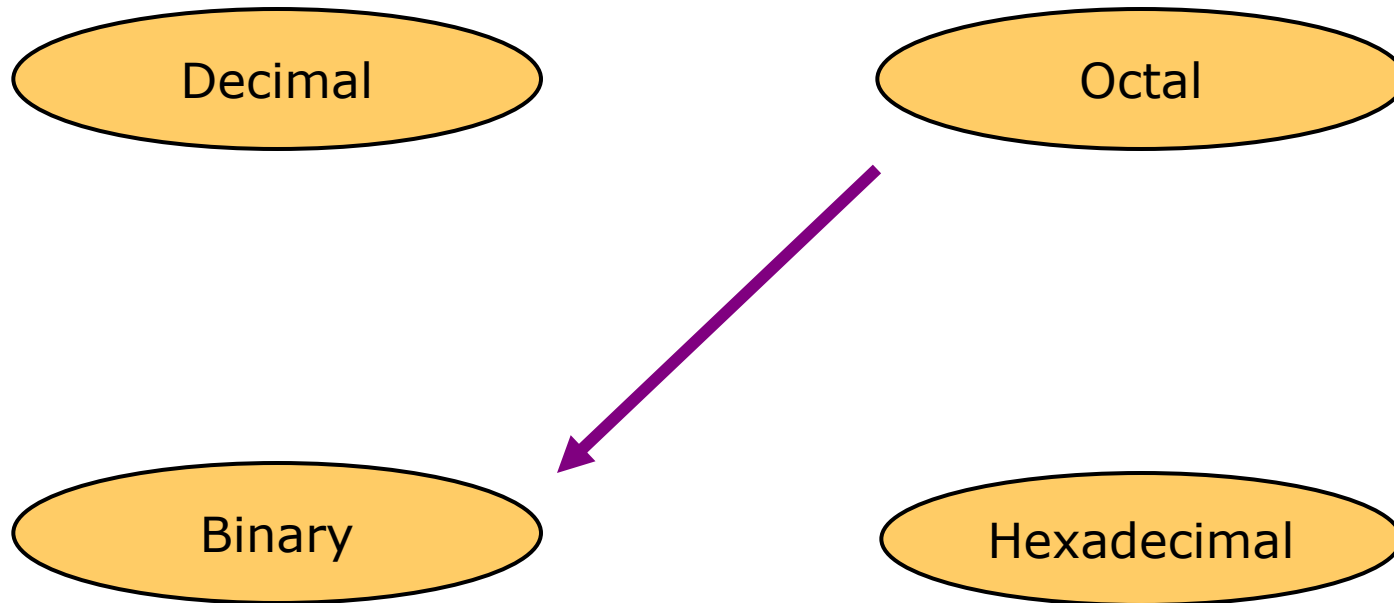
Solve

- Convert following decimal numbers to hexadecimal numbers.

1. $(753.41)_{10} = (?)_{16}$
2. $(123.76)_{10} = (?)_{16}$
3. $(1234.78)_{10} = (?)_{16}$
4. $(9999.99)_{10} = (?)_{16}$
5. $(4926.25)_{10} = (?)_{16}$



Octal to Binary



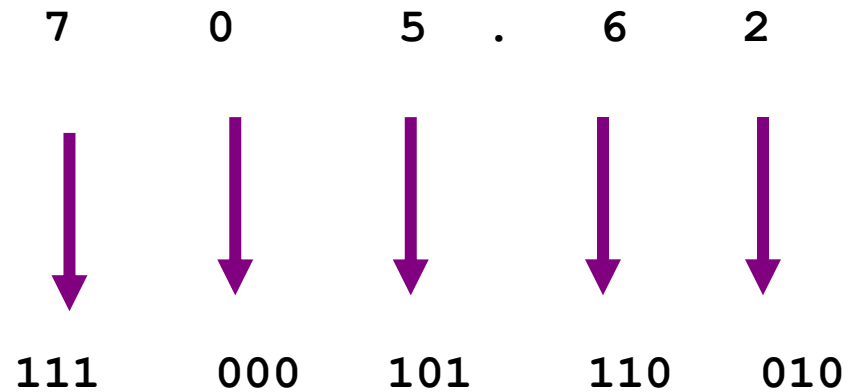
Octal to Binary

- Technique
 - Convert each octal digit to a 3-bit equivalent binary representation



Example

$$705.62_8 = ?_2$$



$$705_8 = 111000101.110010_2$$



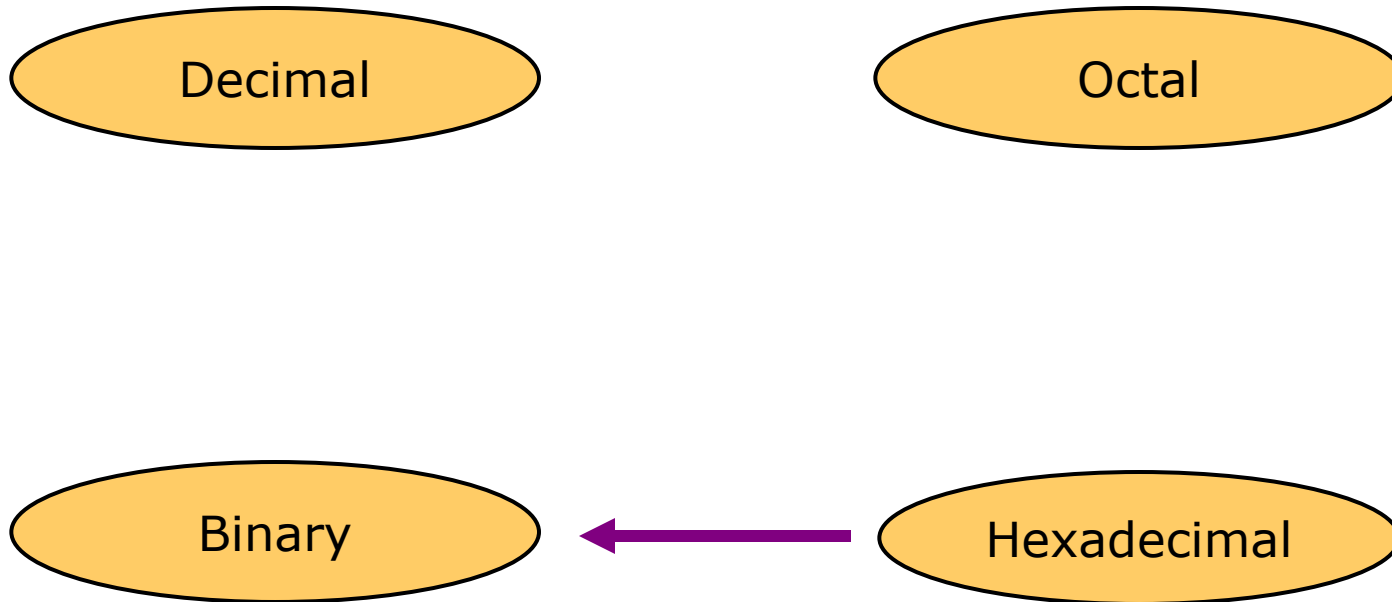


Solve

- $456.12_8 = ?_2$
- $125.37_8 = ?_2$



Hexadecimal to Binary



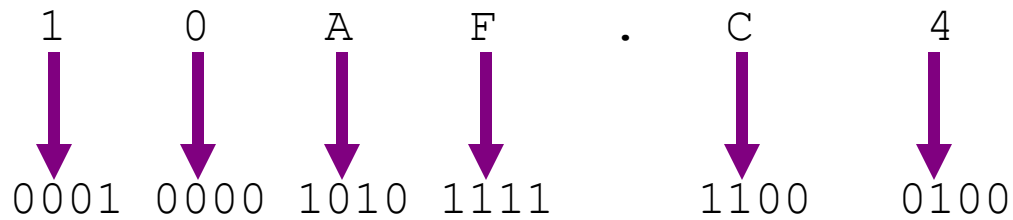
Hexadecimal to Binary

- Technique
 - Convert each hexadecimal digit to a 4-bit equivalent binary representation



Example

$$10AF.C4_{16} = ?_2$$



$$10AF.C4_{16} = 0001000010101111.11000100_2$$



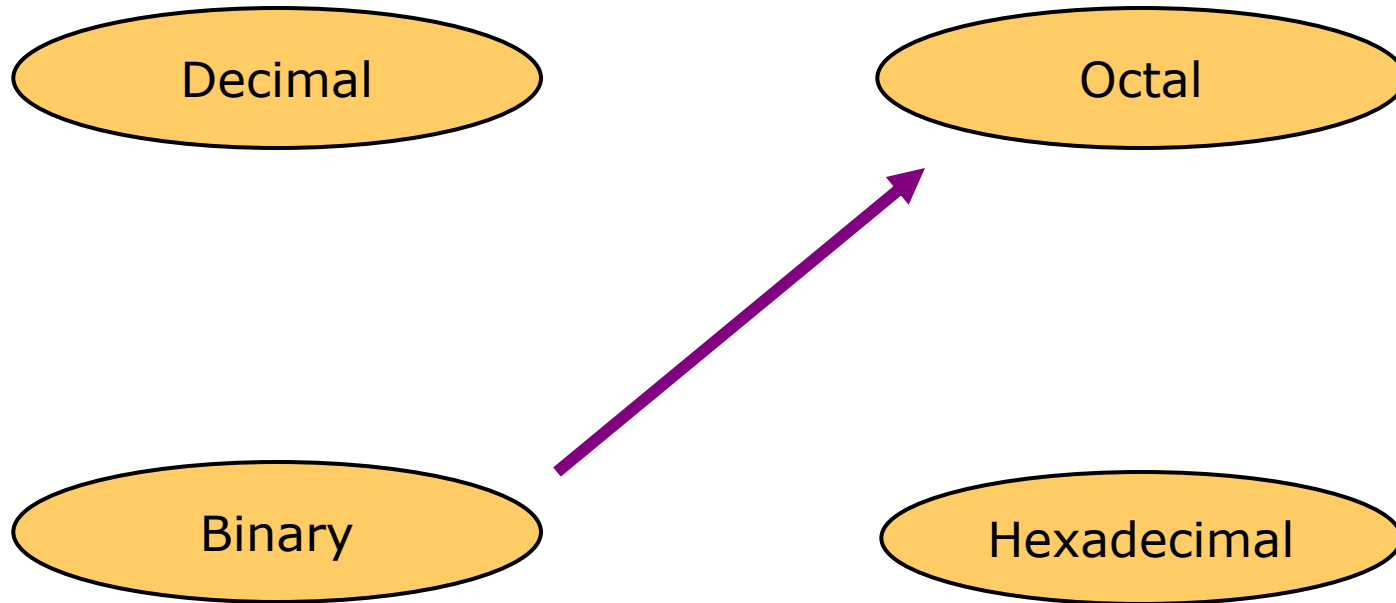


Solve

- $45A6.F2_{16} = ?_2$
- $1CD5.A7_{16} = ?_2$



Binary to Octal



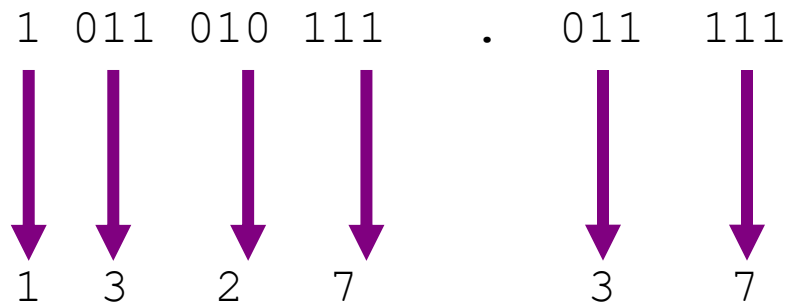
Binary to Octal

- Technique
 - Group bits in threes, starting on right
 - Convert to octal digits



Example

$$1011010111.011111_2 = ?_8$$



$$1011010111.011111_2 = 1327.37_8$$





Solve

1. $(1101)_2 = (?)_8$

2. $(10111)_2 = (?)_8$

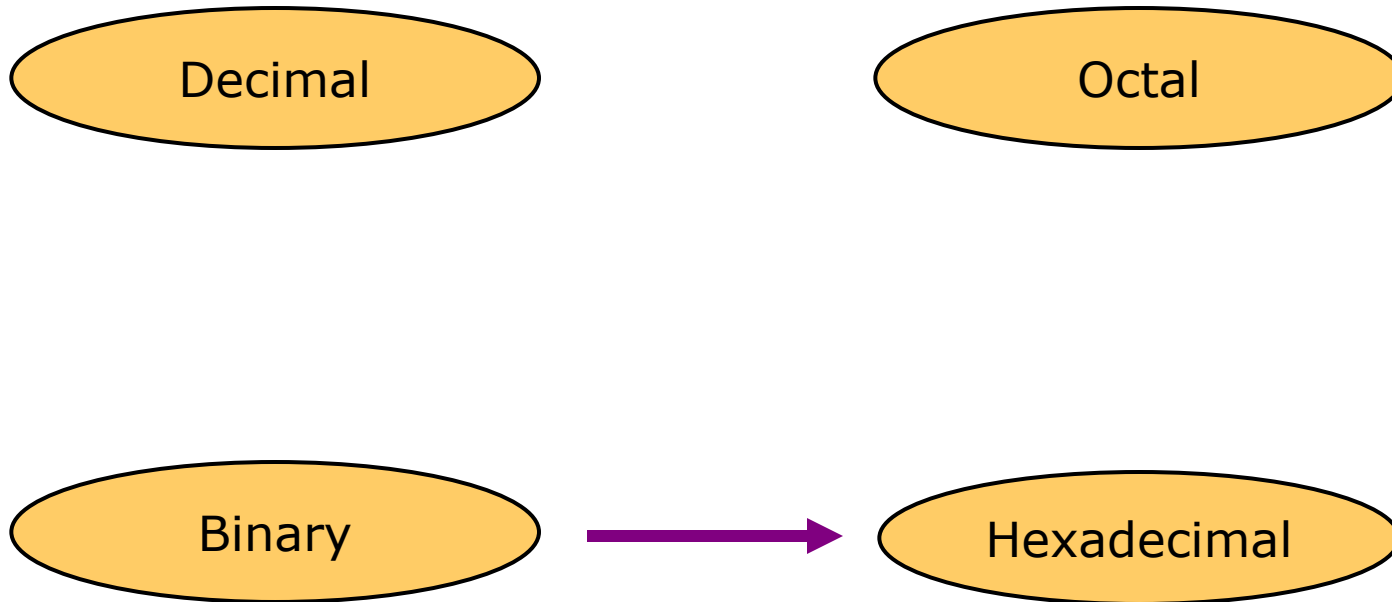
3. $(101101)_2 = (?)_8$

4. $(1011111)_2 = (?)_8$

5. $(101111101)_2 = (?)_8$



Binary to Hexadecimal



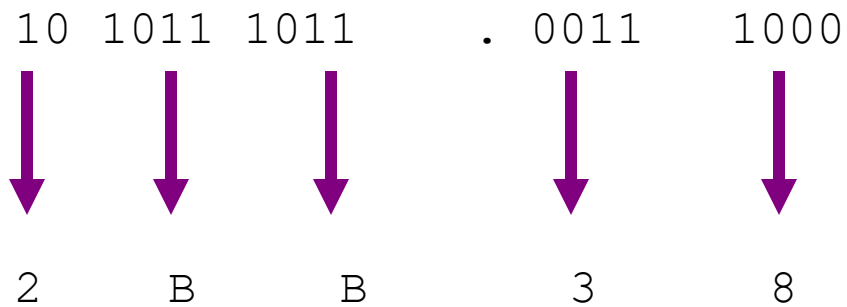
Binary to Hexadecimal

- Technique
 - Group bits in fours, starting on right
 - Convert to hexadecimal digits



Example

$$1010111011.00111_2 = ?_{16}$$



$$1010111011.00111_2 = 2BB . 38_{16}$$





Solve

1. $(1101)_2 = (?)_{16}$

2. $(10111)_2 = (?)_{16}$

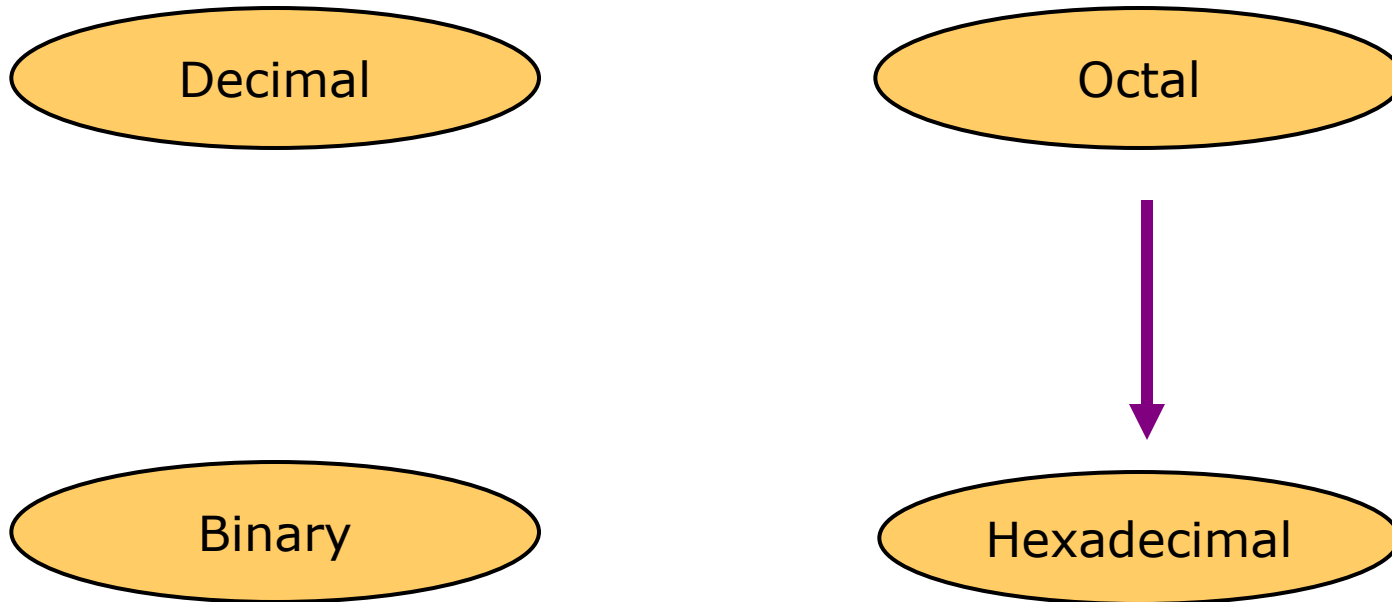
3. $(101101)_2 = (?)_{16}$

4. $(1011111)_2 = (?)_{16}$

5. $(101111101)_2 = (?)_{16}$



Octal to Hexadecimal



Octal to Hexadecimal

- Technique
 - Use binary as an intermediary



$$1076.4_8 = ?_{16}$$



Solve

1. $(765.43)_8 = (?)_{16}$

2. $(1241.567)_8 = (?)_{16}$

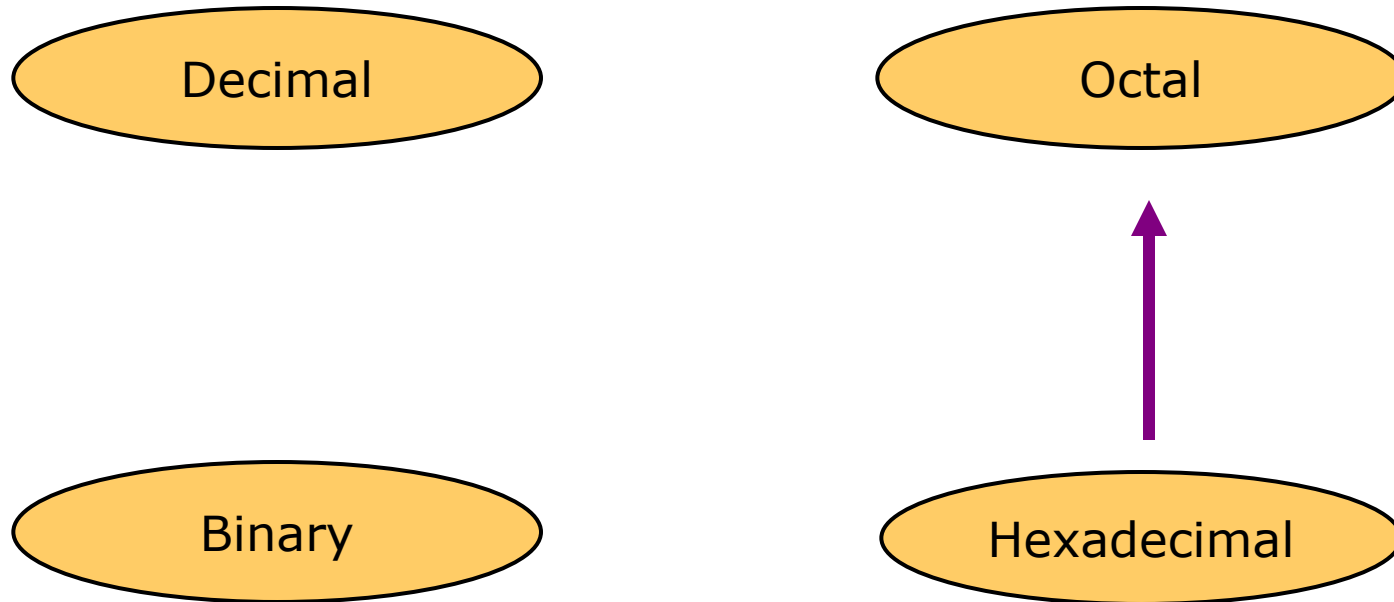
3. $(2354.63)_8 = (?)_{16}$

4. $(1726.54)_8 = (?)_{16}$

5. $(3465.7)_8 = (?)_{16}$



Hexadecimal to Octal



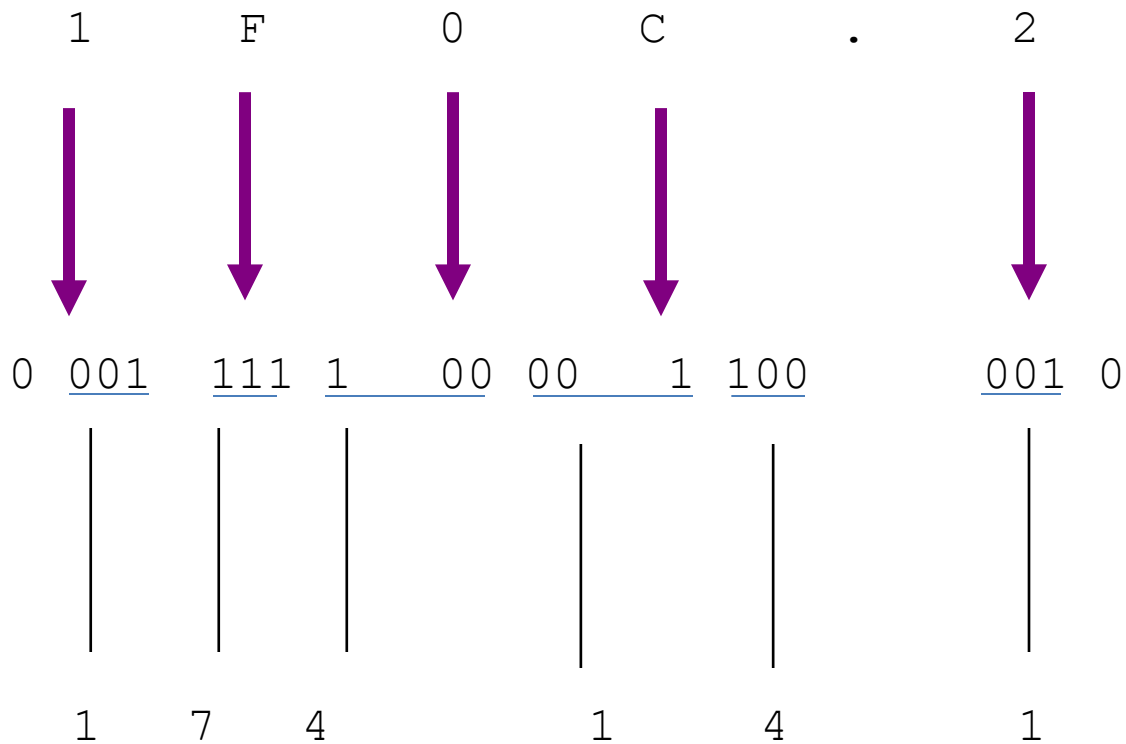
Hexadecimal to Octal

- Technique
 - Use binary as an intermediary



Example

$$1F0C.2_{16} = ?_8$$



$$1F0C.2_{16} = 17414.1_8$$





Solve

1. $(A25.45)_{16} = (?)_8$

2. $(1241.CD)_{16} = (?)_8$

3. $(2F4.6A)_{16} = (?)_8$

4. $(1E.54)_{16} = (?)_8$

5. $(346B.7)_{16} = (?)_8$



Exercise – Convert ...

Decimal	Binary	Octal	Hexa- decimal
33	100001	41	21
117	1110101	165	75
451	111000011	703	1C3
			1AF



Exercise – Convert ...

Decimal	Binary	Octal	Hexa- decimal
33	100001	41	21
117	1110101	165	75
451	111000011	703	1C3
431	110101111	657	1AF



Exercise – Convert ...

Decimal	Binary	Octal	Hexa- decimal
29.8			
	101.1101		
		3.07	
			C.82



Exercise – Convert ...

Decimal	Binary	Octal	Hexa- decimal
29.8	11101.110011...	35.63...	1D.CC...
5.8125	101.1101	5.64	5.D
3.109375	11.000111	3.07	3.1C
12.5078125	1100.10000010	14.404	C.82



Binary Arithmetic:

Binary Addition

- Four cases of binary addition:

Case	A	+	B	Sum	Carry
1	0	+	0	0	0
2	0	+	1	1	0
3	1	+	0	1	0
4	1	+	1	0	1



Addition of binary numbers

$$\begin{array}{r}
 1\ 0\ 0\ 1\ 0 \\
 +\quad 1\ 0\ 0\ 1 \\
 \hline
 1\ 1\ 0\ 1\ 1
 \end{array}$$

$$\begin{array}{r}
 0\ 1\ 1\ 1 \\
 00111\quad 7 \\
 10101\quad 21 \\
 \hline
 11100 = 28
 \end{array}$$

- Ex. 1: Add the following binary numbers.

$$A = (10111)_2 \quad \text{and} \quad B = (11001)_2$$

- Answer:

Step1: add LSBs (Least significant bits)

					1	Column of LSBs
A	1	0	1	1	1	LSB
B	1	1	0	0	1	LSB
					0	

Step 2: complete the addition

	1	1	1	1	1	
A	1	0	1	1	1	
B	1	1	0	0	1	
	1	1	0	0	0	0

Therefore the answer is $(110000)_2$





Examples

					Carry
	1	0	0	1	
+	0	1	1	0	
	1	1	1	1	Sum
	1	1	1		Carry
	1	1	0	1	
+	0	0	1	1	
1	0	0	0	0	Sum
	1	1			Carry
	0	1	1	0	
+	0	1	1	1	
	1	1	0	1	Sum



Solve

Add following numbers into binary numbering system.

1. $(12)_{10} + (8)_{10}$

2. $(15)_{10} + (10)_{10}$

3. $(35)_{10} + (48)_{10}$

4. $(10101)_2 + (10110)_2$

5. $(10111)_2 + (11000)_2$





Binary Subtraction

Binary subtraction:-

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0





Subtraction of large binary numbers

- Ex. 1: Subtract the following binary numbers.

$$A = (11001)_2 \quad \text{and} \quad B = (10111)_2$$

- Answer:

Step1: Subtract LSBs (Least significant bits)

						Column of LSBs
A	1	1	0	0	1	LSB
B	1	0	1	1	1	LSB
					0	



1's Complement of a number

- 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's.
- Ex: 1's complement of a number 10111 is = 01000
- Solve---- Find 1's complement of
 - a. 11010
 - b. 101101
 - c. 1010



2's Complement of a number

- The 2's complement of a number is obtained by adding 1 to the LSB of 1's complement of that number
- 2's complement = 1's complement + 1
- Ex: obtain 2's complement of a number $(10110010)_2$

Solution:

Given number =	1 0 1 1 0 0 1 0	
1's complement =	0 1 0 0 1 1 0 1	
Add 1 to LSB of 1's complement =	0 1 0 0 1 1 0 1	1
		1 carry
2's complement =	0 1 0 0 1 1 1 0	



Binary subtraction using 1's complement method

- To perform subtraction $(A)_2 - (B)_2$
 - ❖ Step 1: Convert number to be subtracted $(B)_2$ to its 1's complement.
 - ❖ Step 2: Add first number $(A)_2$ and 1's complement of $(B)_2$ using rules of binary addition.
 - ❖ Step 3: if final carry is 1 then add it to the result of addition obtained in step 2 to get final result.
 - **If final carry in step 2 is 1 then result obtained in step 2 is Positive and in its true form no conversion required.
 - ❖ Step 4: if final carry in step 2 is 0 then result obtained in step 2 is negative and in 1's complement form. So convert it to its true form.



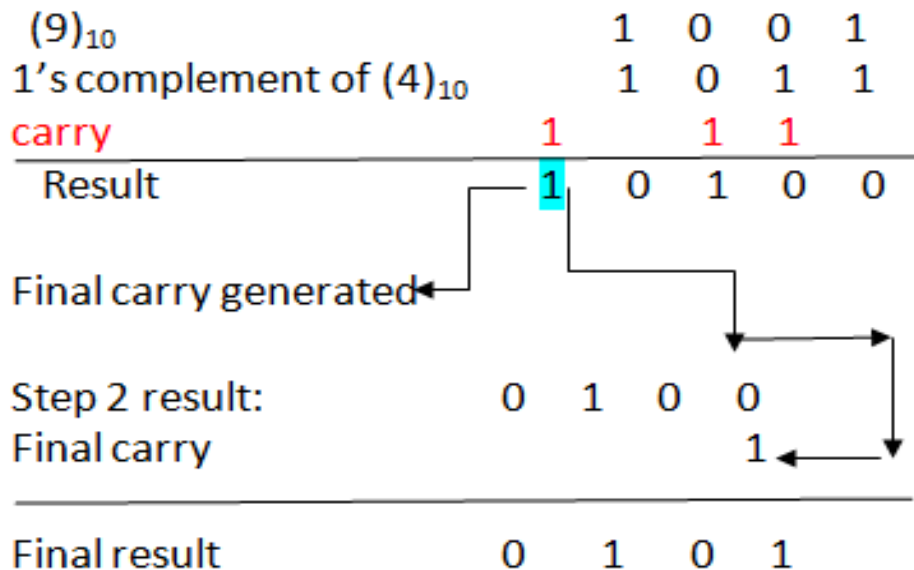
1's complement of B = $(1011)_2$

Step 1: take 1's complement of $(4)_{10}$

Step 2: Add $(9)_{10}$ and 1's complement of $(4)_{10}$

Step 3: add final carry to the result obtained in step 2.

$$\begin{aligned}(A)_{10} - (B)_{10} &= \\(9)_{10} - (4)_{10} &= \\(0101)_2 &= (5)_{10}\end{aligned}$$



Answer is positive and in its true form.



Perform $(4)_{10} - (9)_{10}$ using 1's complement method

$$A = (4)_{10} = (0100)_2 \quad B = (9)_{10} = (1001)_2$$

1's complement of B = $(0110)_2$

Step 1: take 1's complement of $(9)_{10}$

$(4)_{10}$	0	1	0	0	
1's complement of $(9)_{10}$	0	1	1	0	
carry		1			
Result	0	1	0	1	0

Step 2: Add $(4)_{10}$ and 1's complement of $(9)_{10}$

Final carry not generated

Step 2 result:	1	0	1	0
1's complement of step 2 result	0	1	0	1

Step 3: as final carry is 0 answer is negative and in 1's complement form

Final result - $(0 \ 1 \ 0 \ 1)_2$
Answer is Negative

$$\begin{aligned} (A)_{10} - (B)_{10} &= \\ (4)_{10} - (9)_{10} &= \\ -(0101)_2 &= \\ -(5)_{10} \end{aligned}$$



Solve

- Perform following subtraction using 1's complement method.
- $(11011)_2 - (1010)_2$ ans $(10001)_2$
- $(10111)_2 - (11000)_2$ ans $-(0001)_2$
- 1. $(54)_{10} - (33)_{10}$ ans $-(010101)_2$
- 2. $(33)_{10} - (54)_{10}$
- 3. $(99)_{10} - (22)_{10}$



Binary subtraction using 2's complement method

- To perform subtraction $(A)_2 - (B)_2$
 - ❖ Step 1: convert number to be subtracted $(B)_2$ to its 2's complement.
 - ❖ Step 2: Add first number $(A)_2$ and 2's complement of $(B)_2$ using rules of binary addition.
 - ❖ Step 3: if final carry is 1 then the result is Positive and in its true form no conversion required.
 - ❖ Step 4: if final carry in step 2 is 0 then result obtained in step 2 is negative and in 2's complement form. So convert it to its true form.
- ** Carry always be discarded.**



Perform $(9)_{10} - (4)_{10}$ using 2's complement method

$$A = (9)_{10} = (1001)_2 \quad B = (4)_{10} = (0100)_2$$

$$\begin{array}{r} \text{1's complement of } B = (1011)_2 \\ + \quad 1 \\ \hline \end{array}$$

$$\text{2's complement of } B = 1100$$

Step 1: take 2's complement of $(4)_{10}$

$$\begin{array}{r} (9)_{10} \quad \quad \quad 1 \quad 0 \quad 0 \quad 1 \\ \text{2's complement of } (4)_{10} \quad 1 \quad 1 \quad 0 \quad 0 \\ \text{carry} \quad \quad \quad 1 \\ \hline \text{Result} \quad \quad \quad 0 \quad 1 \quad 0 \quad 1 \end{array}$$

Step 2: Add $(9)_{10}$ and 2's complement of $(4)_{10}$

Final carry
Generated
Discard carry

$$\text{Final result} \quad (0 \quad 1 \quad 0 \quad 1)_2$$

Step 3: as final carry is 1 answer is positive and in true form.

$$\begin{aligned} (A)_{10} - (B)_{10} &= \\ (9)_{10} - (4)_{10} &= \\ (0101)_2 &= \\ (5)_{10} \end{aligned}$$



Perform $(4)_{10} - (9)_{10}$ using 2's complement method

$$A = (4)_{10} = (0100)_2 \quad B = (9)_{10} = (1001)_2$$

$$\begin{array}{r} 1\text{'s complement of } B = (0110)_2 \\ + \quad 1 \\ \hline \end{array}$$

$$2\text{'s complement of } B = 0111$$

Step 1: take 2's complement of $(9)_{10}$

$$\begin{array}{r} (4)_{10} \quad \quad \quad 0 \quad 1 \quad 0 \quad 0 \\ 2\text{'s complement of } (9)_{10} \quad 0 \quad 1 \quad \underline{1} \quad \underline{1} \\ \text{carry} \quad \quad \quad 1 \\ \hline \text{Result} \quad \quad \quad 0 \quad 1 \quad 0 \quad 1 \quad \underline{1} \end{array}$$

Step 2: Add $(4)_{10}$ and 2's complement of $(9)_{10}$

Final carry not generated

$$\begin{array}{r} \text{Step 2 result:} \quad \quad 1 \quad 0 \quad 1 \quad \underline{1} \\ 1\text{'s complement of} \\ \text{step 2 result:} \quad \quad 0 \quad 1 \quad 0 \quad 0 \\ 2\text{'s complement of} \\ \text{step 2 result:} \quad \quad 0 \quad 1 \quad 0 \quad 1 \end{array}$$

Step 3: as final carry is 0 answer is negative and in 2's complement form

$$\begin{aligned} (A)_{10} - (B)_{10} &= \\ (4)_{10} - (9)_{10} &= \\ -(0101)_2 &= \\ -(5)_{10} & \end{aligned}$$

$$\begin{array}{r} \text{Final result} \quad \quad - \quad (0 \quad 1 \quad 0 \quad 1)_2 \\ \text{Answer is Negative} \end{array}$$



Solve

- Perform following subtraction using 2's complement method.

1. $(48)_{10} - (61)_{10}$

2. $(33)_{10} - (54)_{10}$

3. $(99)_{10} - (22)_{10}$

4. $(1010)_2 - (101)_2$



Multiplication

- Binary, two 1-bit values

A	B	$A \times B$
0	0	0
0	1	0
1	0	0
1	1	1



Multiplication (3 of 3)

- Binary, two n -bit values
 - As with decimal values
 - E.g.,

$$\begin{array}{r} 1110 \\ \times 1011 \\ \hline 1110 \\ 1110 \\ 0000 \\ 1110 \\ \hline 10011010 \end{array}$$



Solve

- $(205)_{10} \times (3)_{10}$
- $(1110101)_2 \times (1001)_2$
- $(110)_2 \times (10)_2$
- $(1111101)_2 \times (101)_2$
- $(15)_{10} \times (8)_{10}$





Binary division

Ex: $(25)_{10} \div (5)_{10}$

$$(25)_{10} = (11001)_2$$
$$(5)_{10} = (101)_2$$

$$\begin{array}{r} 101 \\ 101 \overline{) 11001} \\ \underline{101} \\ 00101 \\ \underline{101} \\ 00101 \\ \underline{101} \\ 00000 \end{array}$$





Solve

- $(205)_{10} \div (3)_{10}$
- $(1110101)_2 \div (1001)_2$
- $(110)_2 \div (10)_2$
- $(1111101)_2 \div (101)_2$



BCD Code:

- BCD code is an abbreviation for Binary coded Decimal codes. It is a numeric weighted code, in which each digit of a decimal number is represented by a separate group of 4-bits.
- There are several BCD codes like 8421, 2421, 3321, 4221, 5211, 5311, 5421, etc.
- The most common and widely used BCD code is 8421 code.
- In 8421 code, the weights associated with 4 bits are 8, 4, 2, 1 from MSB to LSB.
- That is, the weight associated with 3rd bit is 8, the weight associated with 2nd bit is 4, the weight associated with 1st bit is 2 and the weight associated with 0th bit is 1.



The following table shows the 8421 code for 0-9 decimal numbers.

Decimal code	BCD Code			
	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

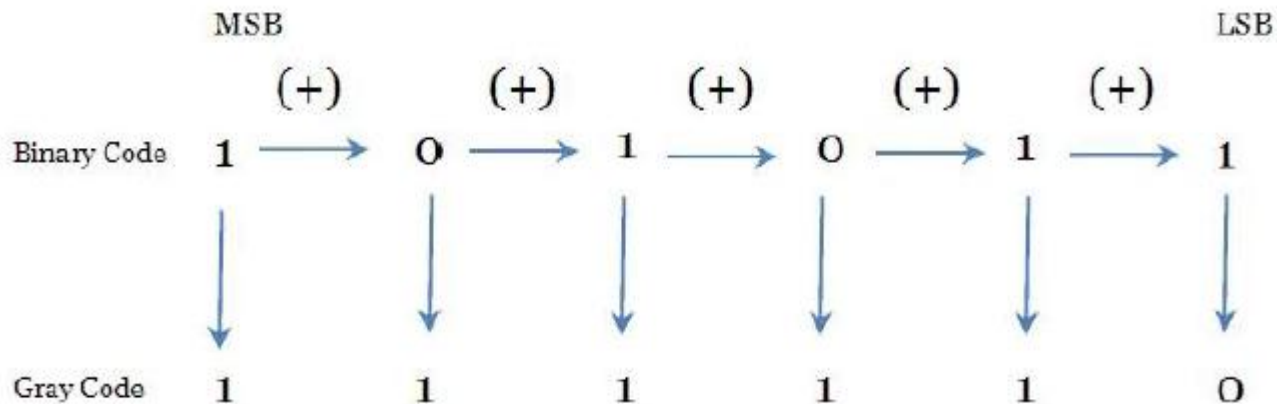


THE GRAY CODE:

BINARY-TO-GRAY CODE CONVERSION

- The MSB in the Gray code is the same as corresponding MSB in the binary number.
- Going from left to right, add each adjacent pair of binary code bits to get the next Gray code bit. Discard carries.

Ex: Convert $(101011)_2$ to Gray code

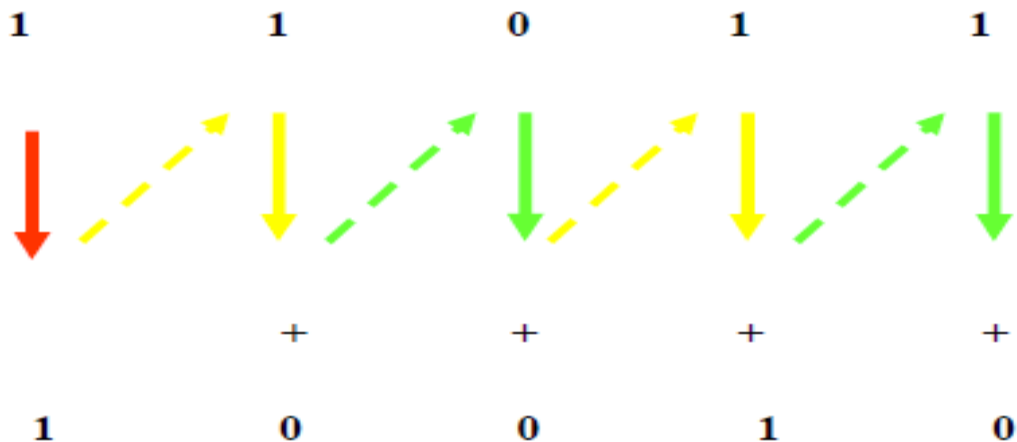


GRAY-TO-BINARY CONVERSION

- The MSB in the binary code is the same as the corresponding bit in the Gray code.
- Add each binary code bit generated to the Gray code bit in the next adjacent position.

Discard carries.

Ex: Convert the Gray code word 11011 to binary



APPLICATION OF GRAY CODE:

The gray code is used in a few specific applications.

- The main applications include being used in analog to digital converters, as well as being used for error correction in digital communication.
- Gray code is used to minimize errors in converting analog signals to digital signals.

ADVANTAGES OF GRAY CODE:

- Better for error minimization in converting analog signals to digital signals.
- Reduces the occurrence of “Hamming Walls” (an undesirable state) when used in genetic algorithms.
- Can be used to in to minimize a logic circuit.
- Useful in clock domain crossing.

DISADVANTAGES OF GRAY CODE:

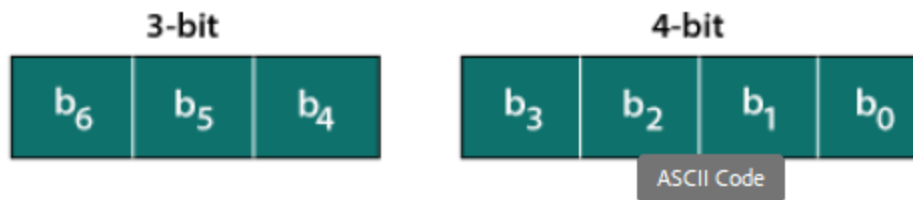
- Not suitable for arithmetic operations.
- Limited practical use outside of a few specific applications.



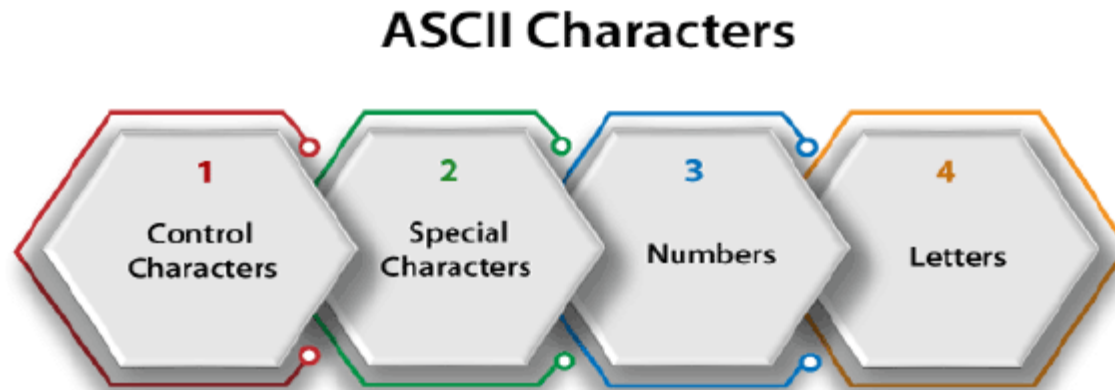
ASCII CODE

- The ASCII stands for American Standard Code for Information Interchange.
- The ASCII code is an alphanumeric code used for data communication in digital computers.
- The ASCII is a 7-bit code capable of representing 27 or 128 number of different characters.
- The ASCII code is made up of a three-bit group, which is followed by a four-bit code.
- The ASCII Code is a 7 or 8-bit alphanumeric code.
- This code can represent 127 unique characters.
- The ASCII code starts from 00h to 7Fh. In this, the code from 00h to 1Fh is used for control characters, and the code from 20h to 7Fh is used for graphic symbols.
- The 8-bit code holds ASCII, which supports 256 symbols where math and graphic symbols are added.
- The range of the extended ASCII is 80h to FFh.

Representation of ASCII Code



The ASCII characters are classified into the following groups:



1. CONTROL CHARACTERS

- The non-printable characters used for sending commands to the PC or printer are known as control characters.
- We can set tabs, and line breaks functionality by this code.
- The control characters are based on telex technology. Nowadays, it's not so much popular in use.
- The character from 0 to 31 and 127 comes under control characters.

2. SPECIAL CHARACTERS

- All printable characters that are neither numbers nor letters come under the special characters.
- These characters contain technical, punctuation, and mathematical characters with space also.
- The character from 32 to 47, 58 to 64, 91 to 96, and 123 to 126 comes under this category.

3. NUMBERS CHARACTERS

This category of ASCII code contains Ten Arabic numerals from 0 to 9.

4. LETTERS CHARACTERS

- In this category, two groups of letters are contained, i.e., the group of uppercase letters and the group of lowercase letters.
- The range from 65 to 90 and 97 to 122 comes under this category.





Example 1: (10010101100001111011011000011010100111000011011111101001 1101110 11101001000000011000101100100110011)2

Step 1: In the first step, we we make the groups of 7-bits because the ASCII code is 7 bit. 10010101100001 1110110 1100001 1010100 1110000 1101111 1101001 1101110 1110100 1000000 0110001 0110010 0110011

Step 2: Then, we find the equivalent decimal number of the binary digits either from the ASCII table or 64 32 16 8 4 2 1 scheme.

BINARY							DECIMAL
64	32	16	8	4	2	1	
1	0	0	1	0	1	0	64+8+2=74
64	32	16	8	4	2	1	
1	1	0	0	0	0	1	64+32+1=94
64	32	16	8	4	2	1	
1	1	1	0	1	1	0	64+32+16+4+2=118
64	32	16	8	4	2	1	
1	1	0	0	0	0	1	64+32+1=97
64	32	16	8	4	2	1	
1	0	1	0	1	0	0	64+16+4=84
64	32	16	8	4	2	1	
1	1	1	0	0	0	0	64+32+16=112
64	32	16	8	4	2	1	
1	1	0	1	1	1	1	64+32+8+4+2+1=111
64	32	16	8	4	2	1	
1	1	0	1	0	0	1	64+32+8+1=105
64	32	16	8	4	2	1	
1	1	0	1	1	1	0	64+32+8+4+2=110
64	32	16	8	4	2	1	
1	1	1	0	1	0	0	64+32+16+4=116
64	32	16	8	4	2	1	
1	0	0	0	0	0	0	64
64	32	16	8	4	2	1	
0	1	1	0	0	0	1	32+16+1=49
64	32	16	8	4	2	1	
0	1	1	0	0	1	0	32+16+2=50
64	32	16	8	4	2	1	
0	1	1	0	0	1	1	32+16+2+1=51



Step 3: Last, we find the equivalent symbol of the decimal number from the ASCII table.

S.NO.	DECIMAL	SYMBOL	S.NO.	DECIMAL	SYMBOL
1	74	J	8	105	i
2	94	a	9	110	n
3	118	v	10	116	t
4	97	a	11	64	@
5	84	T	12	49	1
6	112	p	13	50	2
7	111	o	14	51	3



Thanks

Unit-3 Syllabus

Combinational logic and arithmetic circuit 6 6

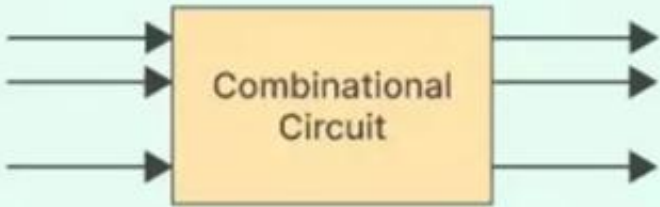
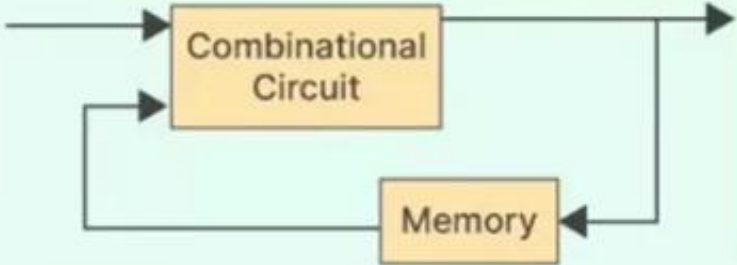
Encoders and Decoders, Multiplexers and Demultiplexers, Adder, subtractors, comparators, introduction to Programmable Logic Devices (PLDs) like FPGA, Overview of IC classification.



Classification of Digital Circuits

The digital systems in general are classified into two categories namely:

1. **Combinational logic circuits**
2. **Sequential logic circuits.**

Combinational Circuit	Sequential Circuit
Output only depends on the present input	Output depends on present input & past output
Memory element is absent	Memory element is present
No clock signal is applied	Clock signal is required
	
Example - Half Adder, Full Adder, Multiplexer	Example - Flipflop, Counters, Registers

Combinational Circuits

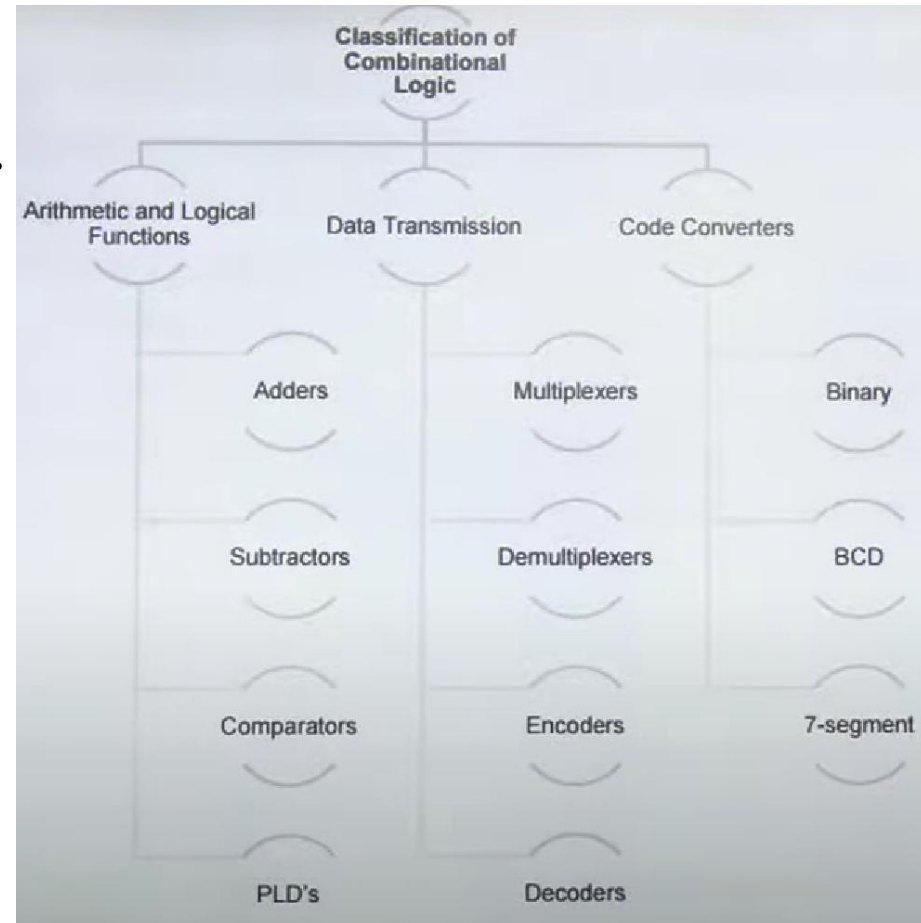
1. Identify number of Input & Output .
2. Construct Truth Table.
3. Write Logical Expression in SOP and POS Form.
4. Minimize Logical Expression.
5. Implement Logic Circuits.



Examples of Combinational Circuits

Following are the examples of some combinational circuits:

- **Half Adder, Full Adder**
- **Half Subtractor, Full Subtractor**
- **Multiplexers, Demultiplexers**
- **Encoders, Decoders**
- **Comparator & Code Converters**



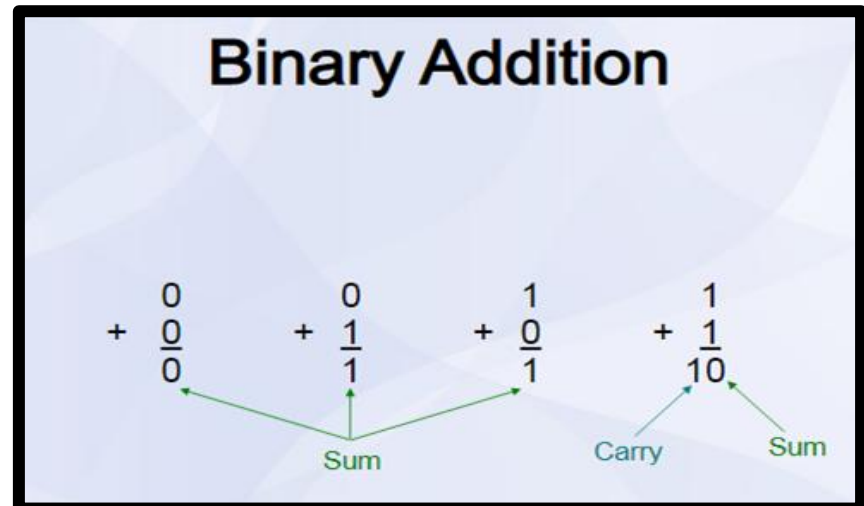
Adder

An **Adder** is a **combinational circuit** in digital electronics that performs **binary addition**.

- It takes two or more binary numbers as input.
- Produces the **sum** and sometimes a **carry** as output.
- Adders are the basic building blocks in **ALU (Arithmetic Logic Unit)**, processors, and calculators.

Types of Adders

- 1) **Half Adder (HA)**
- 2) **Full Adder (FA)**



Half Adder (HA)

A **Half Adder** is the **simplest combinational circuit** in digital electronics that adds **two single-bit binary numbers**.

It has:

- **Inputs** → 2 (A, B)
- **Outputs** → 2 (Sum, Carry)





HA-Truth Table

Step-1: Identify the Input and Output Variables

- Input variables = A, B (either 0 or 1)
- Output variables = S, C where S = Sum and C = Carry

Step-2: Draw the Truth Table

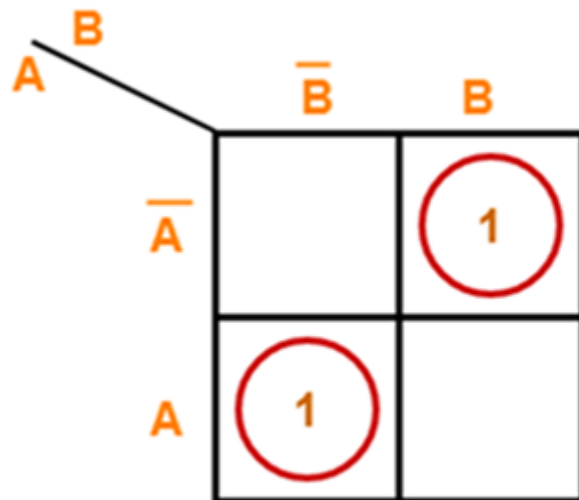
Inputs		Outputs	
X	Y	Carry (C)	Sum (S)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



HA-K Maps

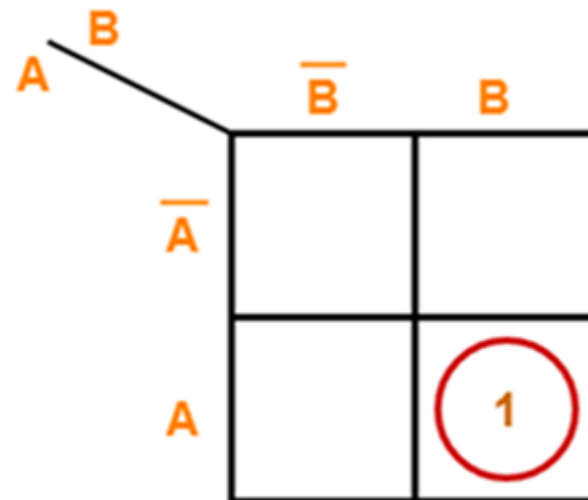
Step-3: Draw K-Map using Truth Table & determine the simplified Boolean Expression

For S:



$$S = A \oplus B$$

For C:



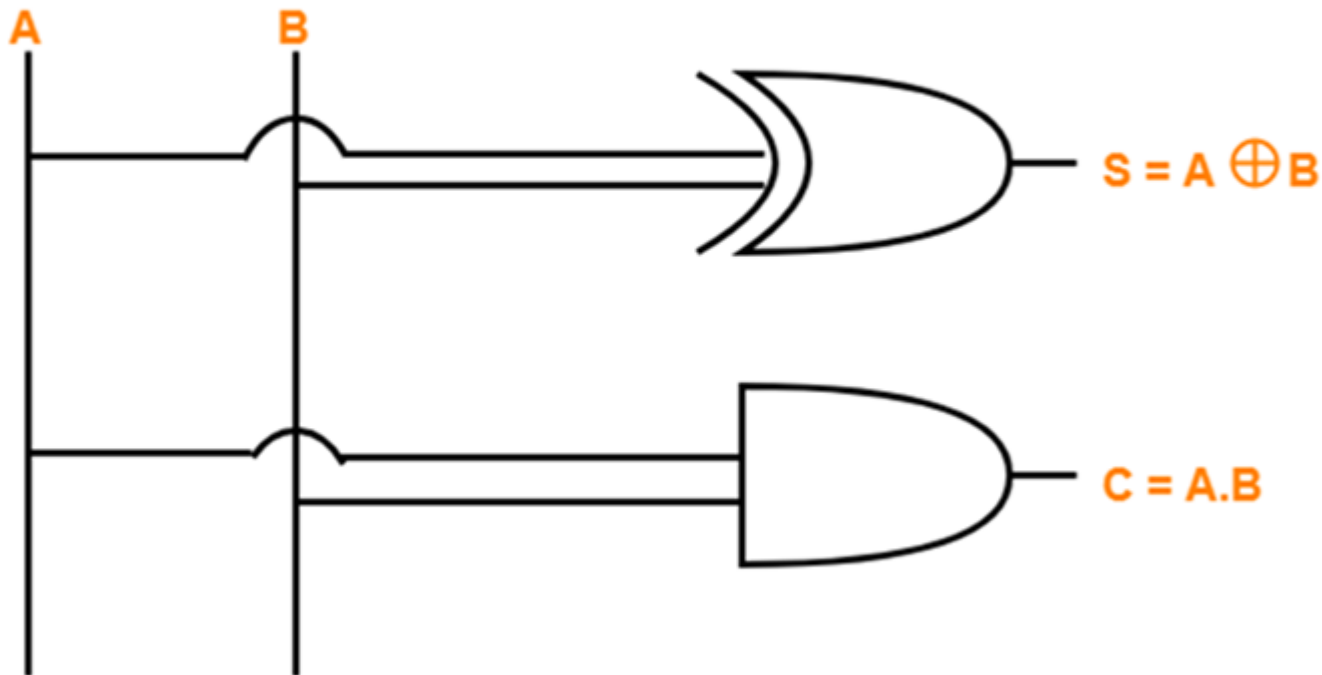
$$C = A . B$$

HA-Logic Diagram

Step-04:

Draw the logic diagram.

The implementation of half adder using 1 XOR gate and 1 AND gate is as shown below-



Half Adder Logic Diagram

HA-Limitations

Limitation of Half Adder

- It cannot perform addition when there is an **input carry** from a previous stage.
- That's why we need a **Full Adder** for multi-bit addition.

✓ In short:

- A **Half Adder** adds two binary inputs (A, B).
- Outputs: **Sum** = $A \oplus B$, **Carry** = $A \cdot B$.
- Built using **XOR and AND gates**.
- Useful for simple 1-bit addition but **cannot handle carry input**.

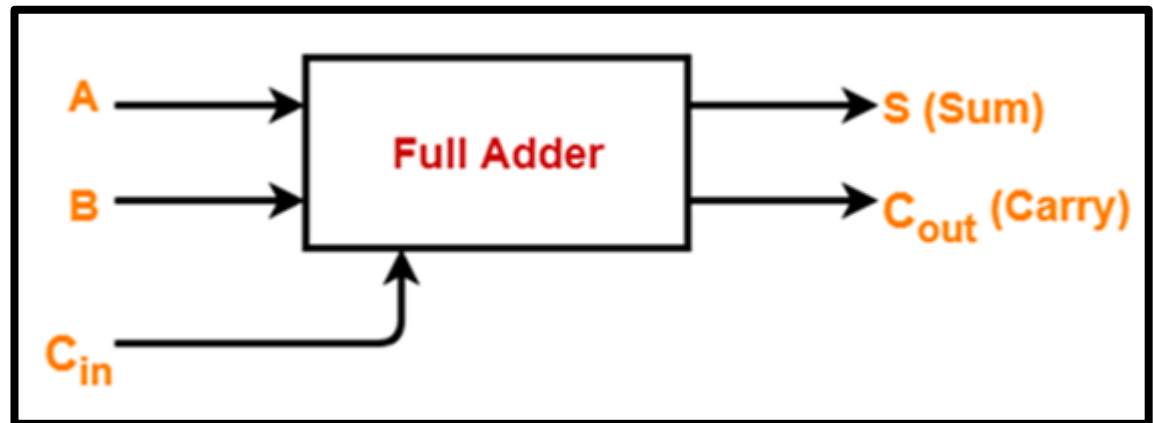
Full Adder (FA)

A **Full Adder** is a **combinational logic circuit** that performs the **addition of three input bits**:

- **A** → First input
- **B** → Second input
- **C_{in}** → Carry input (from a previous addition stage)

It produces two outputs:

- **Sum (S)**
- **Carry-out (C_{out})**



👉 Unlike the **Half Adder**, the **Full Adder** can handle carry input, making it suitable for **multi-bit addition**.



FA-Truth Table

Step-1: Identify the Input and Output Variables

- Input variables = A, B, C_{in} (either 0 or 1)
- Output variables = S, C_{out} where S = Sum and C_{out} = Carry

Step-2: Draw the Truth Table

Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



FA-K Map

Step-3: Draw K-Map using Truth Table & determine the simplified Boolean Expression

For S:

A	BC _{in}			
	$\overline{B}\overline{C}_{in}$	$\overline{B}C_{in}$	BC_{in}	$B\overline{C}_{in}$
$\overline{A}$		1		1
A	1		1	

$$S = A \oplus B \oplus C_{in}$$

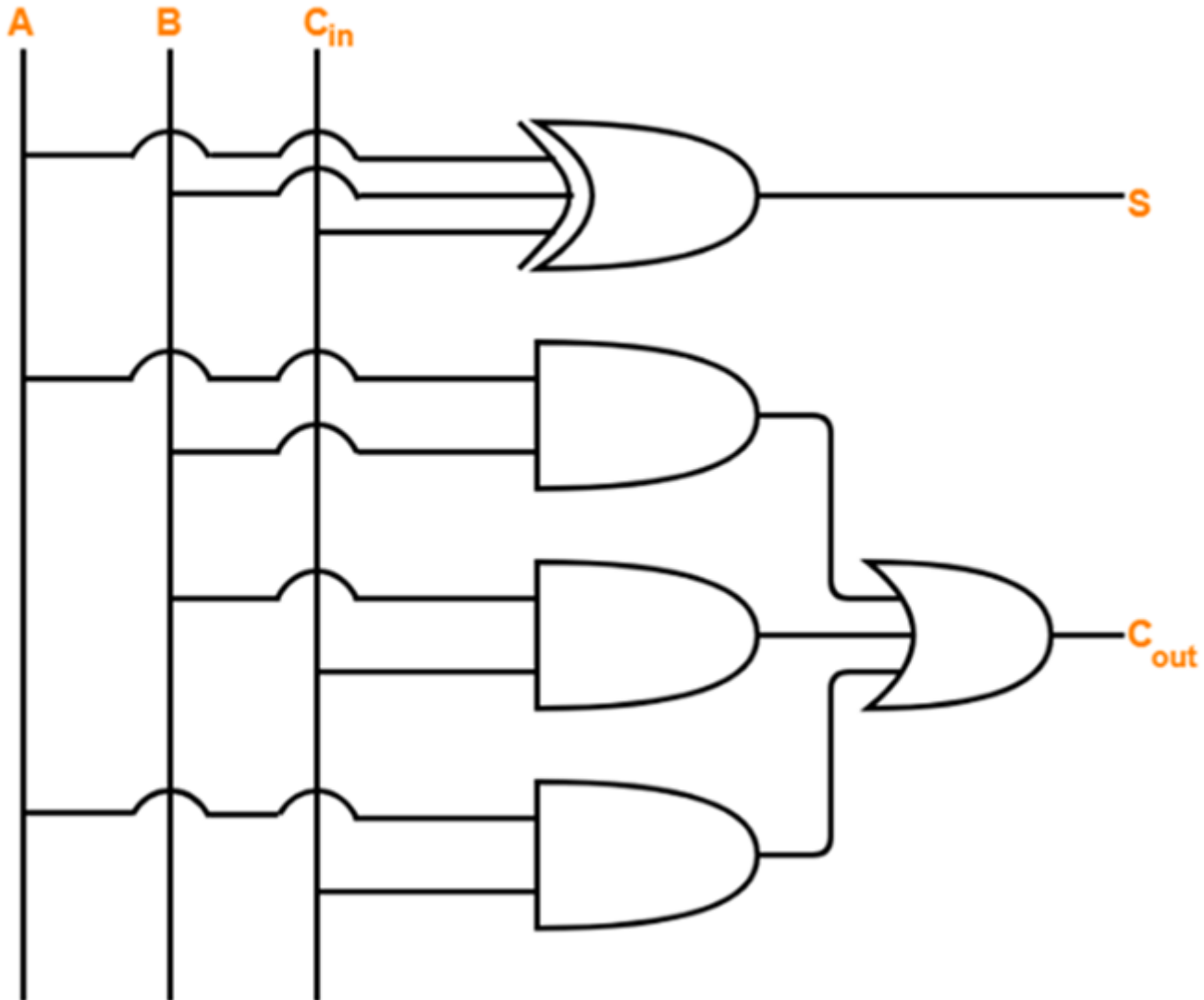
For C_{in}:

A	BC _{in}			
	$\overline{B}\overline{C}_{in}$	$\overline{B}C_{in}$	BC_{in}	$B\overline{C}_{in}$
$\overline{A}$			1	
A		1	1	1

$$C_{out} = AB + BC_{in} + C_{in}A$$

FA-Logic Diagram

Full Adder Logic Diagram



Cont...

Advantages

- Supports carry input.
- Building block for multi-bit adders.
- Widely used in ALUs, CPUs.
- Can be built using simple Half Adders.

Limitations

- Slower for large bit operations (carry delay).
- More hardware complexity than Half Adder.
- Limited to binary addition.

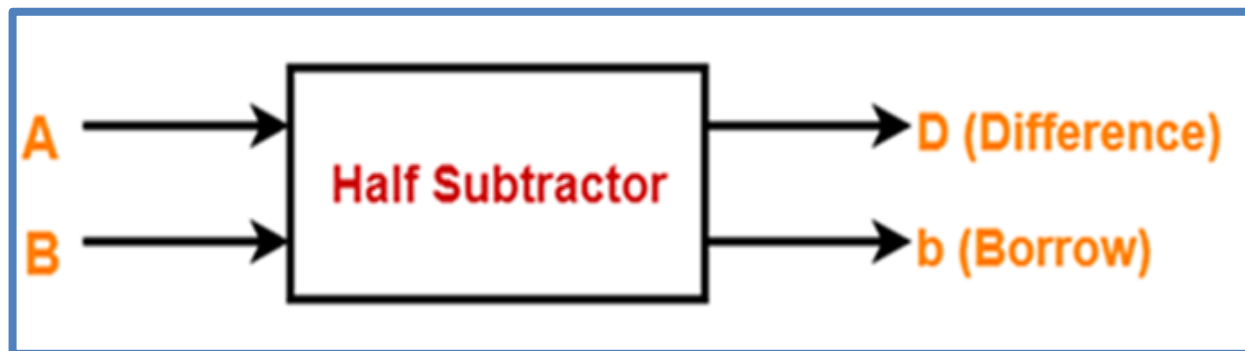
Half Subtractor (HS)

A **Half Subtractor** is a combinational logic circuit used to perform the subtraction of **two binary digits**.

It has **two inputs** and **two outputs**:

- Inputs → **A (minuend)** and **B (subtrahend)**
- Outputs → **Difference (D)** and **Borrow (B)**

It is called "**half**" because it cannot handle borrow input from a previous stage.



HS-Truth Table

Step-1: Identify the Input and Output Variables

- Input variables = A, B (either 0 or 1)
- Output variables = D, b where D = Difference and b = borrow

Step-2: Draw the Truth Table

Inputs		Outputs	
A	B	D (Difference)	b (Borrow)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

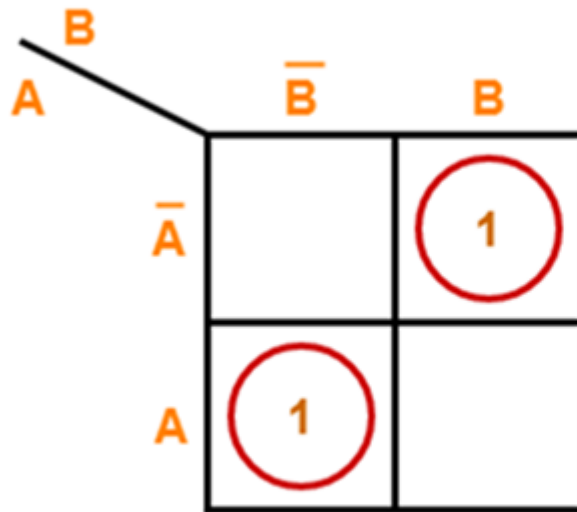
Truth Table



HS-K Map

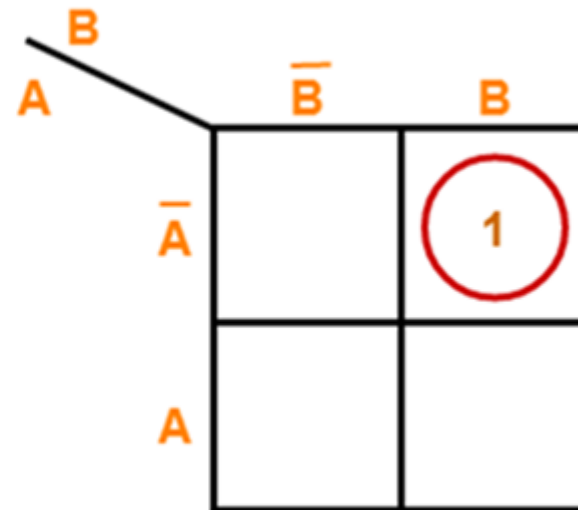
Step-3: Draw K-Map using Truth Table & determine the simplified Boolean Expression

For D:



$$D = A \oplus B$$

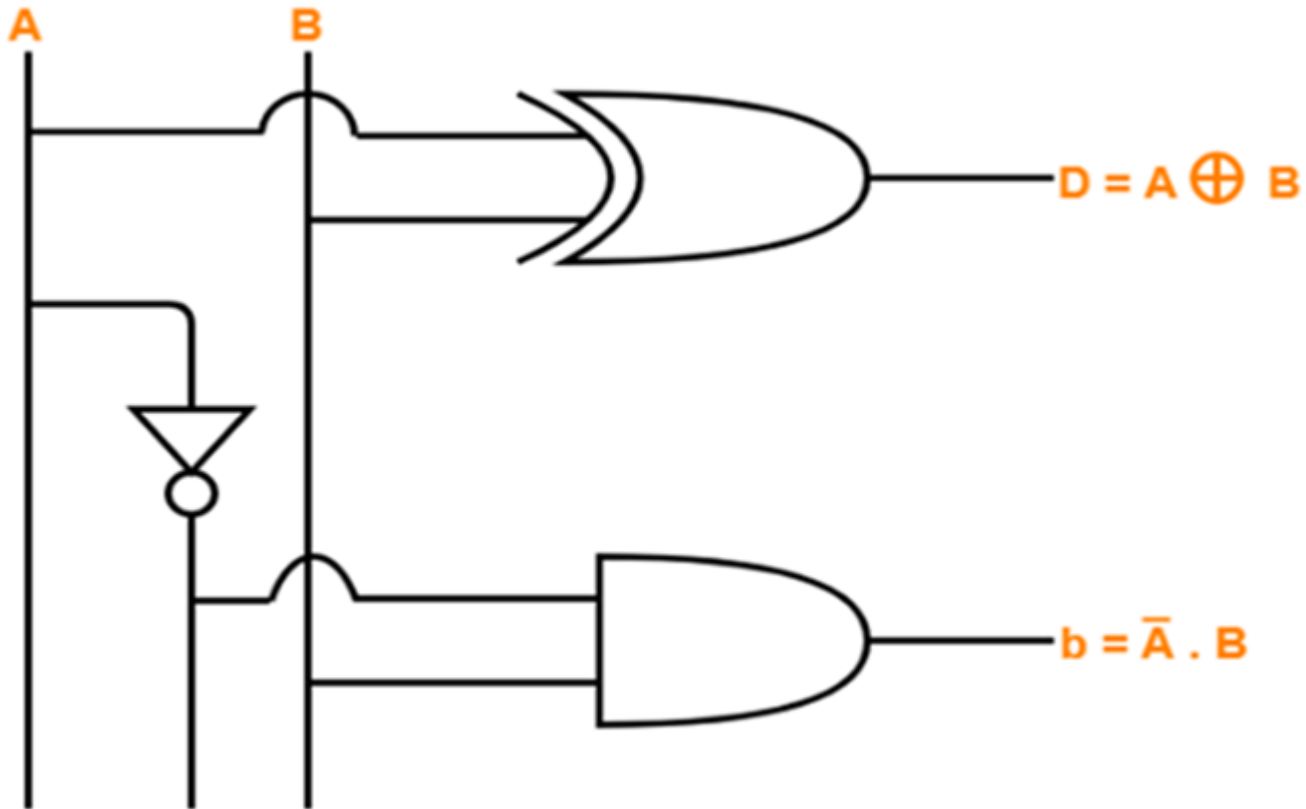
For b:



$$b = \bar{A}.B$$

HS-Logic Diagram

Step-4: Draw the Logic Diagram





संयुक्त प्रौद्योगिकी

SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR

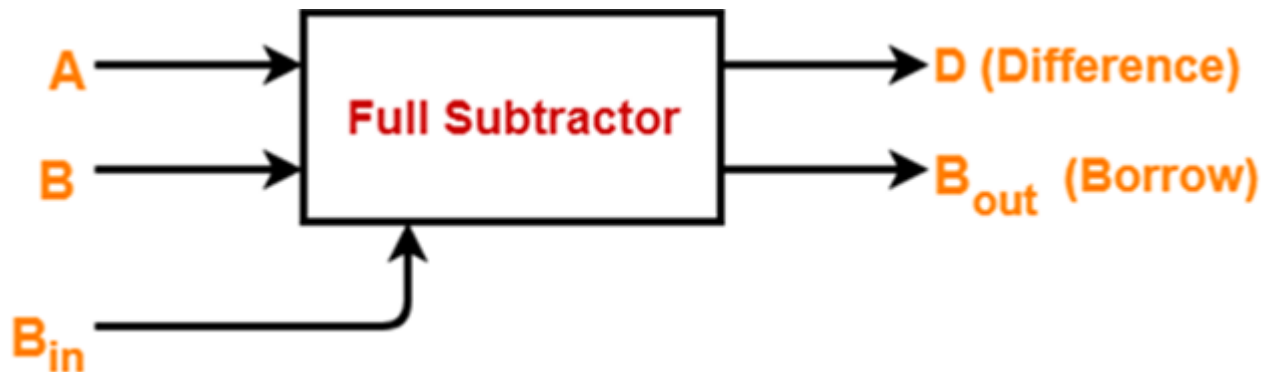
HA-Limitations

- Half subtractors do not take into account "Borrow-in" from the previous circuit.
- This is a major drawback of half subtractors.
- This is because real time scenarios involve subtracting the multiple number of bits which can not be accomplished using half subtractors.



Half Subtractor (HS)

- Full Subtractor is a combinational logic circuit.
- It is used for the purpose of subtracting two single bit numbers.
- It also takes into **consideration borrow of the lower significant stage**.
- Thus, full subtractor has the ability to perform the subtraction of three bits.
- Full subtractor **contains 3 inputs and 2 outputs (Difference and Borrow)** as shown below:



FS-Truth Table

Step-1: Identify the Input and Output Variables

- Input variables = A , B , B_{in} (either 0 or 1)
- Output variables = D , B_{out} where D = Difference and B_{out} = Borrow

Step-2: Draw the Truth Table

Inputs			Outputs	
A	B	B_{in}	B_{out} (Borrow)	D (Difference)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Truth Table



HS-K Map

Step-3: Draw K-Map using Truth Table & determine the simplified Boolean Expression

For D:

A	$B B_{in}$ $\bar{B} \bar{B}_{in}$ $\bar{B} B_{in}$ $B \bar{B}_{in}$			
	$\bar{A}$		1	
A		1		1

$$D = A \oplus B \oplus B_{in}$$

For B_{in} :

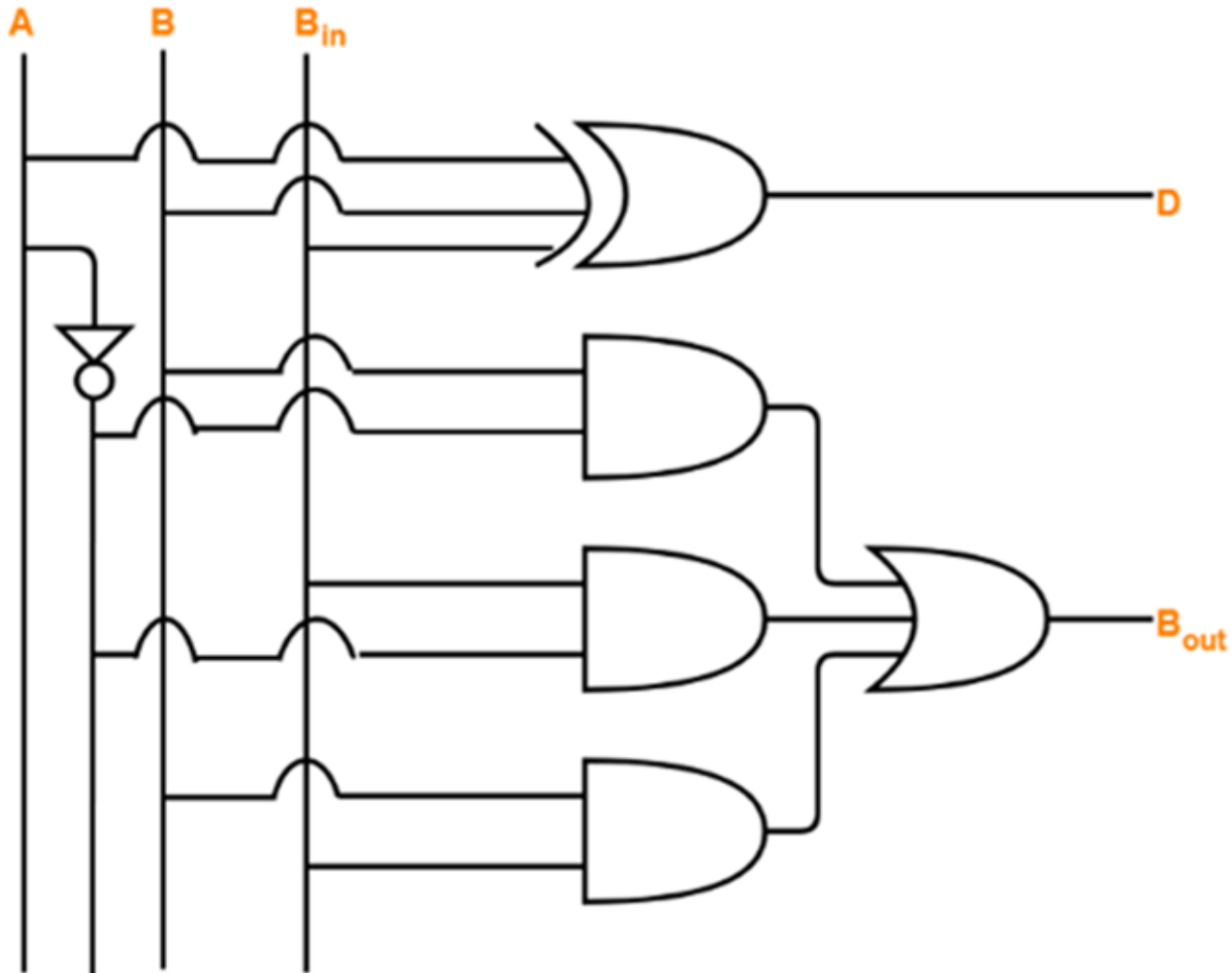
A	$B B_{in}$ $\bar{B} \bar{B}_{in}$ $\bar{B} B_{in}$ $B \bar{B}_{in}$			
	$\bar{A}$		1	1
A			1	

$$B_{out} = \bar{A} B + (\bar{A} + B) B_{in}$$



HS-Logic Diagram

Step-4: Draw the Logic Diagram





संयुक्त प्रौद्योगिकी

SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR

Cont...

MUX & DEMUX

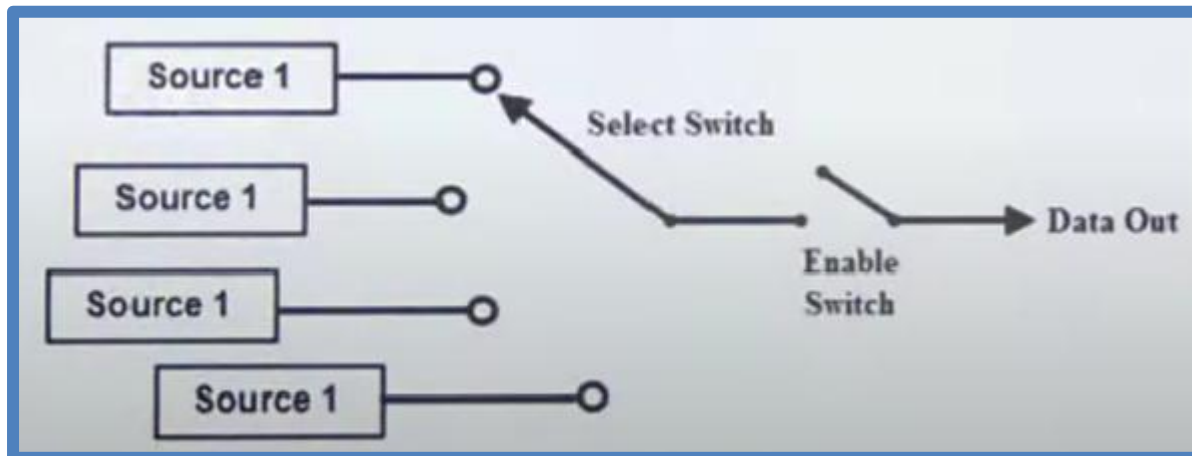


MULTIPLEXER AND DEMULTIPLEXER

Multiplexer (Data Selectors)

What are multiplexers? Draw the logical circuits and describe its working.

- The term Multiplexer means **many into one**.
- A Digital Multiplexer (MUX) is a combinational Circuit with more than One Input Lines, One output lines and more than one select lines.
- A Multiplexer is a **Digital Switch**, also called a **Data Selector**.
- It allows the binary information from several input lines or sources and depending on the set of select lines, particular input line, is routed onto a single output line.

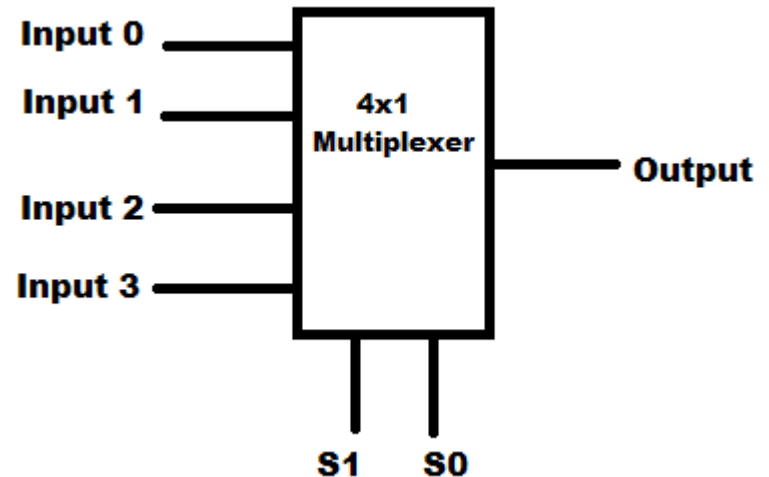


Cont...

➤ MUX directs one of the inputs to its output line by using a control bit word (*selection line*) to its *select lines*.

➤ Multiplexer contains the followings:

- Data Inputs
- Selection Inputs
- A *Single Output*



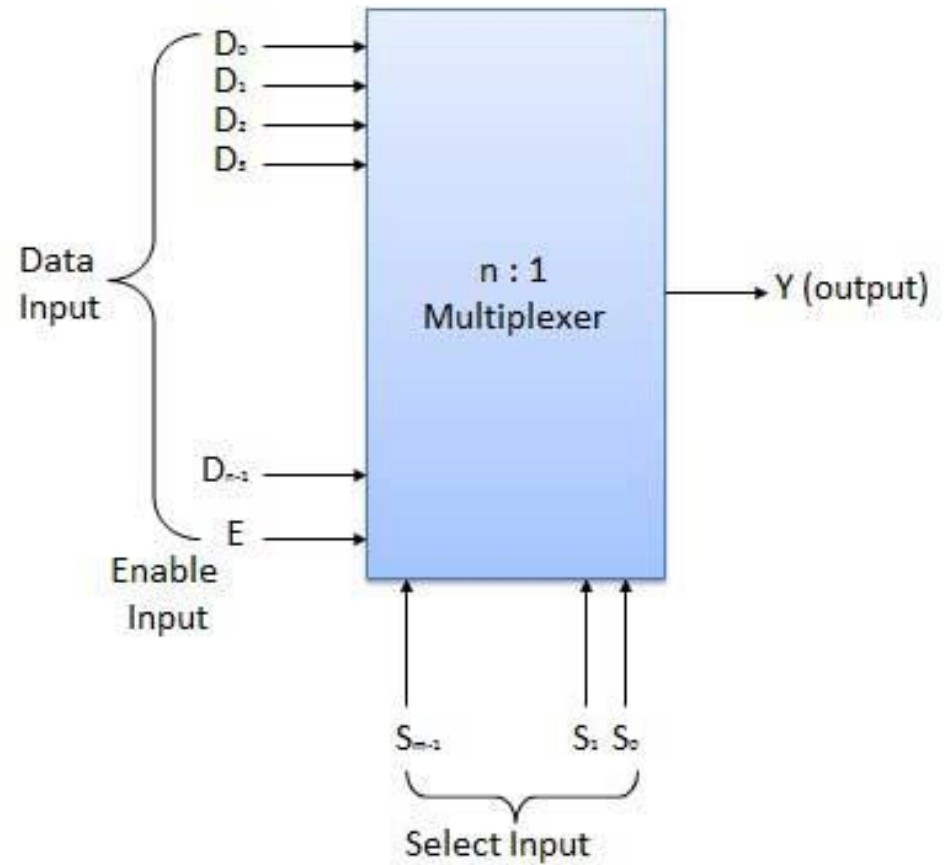
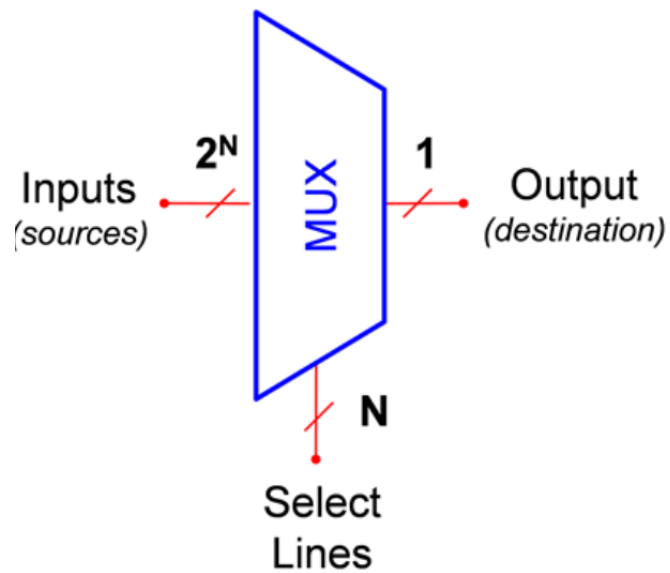
➤ Selection input determines the input that should be connected to the output.

➤ The multiplexer **acts like an electronic switch** that selects one from different.



Cont...

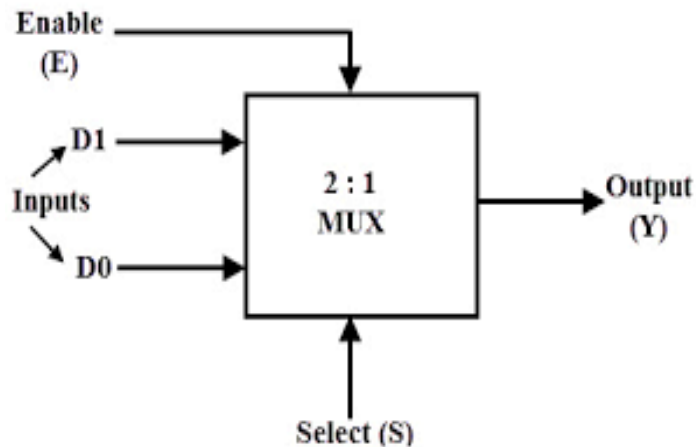
Multiplexer
Block Diagram



2:1 Multiplexer

Draw the symbol and write truth table of 2:1 MUX

- The block schematic of a 2 : 1 multiplexer is shown below:
- It has two data inputs D_0 and D_1 , one select input S , an enable input E and one output Y .
- The truth table of this MUX is shown below:



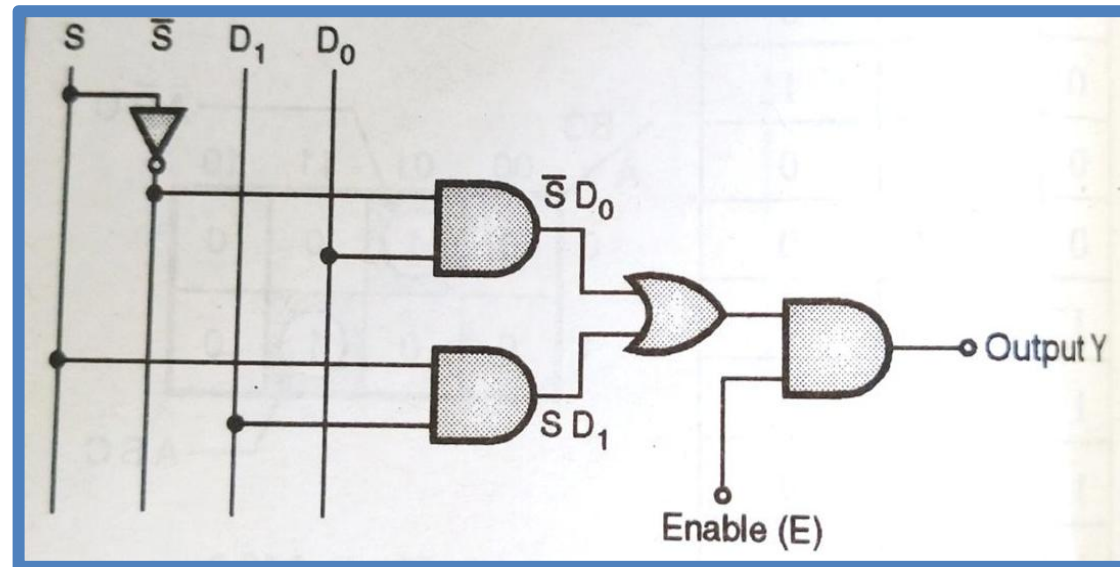
Select	D1 Inputs	D0	Output
0	0	0	0
0	0	1	1
1	1	0	1
1	1	1	1



Cont...

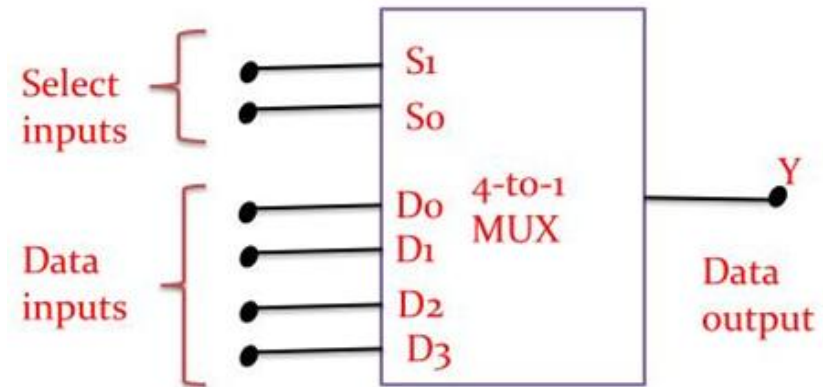
When enable i/p is high then only works.

Enable	Select Input S	Output Y
0	X	0
1	0	D0
1	1	D1



4:1 Multiplexer

- Fig. shows the block diagram of a 4: 1 multiplexer and Table gives its truth table.
- Note that $n = 4$ hence number of select lines i.e. $m = 2$ so that $2^m = n$.
- The truth table tells us that if $S_1 S_0 = 00$, the data bit D_0 is selected and routed to output $Y = D_0$
- Similarly if $S_1 S_0 = 01$, then D_1 is selected and routed to the output $Y = D_1$
- $Y = D_2$ for $S_1 S_0 = 10$ and $Y = D_3$ for $S_1 S_0 = 11$

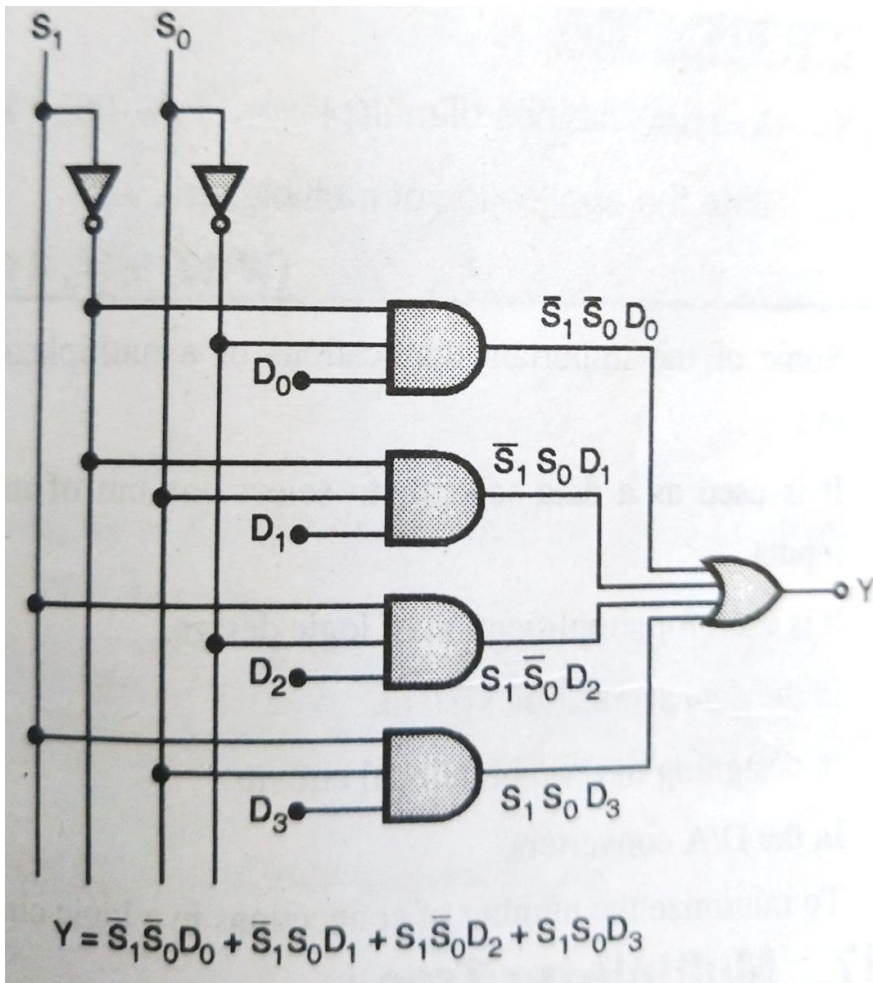


Data select inputs		Output
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

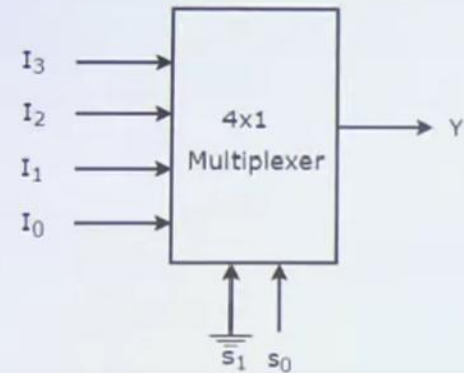
$$Y = \bar{S}_1 \bar{S}_0 D_0 + S_1 \bar{S}_0 D_1 + \bar{S}_1 S_0 D_2 + S_1 S_0 D_3$$

Cont...

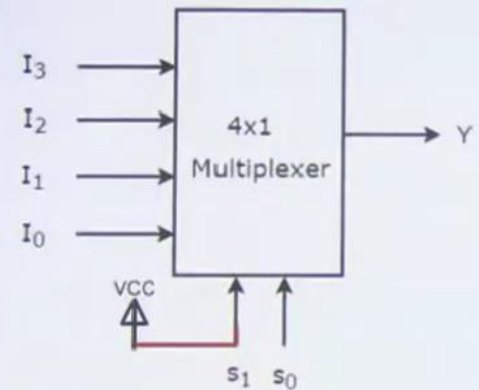
This expression can be realized using basic gates as shown in



4 x 1 (Connecting S_1 pin to Ground)

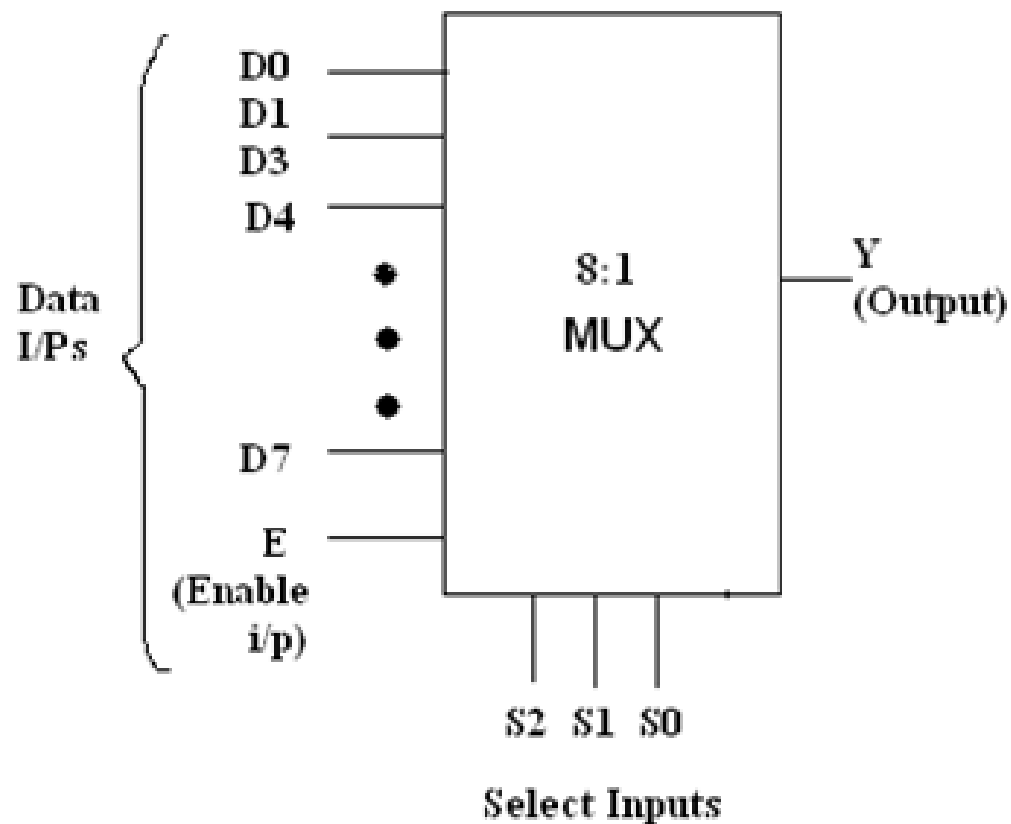


4 x 1 (Connecting Supply to S_1 pin)





8:1 Multiplexer



Enable	Select Inputs			Output
	S2	S1	S0	
0	X	X	X	0
1	0	0	0	D0
1	0	0	1	D1
1	0	1	0	D2
1	0	1	1	D3
1	0	0	0	D4
1	0	0	1	D5
1	0	1	0	D6
1	0	1	1	D7

Cont...

The block diagram of an 8 : 1 MUX is shown in Fig. 1 and its truth table is shown in Fig. 2. It has eight data inputs, one enable input, three select inputs and one output.

Operating principle :

- When the strobe or enable input is 0, the multiplexer will be 0 irrespective of any input. With $E = 1$, we can select any one of the eight data inputs and connect it to the output.
- For example if $S_2 S_1 S_0 = 0 1 1$ then the data input D_3 is selected and output Y will follow the selected input D_3 .
- Realization using gates :
- The expression for output Y can be obtained from the truth table of Fig. as

$$Y = D_0 \overline{S_2} \overline{S_1} \overline{S_0} + D_1 \overline{S_2} \overline{S_1} S_0 + D_2 \overline{S_2} S_1 \overline{S_0} + D_3 \overline{S_2} S_1 S_0 + D_4 S_2 \overline{S_1} \overline{S_0} + D_5 S_2 \overline{S_1} S_0 \\ + D_6 S_2 S_1 \overline{S_0} + D_7 S_2 S_1 S_0$$

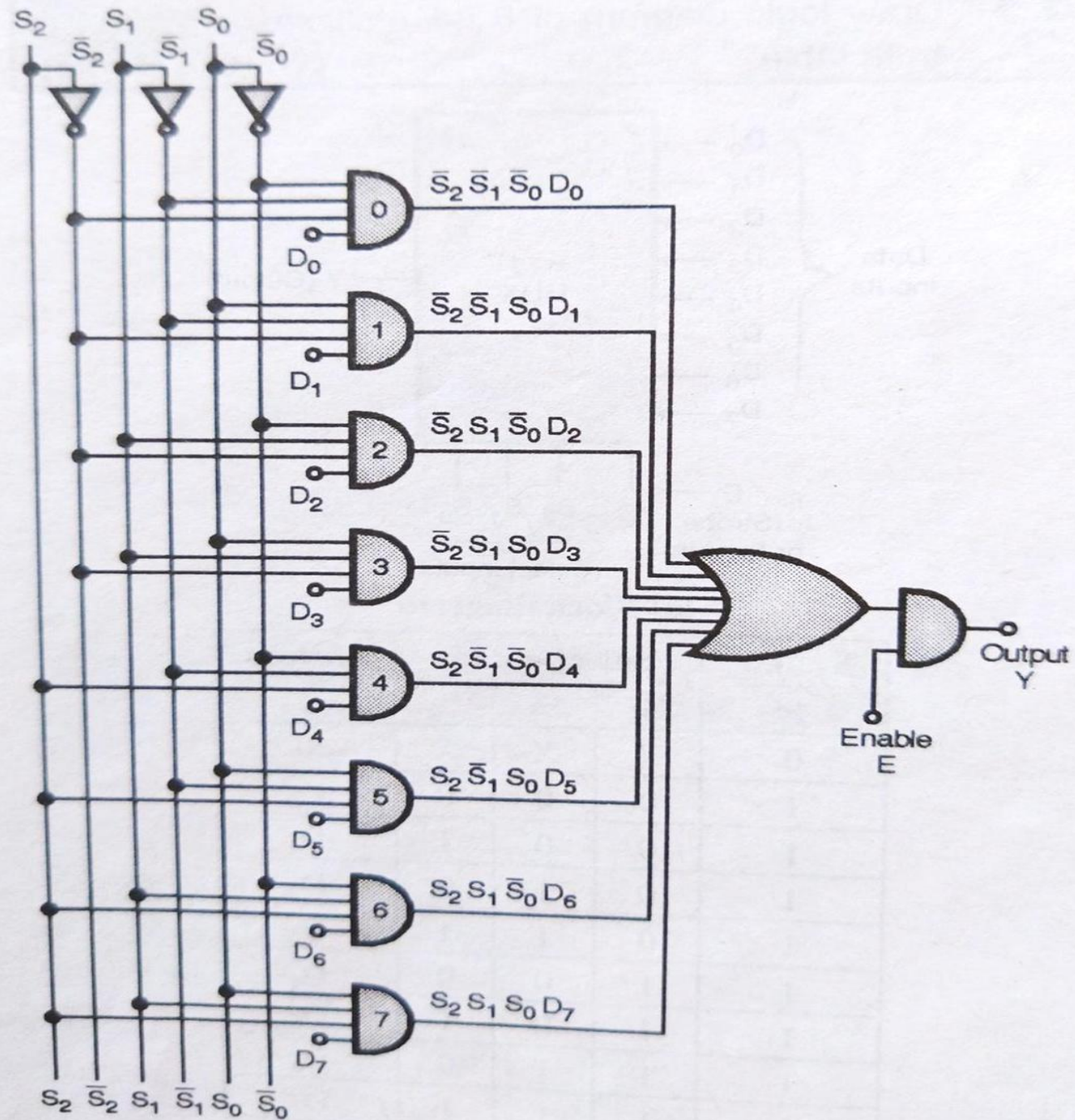
- The realization of 8 : 1 MUX using gates is shown in Fig.



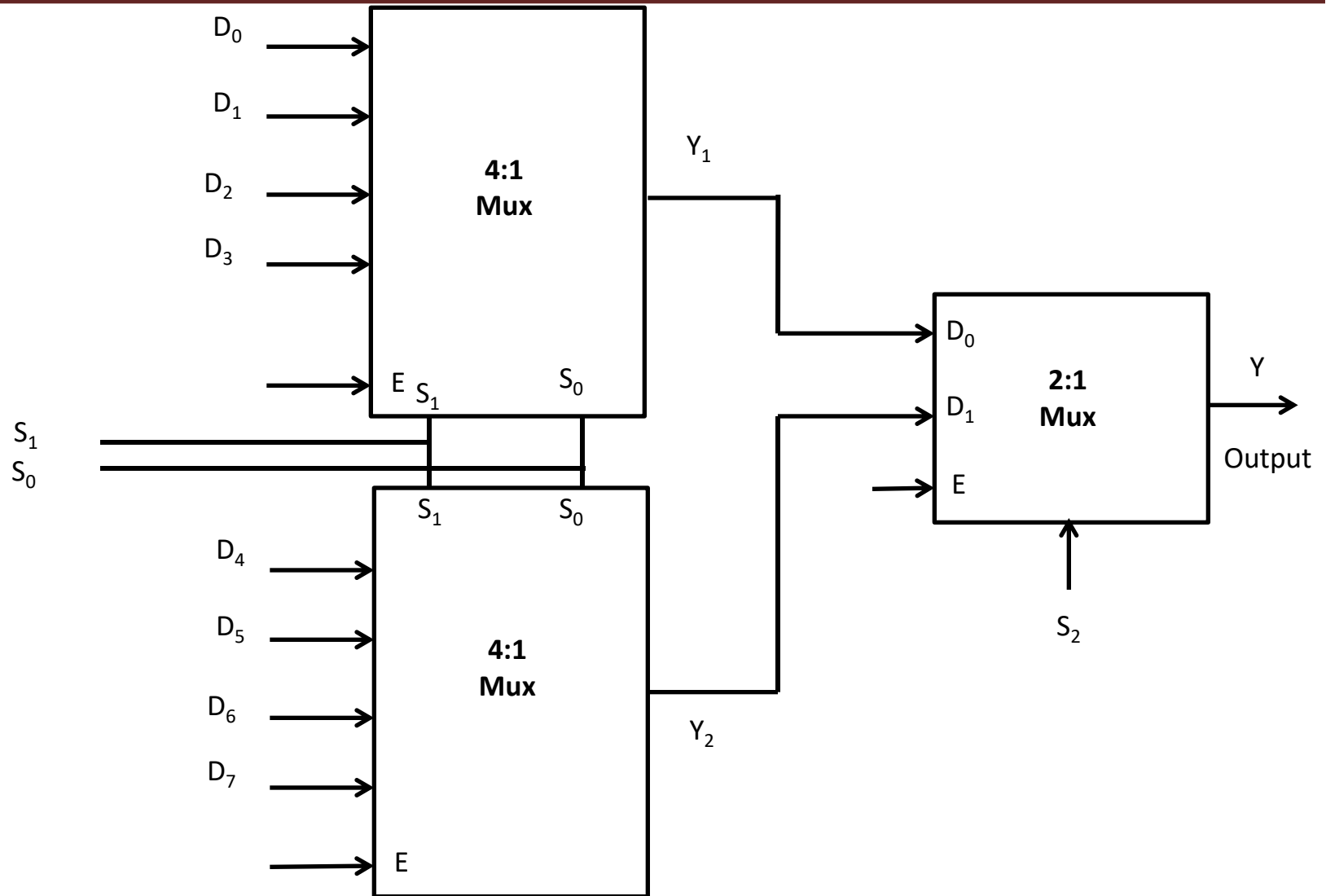
SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR

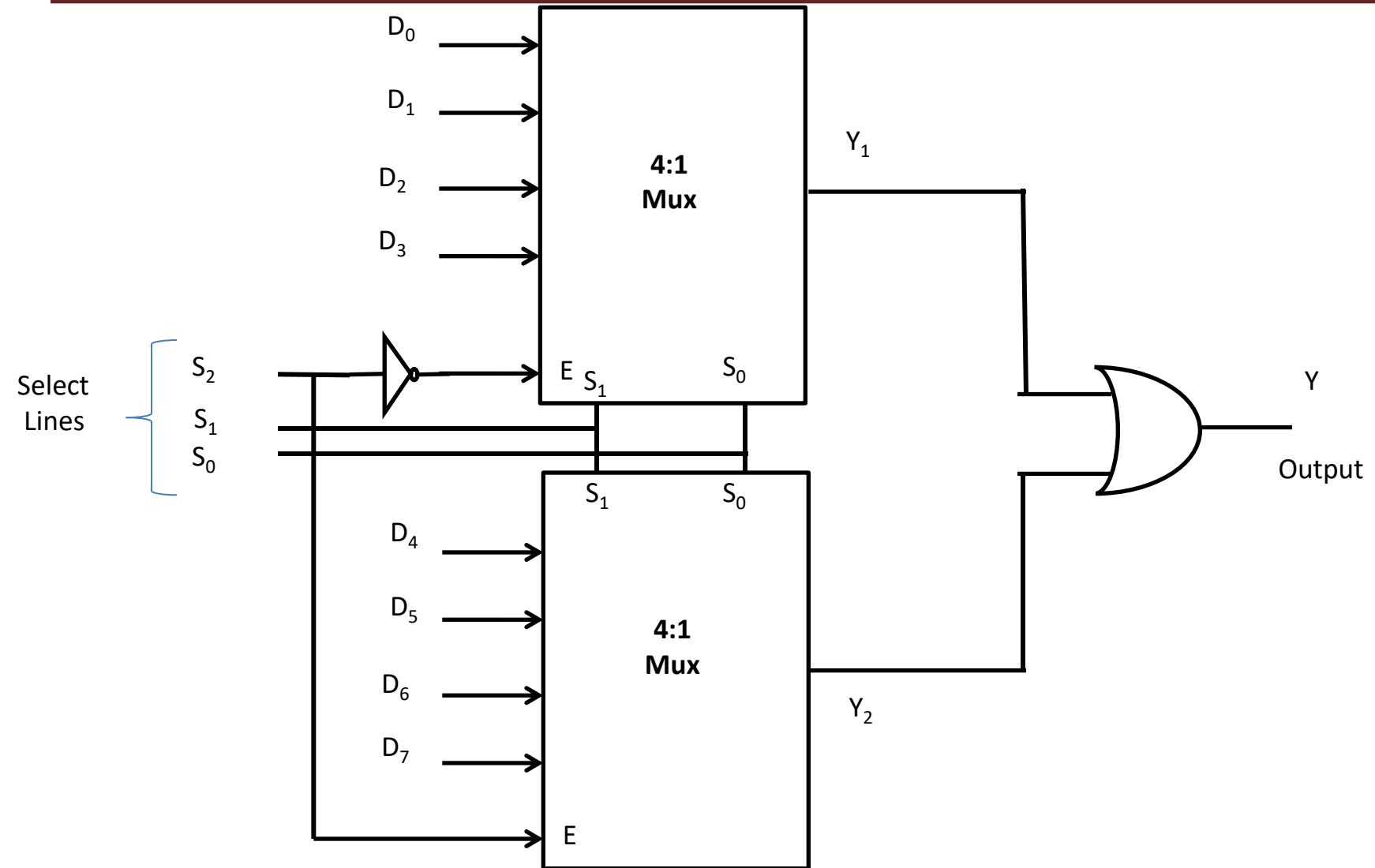
Cont...



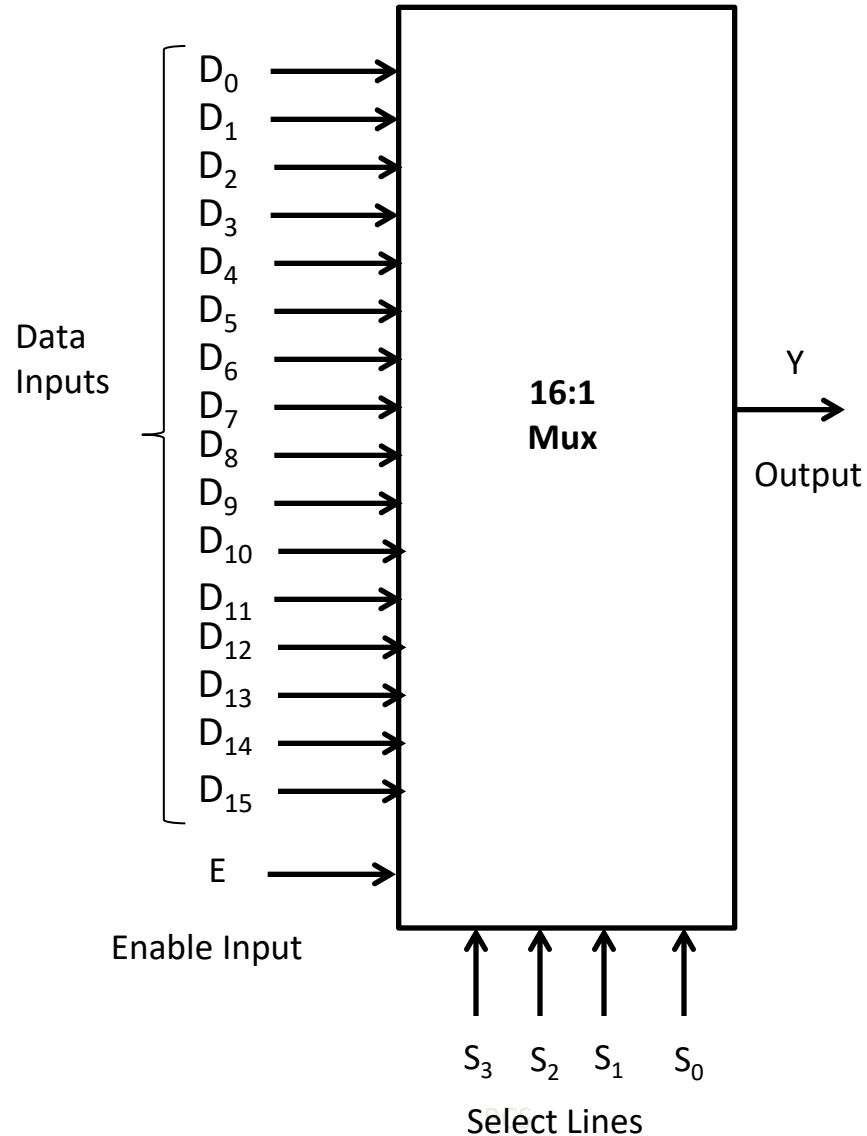
8:1 Multiplexer using 4:1 Multiplexer



8:1 Multiplexer using 4:1 Multiplexer



16:1 Multiplexer



Block Diagram

16:1 Multiplexer

Truth Table

Enable	Select Lines				Output
E	S ₃	S ₂	S ₁	S ₀	Y
0	X	X	X	X	0
1	0	0	0	0	D ₀
1	0	0	0	1	D ₁
1	0	0	1	0	D ₂
1	0	0	1	1	D ₃
1	0	1	0	0	D ₄
1	0	1	0	1	D ₅
1	0	1	1	0	D ₆
1	0	1	1	1	D ₇
1	1	0	0	0	D ₈
1	1	0	0	1	D ₉
1	1	0	1	0	D ₁₀
1	1	0	1	1	D ₁₁
1	1	1	0	0	D ₁₂
1	1	1	0	1	D ₁₃
1	1	1	1	0	D ₁₄
1	1	1	1	1	D ₁₅



Applications of a Multiplexer

Some of the important applications of a multiplexer are as follows:

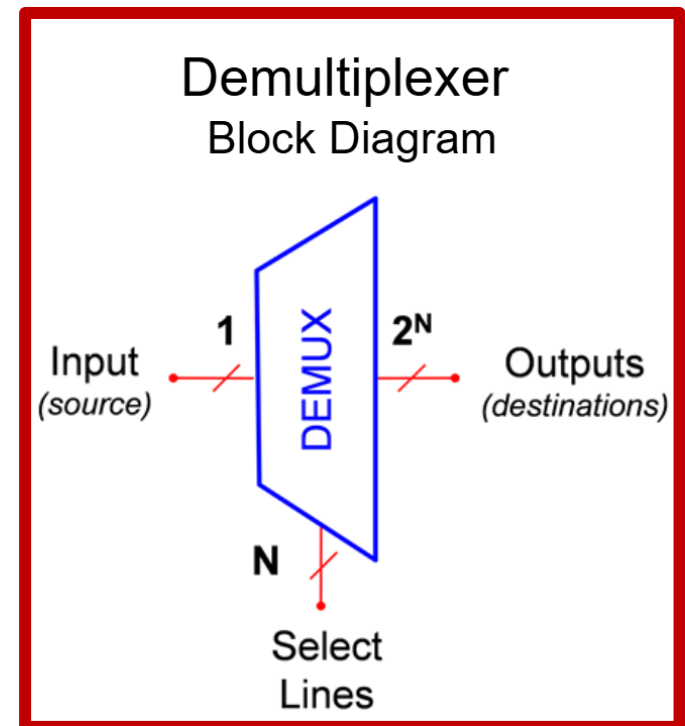
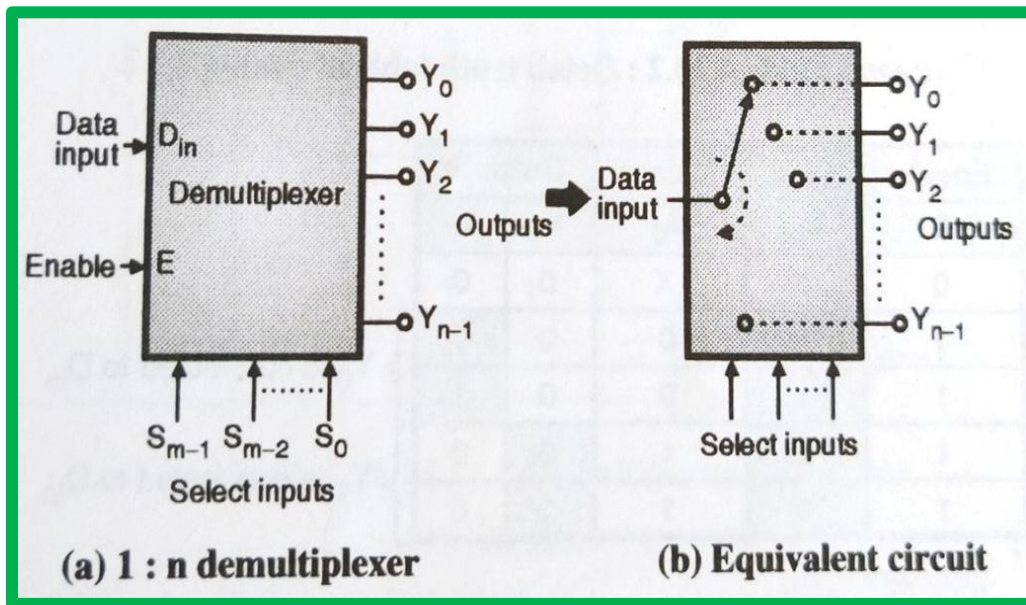
- It is used as a **data selector** to select one out of many data inputs.
- It is used **for simplification of logic design**.
- In the **data acquisition system**. In designing the combinational circuits.
- In the **D/A converters**.
- To **minimize the number of connections in a logic circuit**.

De-Multiplexer

- The De-Multiplexer is a combinational logic circuit that **performs the reverse operation of multiplexer** (Several output lines, one input line).
- De -Multiplexer means **one to many**.
- A De-Multiplexer is a circuit with one input and many output. By applying control signal, we can steer any input to the output. **Few types of De - Multiplexer are 1-to 2, 1-to-4, 1-to-8 and 1-to 16 De -Multiplexer .**
- De-Multiplexer is the process of taking information from one input and transmitting the same over one of several outputs.

Cont...

- A demultiplexer performs the reverse operation of a multiplexer i.e. it receives one input and distributes it over several outputs.
- At a time only one output line is selected by the select lines and the input is routed to the selected output line.
- Hence a demultiplexer is equivalent to a single pole multiple way switch as shown in Fig.





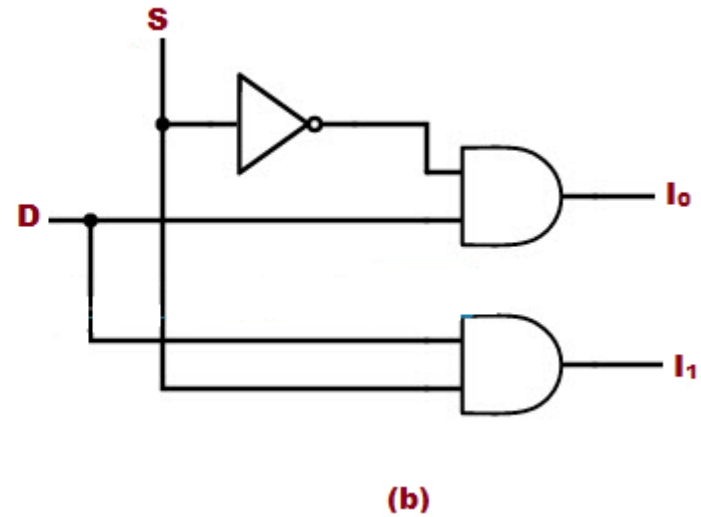
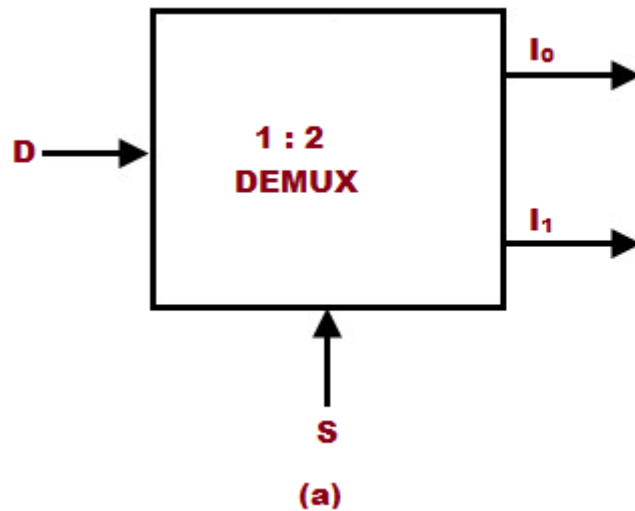
Types of Demultiplexers

Similar to the multiplexers, the demultiplexers are classified as follows:

- 1:2 demultiplexer
- 1:4 demultiplexer
- 1:8 demultiplexer
- 1: 16 demultiplexer



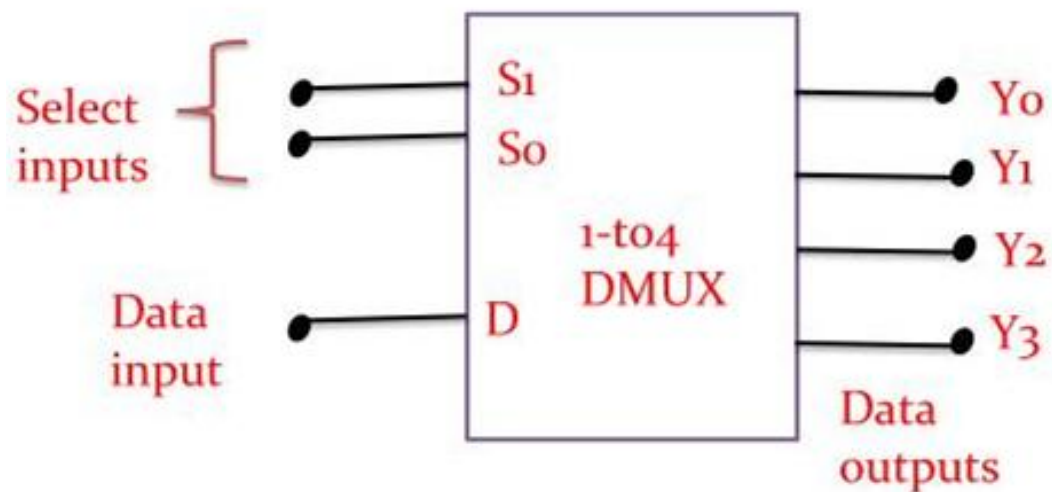
1:2 Demux



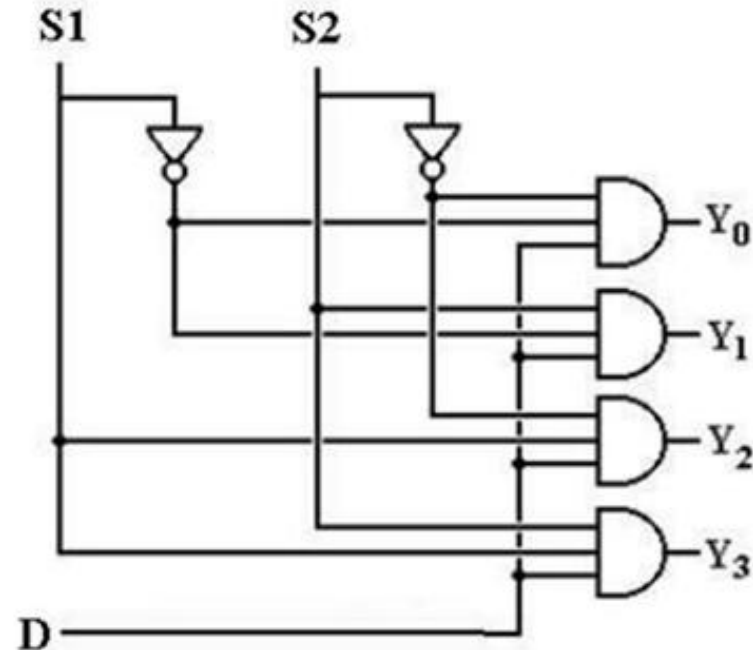
Select	Input	Outputs	
S	D	Y_1	Y_0
0	0	0	0
0	1	0	1
1	0	0	0
1	1	1	0



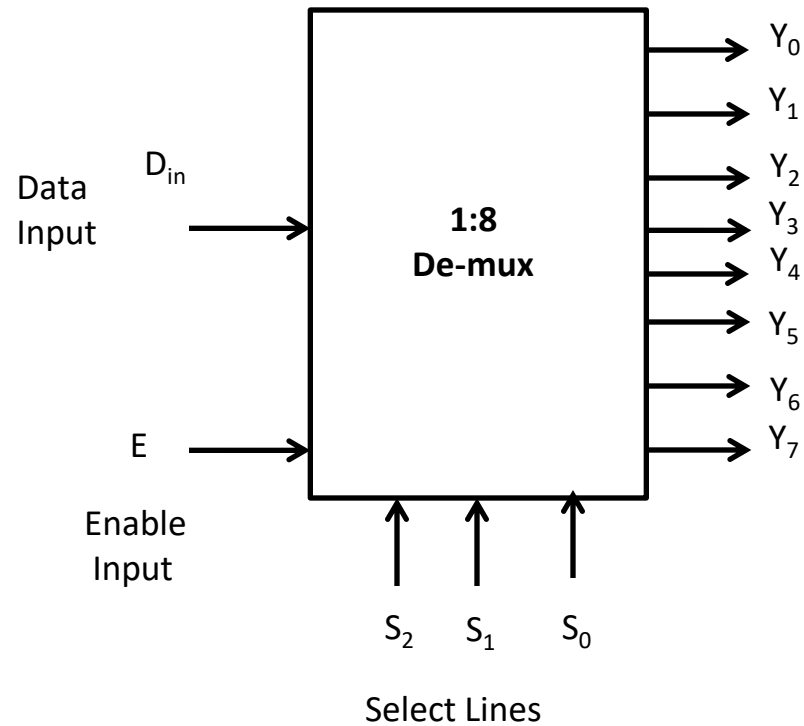
1:4 Demux



Data Input	Select Inputs		Output			
D	S_1	S_0	Y_3	Y_2	Y_1	Y_0
D	0	0	0	0	0	D
D	0	1	0	0	D	0
D	1	0	0	D	0	0
D	1	1	D	0	0	0



1: 8 De-multiplexer



Block Diagram

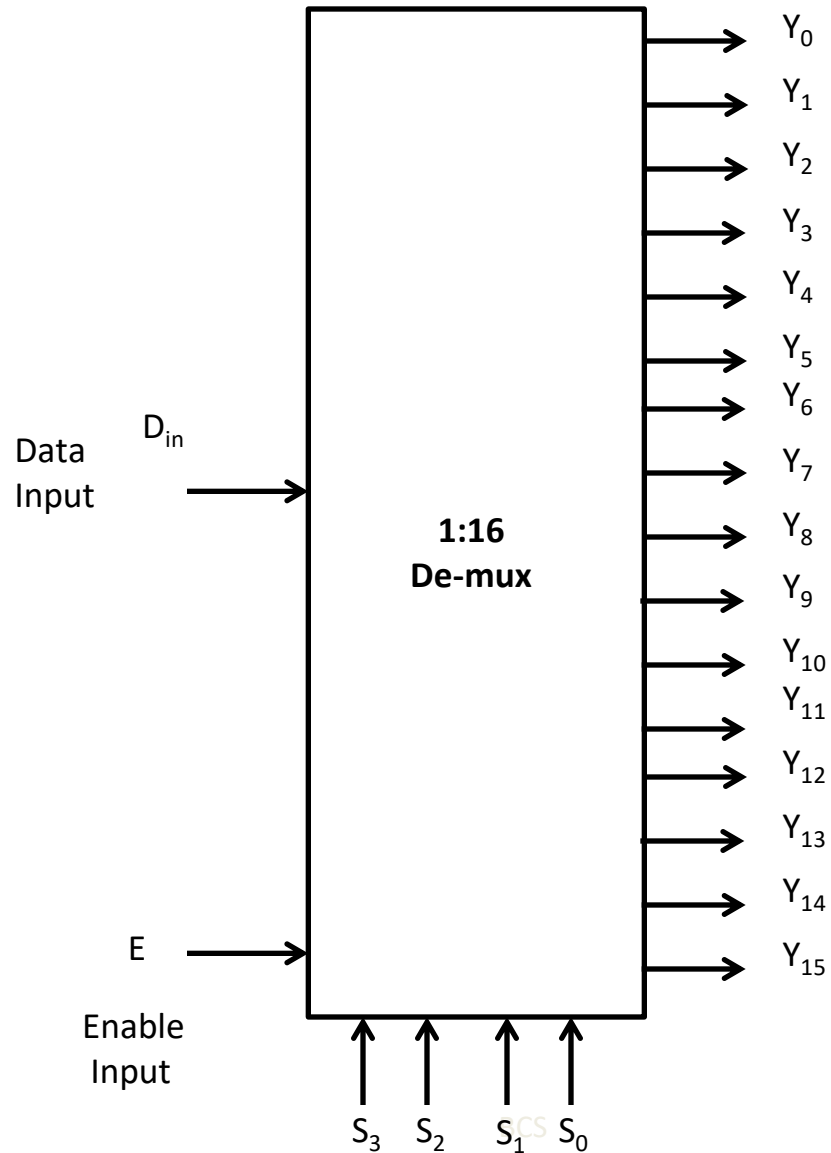
1: 8 De-multiplexer

Truth Table

Enable i/p		Select i/p			Outputs						
E	S ₂	S ₁	S ₀	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	X	X	X	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	D _{in}
1	0	0	1	0	0	0	0	0	0	D _{in}	0
1	0	1	0	0	0	0	0	0	D _{in}	0	0
1	0	1	1	0	0	0	0	D _{in}	0	0	0
1	1	0	0	0	0	0	D _{in}	0	0	0	0
1	1	0	1	0	0	D _{in}	0	0	0	0	0
1	1	1	0	0	D _{in}	0	0	0	0	0	0
1	1	1	1	D _{in}	0	0	0	0	0	0	

1: 16 De-multiplexer

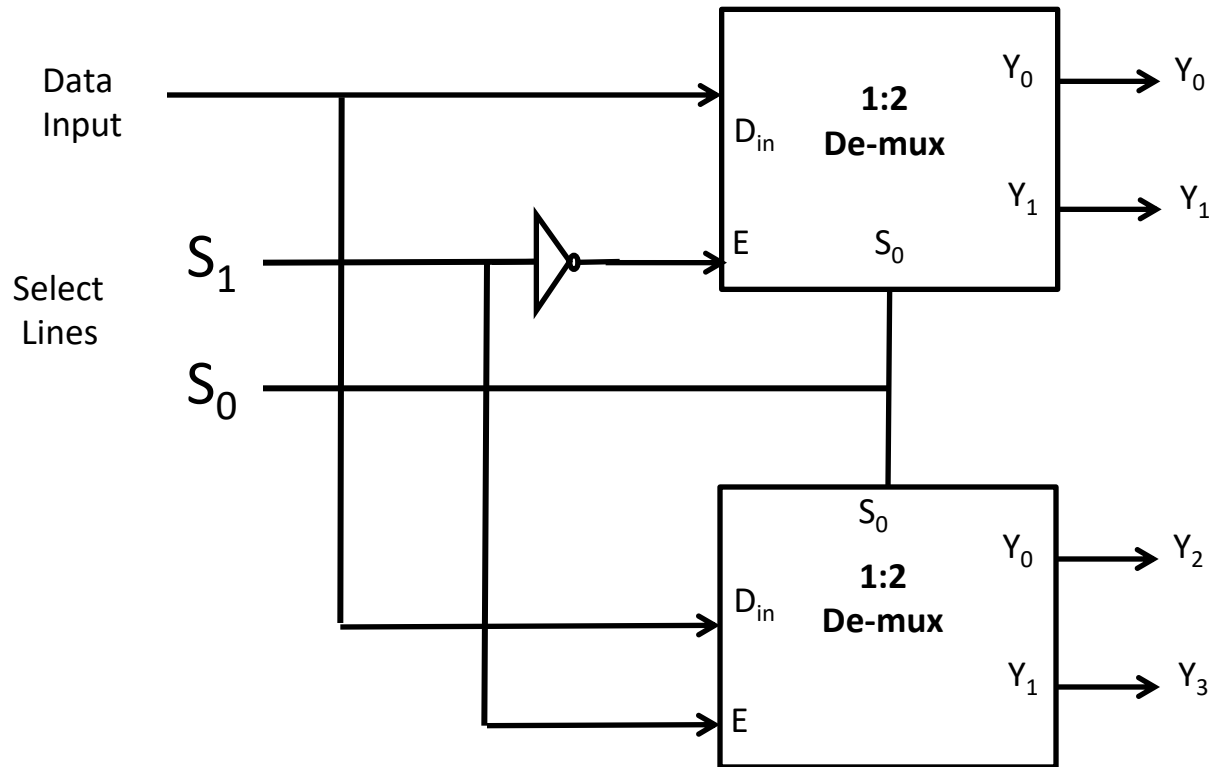
Block Diagram



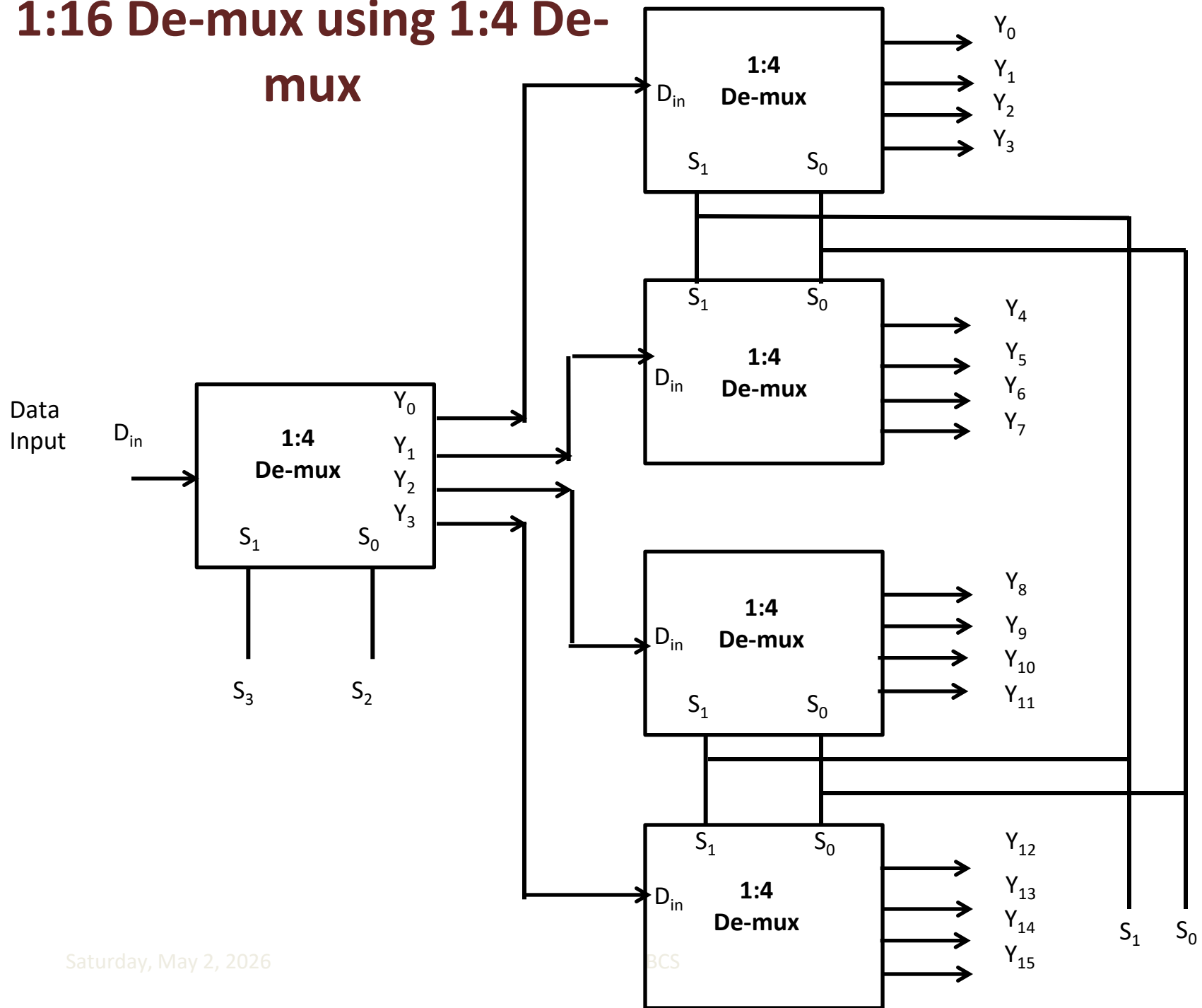
De-mux Tree

- ✓ Similar to multiplexer we can construct the de-multiplexer with more number of lines using de-multiplexer having less number of lines. This is call as “De-mux Tree”.

1:4 De-mux using 1:2 De-mux



1:16 De-mux using 1:4 De-mux



Applications of Multiplexer

Communication system – Communication system is a set of system that enable communication like transmission system, relay and tributary station, and communication network. The efficiency of communication system can be increased considerably using multiplexer. Multiplexer allow the process of transmitting different type of data such as audio, video at the same time using a single transmission line.

Computer Memory – A Multiplexer is used in computer memory to keep up a vast amount of memory in the computers, and also to decrease the number of copper lines necessary to connect the memory to other parts of the computer.

Applications of De-Multiplexer

Communication System - Communication system use multiplexer to carry multiple data like audio, video and other form of data using a single line for transmission. This process make the transmission easier. The demultiplexer receive the output signals of the multiplexer and converts them back to the original form of the data at the receiving end. The multiplexer and demultiplexer work together to carry out the process of transmission and reception of data in communication system.

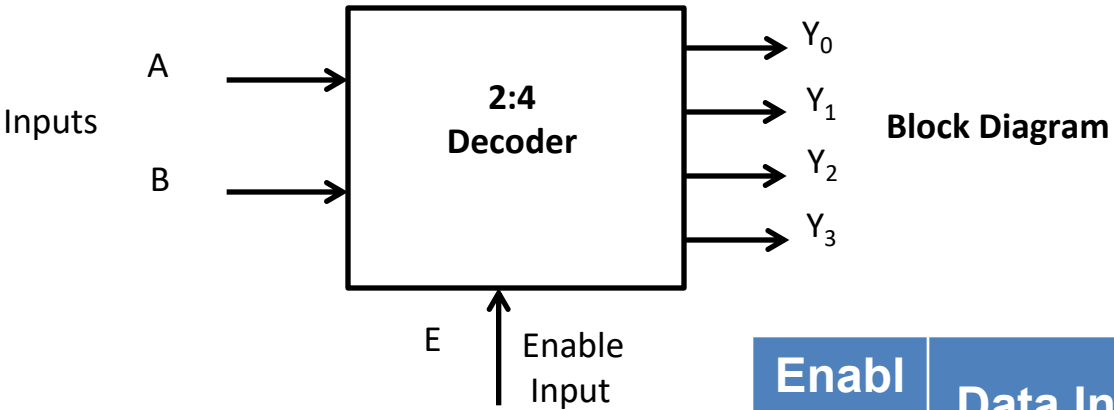
74153 Multiplexer IC

- **Specifications :**
- TTL ICs i.e. Transistor Transistor Logic ICs.
- Total pins = 16 pins.
- Dual 4:1 line multiplexer circuit in single package.
- Supply voltage = 5V DC ± 0.25 V.
- Operating temperature range = 0°C to +70°C.
- High fan-out = 10.
- Low impedance < 100 Ω
- Totem pole outputs.

Decoder

- ✓ Decoder is a combinational circuit.
- ✓ It converts n bit binary information at its input into a maximum of 2_n output lines.
- ✓ For example, if $n=2$ then we can design upto 2:4 decoder

2:4 Decoder



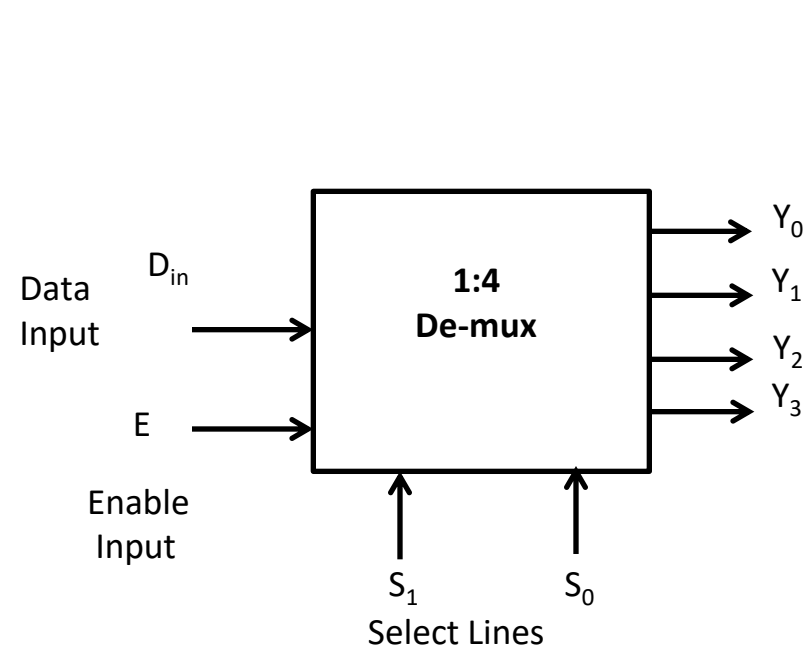
Truth Table

Enabl e i/p	Data Inputs		Outputs			
E	A	B	Y_0	Y_1	Y_2	Y_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

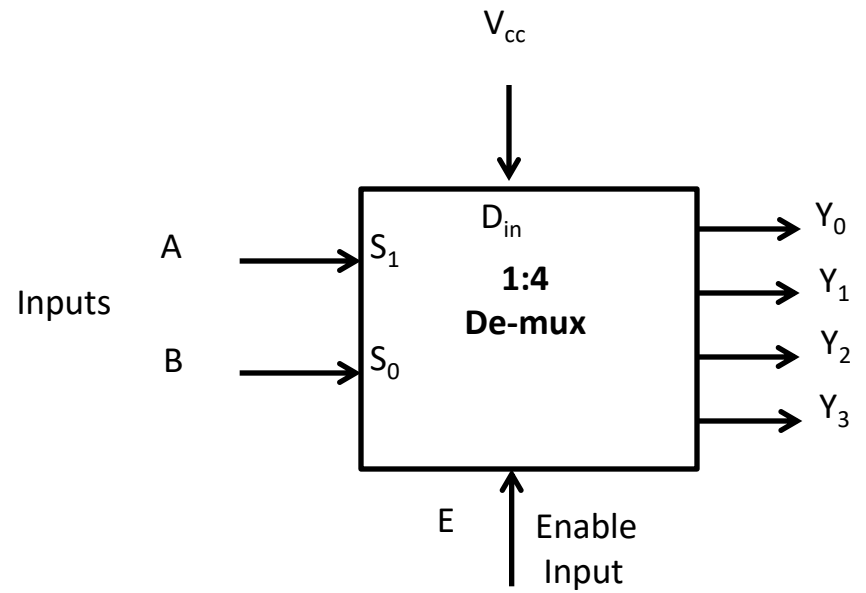
De-multiplexer as Decoder

- ✓ It is possible to operate a de-multiplexer as a decoder.
- ✓ Let us consider an example of 1:4 de-mux can be used as 2:4 decoder

1:4 De-multiplexer as 2:4 Decoder



1: 4 De-multiplexer



1: 4 De-multiplexer as 2:4 Decoder

Realization of Boolean expression using De-mux

- ✓ We can implement any Boolean expression using de-multiplexers.
- ✓ It reduces circuit complexity.
- ✓ It does not require any simplification

Example 1

Implement following Boolean expression using de-multiplexer

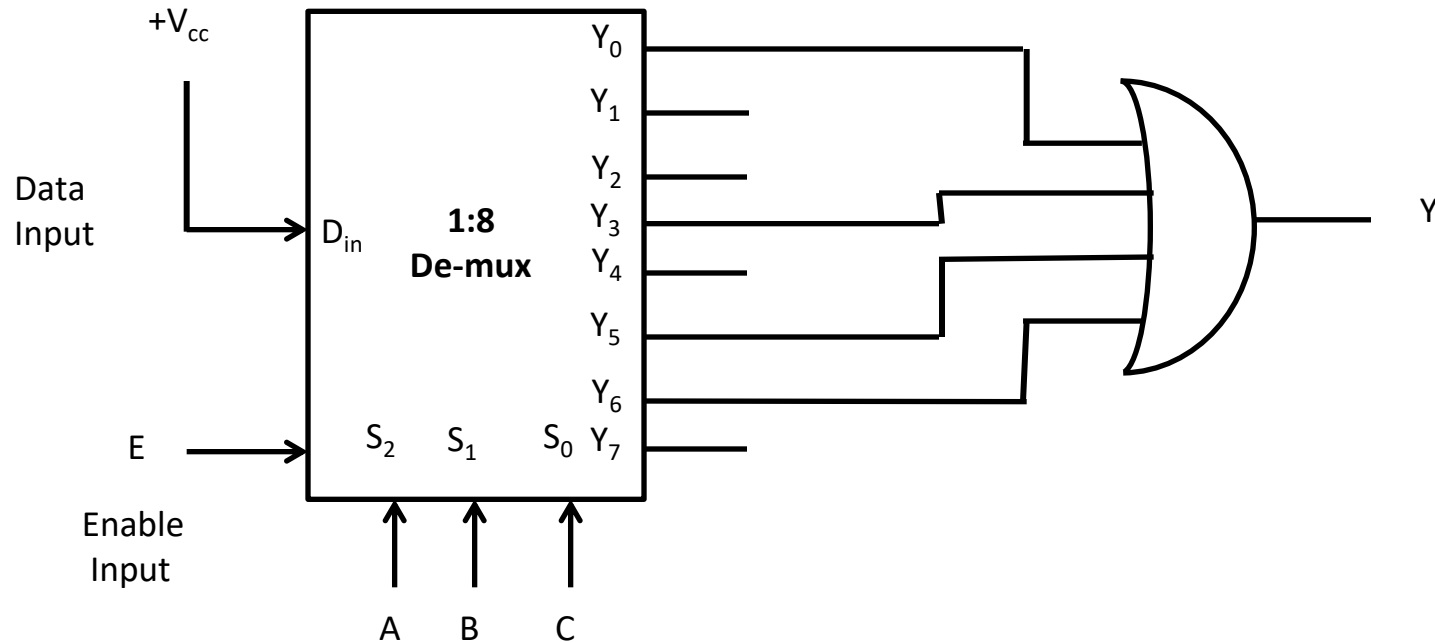
$$f(A, B, C) = \sum m(0, 3, 5, 6)$$

- ✓ Since there are three variables, therefore a de-multiplexer with three select input is required i.e. 1:8 de-multiplexer is required
- ✓ The 1:8 de-multiplexer is configured as below to implement given Boolean expression

Example 1

continue.....

$$f(A, B, C) = \sum m(0, 3, 5, 6)$$



Example 2

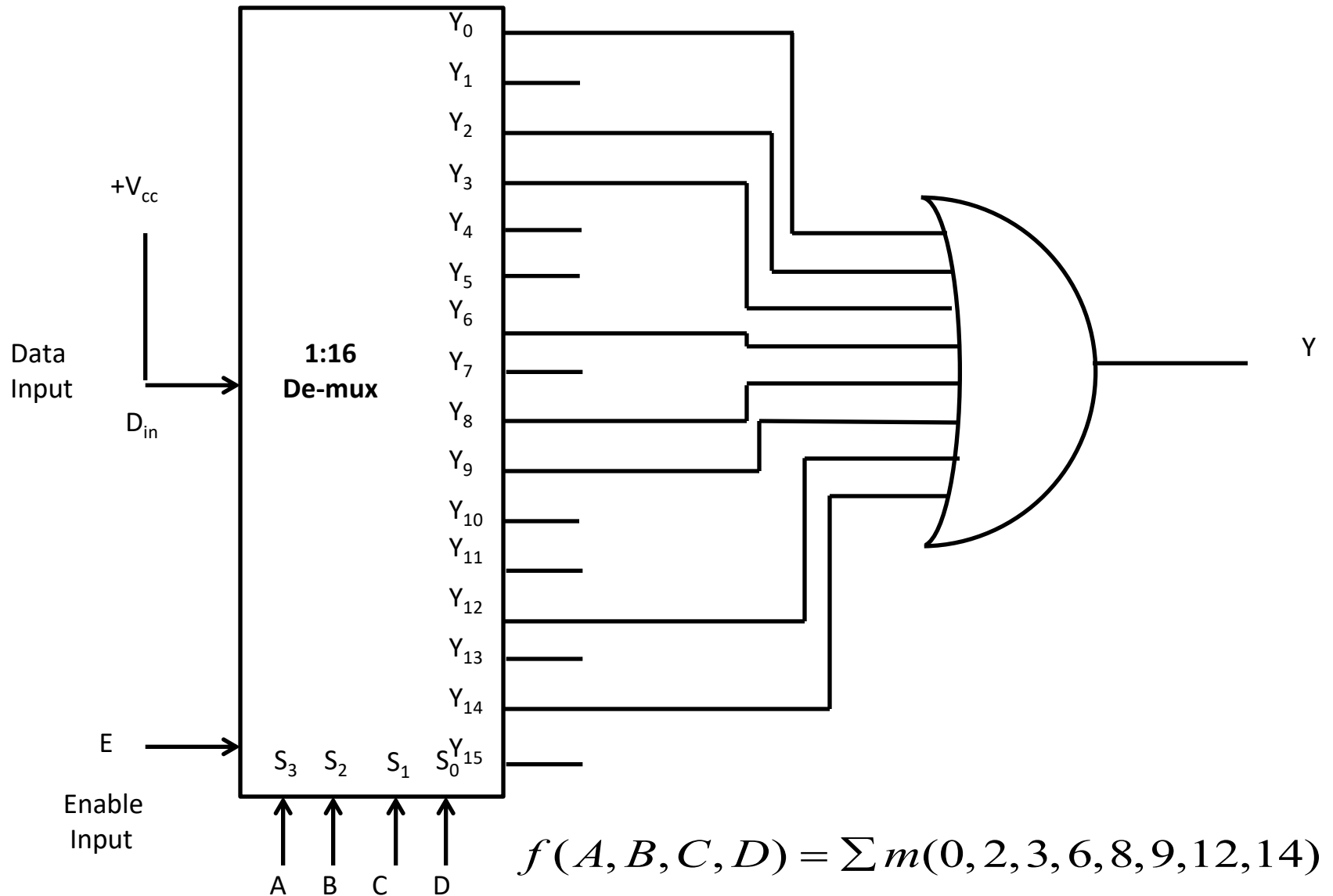
Implement following Boolean expression using de-multiplexer

$$f(A, B, C, D) = \sum m(0, 2, 3, 6, 8, 9, 12, 14)$$

- ✓ Since there are four variables, therefore a de-multiplexer with four select input is required i.e. 1:16 de-multiplexer is required
- ✓ The 1:16 de-multiplexer is configured as below to implement given Boolean expression

Example 2

continue.....



Multiplexer ICs

IC Number	Description	Output
IC 74157	Quad 2:1 Mux	Same as input
IC 74158	Quad 2:1 Mux	Inverted Output
IC 74153	Dual 4:1 Mux	Same as input
IC 74352	Dual 4:1 Mux	Inverted Output
IC 74151	8:1 Mux	Inverted Output
IC 74152	8:1 Mux	Inverted Output
IC 74150	16:1 Mux	Inverted Output

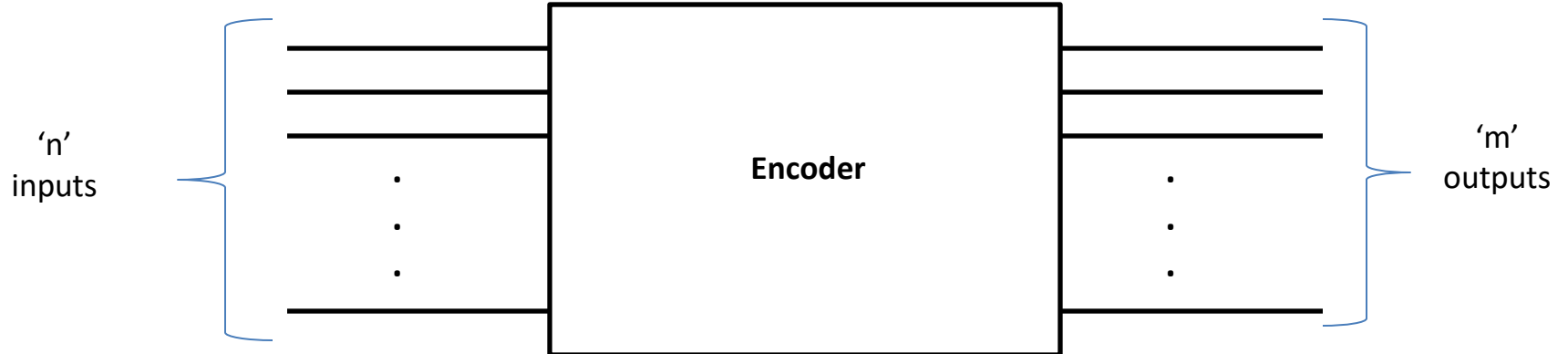
De-multiplexer ICs

IC Number	Description
IC 74138	1:8 De-multiplexer
IC 74139	Dual 1:4 De-multiplexer
IC 74154	1:16 De-multiplexer
IC 74155	Dual 1:4 De-multiplexer

Encoder

- ✓ Encoder is a combinational circuit which is designed to perform the inverse operation of decoder.
- ✓ An encoder has 'n' number of input lines and 'm' number of output lines.
- ✓ An encoder produces an m bit binary code corresponding to the digital input number.
- ✓ The encoder accepts an n input digital word and converts it into m bit another digital word

Encoder



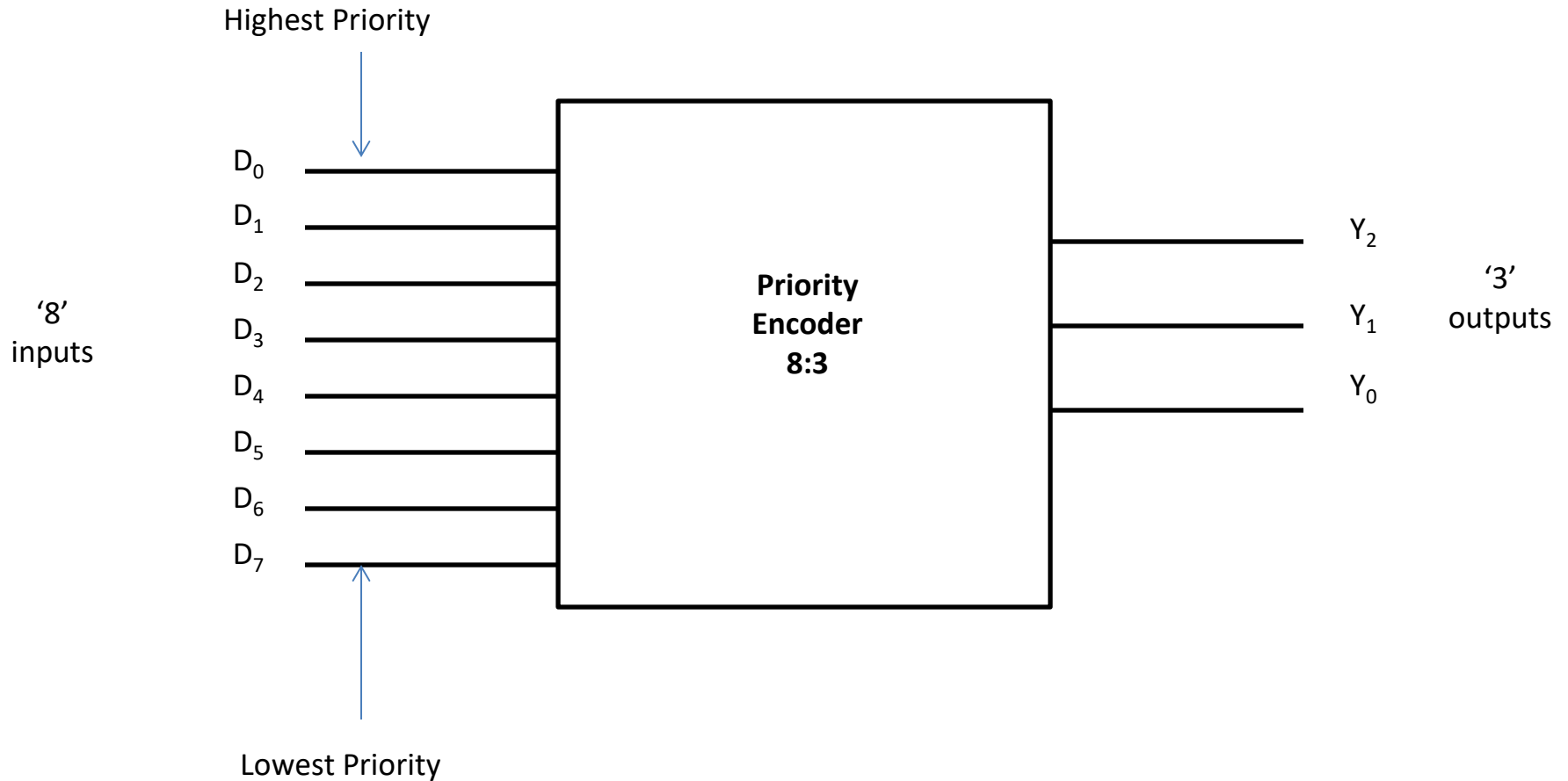
Types of Encoders

- ✓ Priority Encoder
- ✓ Decimal to BCD Encoder
- ✓ Octal to BCD Encoder
- ✓ Hexadecimal to Binary Encoder

Priority Encoder

- ✓ This is a special type of encoder.
- ✓ Priorities are given to the input lines.
- ✓ If two or more input lines are “1” at the same time, then the input line with highest priority will be considered.

Priority Encoder 8:3

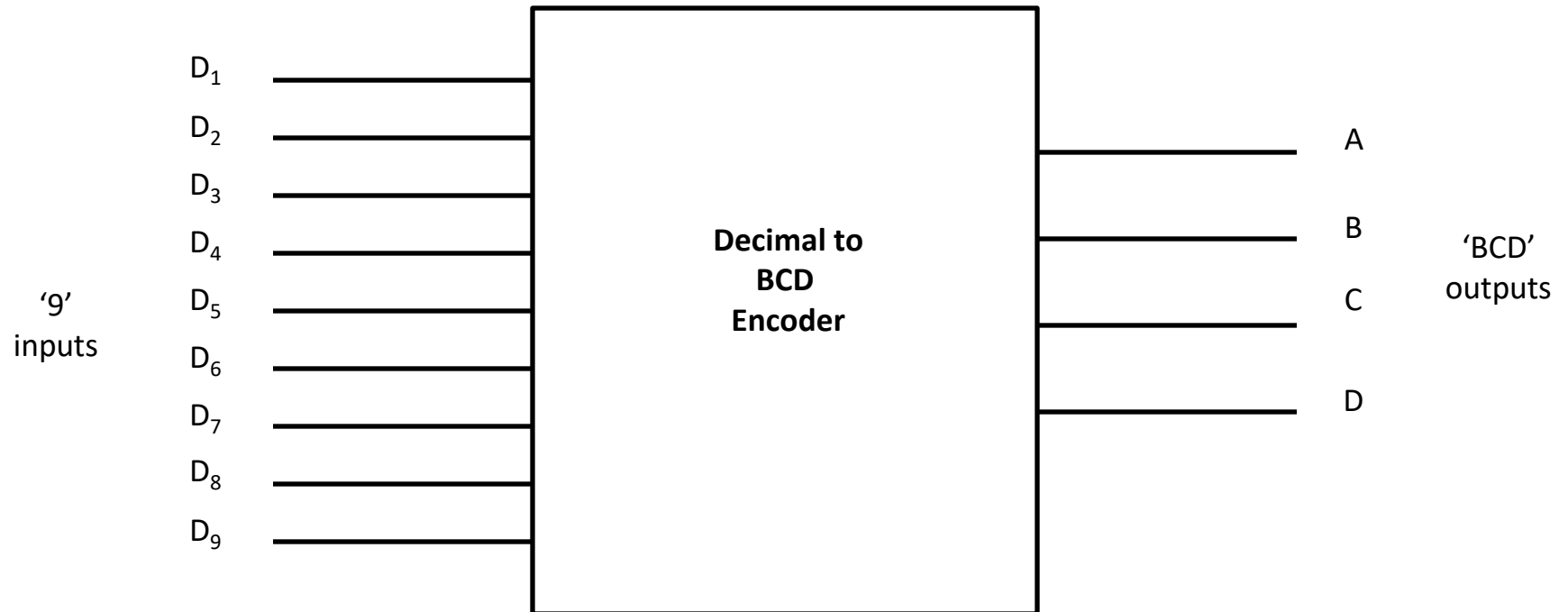


Priority Encoder 8:3

Truth Table:

[illegible]

Decimal to BCD Encoder



Decimal to BCD Encoder

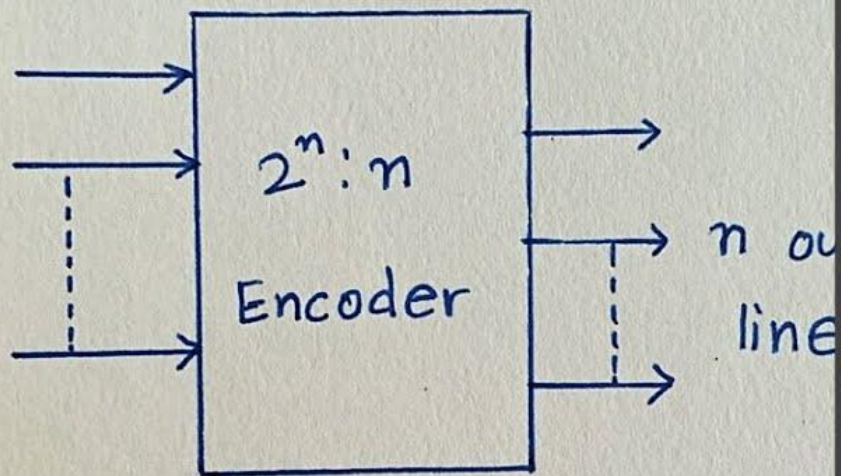
Truth Table:

Inputs									Outputs			
D ₉	D ₈	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D	C	B	A
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	1	X	X	0	0	1	1
0	0	0	0	0	1	X	X	X	0	1	0	0
0	0	0	0	1	X	X	X	X	0	1	0	1
0	0	0	1	X	X	X	X	X	0	1	1	0
0	0	1	X	X	X	X	X	X	0	1	1	1
0	1	X	X	X	X	X	X	X	1	0	0	0
1	X	X	X	X	X	X	X	X	1	0	0	1

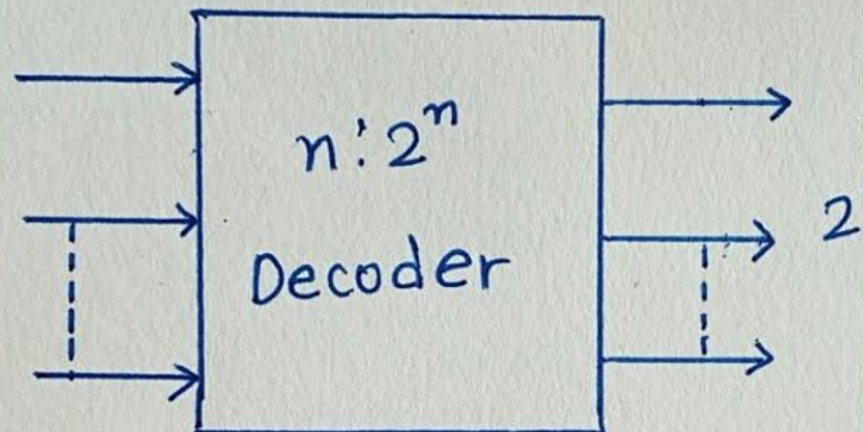
Applications of Encoders

- **Multiplexers:** Converts parallel data bus to serial data stream for transmission or vice versa.
- **Communication Systems:** Encodes digital signals for efficient transmission over limited bandwidth channels.
- **Keyboards and Scanners:** Encodes key presses/scan lines into digital format.
- **Seven Segment Displays:** Uses BCD encoders to display decimal numbers.
- **Barcode Readers:** Decodes barcodes scanned as varying reflectance into digital data format.

Encoders



Decoders



THANK YOU



संयुक्त प्रौद्योगिकी

SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR

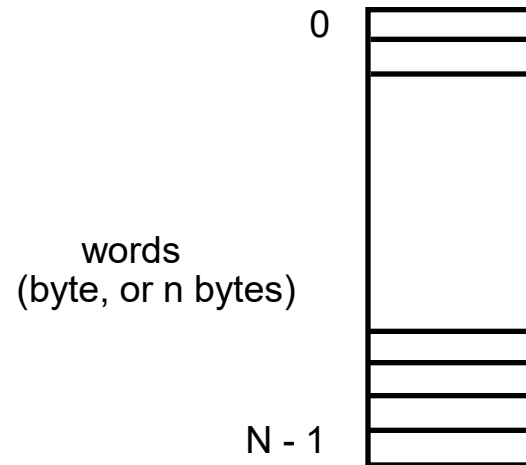
Unit-5 Syllabus

Memory System 2 2

RAM, ROM, EPROM, EEPROM, concepts of DRAM, SRAM, and Cache memory.

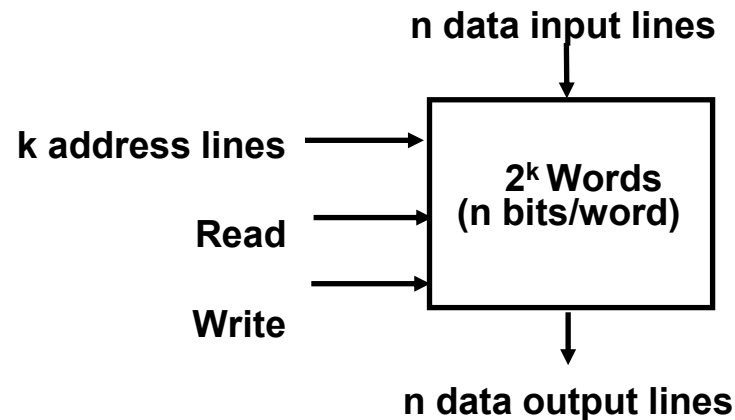
MEMORY COMPONENTS

Logical Organization



Random Access Memory

- Each word has a unique address
- Access to a word requires the same time independent of the location of the word
- Organization

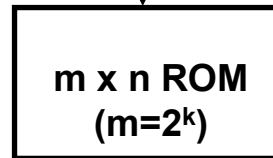


READ ONLY MEMORY(ROM)

Characteristics

- Perform read operation only, write operation is not possible
- Information stored in a ROM is made permanent during production, and cannot be changed
- Organization

k address input lines



n data output lines

Information on the data output line depends only on the information on the address input lines.

--> Combinational Logic Circuit

$$\begin{aligned} X_0 &= A'B' + B'C \\ X_1 &= A'B'C + A'BC' \\ X_2 &= BC + AB'C' \\ X_3 &= A'BC' + AB' \\ X_4 &= AB \end{aligned}$$

Canonical minterms

$$\begin{aligned} X_0 &= A'B'C' + A'B'C + AB'C \\ X_1 &= A'B'C + A'BC' \\ X_2 &= A'BC + AB'C' + ABC \\ X_3 &= A'BC' + AB'C' + AB'C \\ X_4 &= ABC' + ABC \end{aligned}$$

address ABC	Output				
	X_0	X_1	X_2	X_3	X_4
000	1	0	0	0	0
001	1	1	0	0	0
010	0	1	0	1	0
011	0	0	1	0	0
100	0	0	1	1	0
101	1	0	0	1	0
110	0	0	0	0	1
111	0	0	1	0	1

TYPES OF ROM

ROM

- Store information (function) during production
- Mask is used in the production process
- Unalterable
- Low cost for large quantity production --> used in the final products

PROM (Programmable ROM)

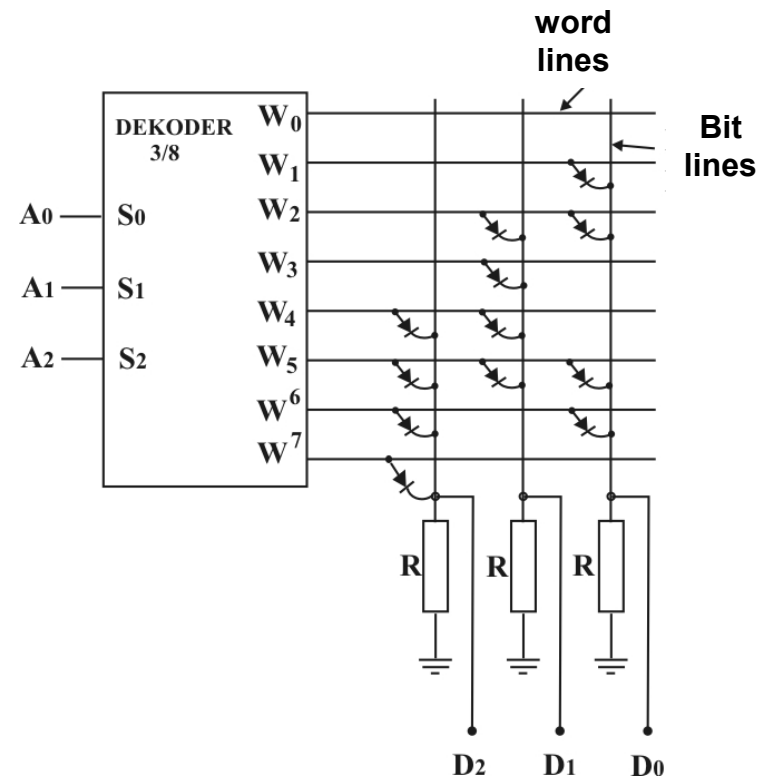
- Store info electrically using PROM programmer at the user's site
- Unalterable
- Higher cost than ROM -> used in the system development phase
-> Can be used in small quantity system

EPROM (Erasable PROM)

- Store info electrically using PROM programmer at the user's site
- Stored info is erasable (alterable) using UV light (electrically in some devices) and rewriteable
- Higher cost than PROM but reusable --> used in the system development phase. Not used in the system production due to eras ability

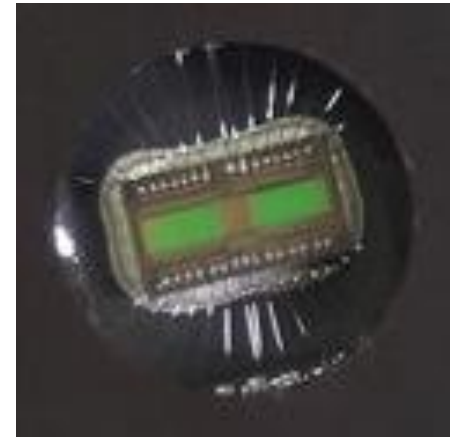
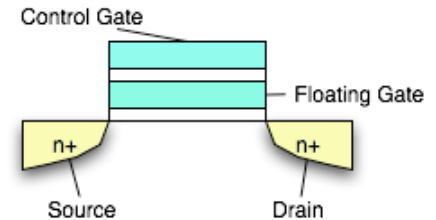
Mask ROM

- The “simplest” memory technology
- Presence/absence of diode at each cell denote value
- Pattern of diodes defined by mask used in fab process
- Contents are fixed when chip is made; cannot be changed
- High upfront setup costs (mask costs)
- Small recurring marginal costs
- Good for applications where
 - Cost sensitivity drives design
 - Upgrading contents not an issue
 - e.g. boot ROM, CPU microcode
- Exercise:
 - What “value” does a diode encode?
 - What are the contents:
 - Where $A_{<2:0>} = 101$?
 - Where $A_{<2:0>} = 110$?



EPROM

- Erasable Programmable Read-Only Memory
- Constructed from floating gate FETs
 - Charge trapped on the FG erases cell
 - High voltage (13V +) applied to the control gate
 - “Writes” the cell with a 0
 - Allows FG charge to be dissipated
- Erasing means changing from 0 $\rightarrow$ 1
 - Uses UV light (not electrically!)
 - Electrons are trapped on a floating gate
- Writing means changing from 1 $\rightarrow$ 0
- Erase unit is the whole device
- Retains data for 10-20 years
- Not used much these days
- Costly because
 - Use of quartz window (UV transparent)
 - Use of ceramic package
- PROM (or OTP) is same, just w/o window



Flash Memory

- Electrically erasable (like EEPROM, unlike EPROM)
- Used in many reprogrammable systems these days
- Erase size is block (not word); can't do byte modifications
- Erase circuitry moved out of cells to periphery
 - Smaller size
 - Better density
 - Lower cost
- Reads are like standard RAM
- Can “write” bits/words (actually, change from 1 $\rightarrow$ 0)
 - Write cycle is O(microseconds)
 - Slower than RAM but faster than EEPROM
 - To (re)write from 0 $\rightarrow$ 1, must explicitly erase entire block
 - Erase is time consuming O(milliseconds to seconds)
- Floating gate technology
 - Erase/write cycles are limited (10K to 100K, typically)

Static RAM (SRAM) and Dynamic RAM (DRAM)

Feature	Static RAM (SRAM)	Dynamic RAM (DRAM)
Full form	Static Random Access Memory	Dynamic Random Access Memory
Storage principle	Uses flip-flops (latches)	Uses capacitors
Data refresh	Not required	Required periodically (refresh needed)
Speed	Very fast	Slower compared to SRAM
Complexity	More complex circuitry	Simpler circuitry
Cost	Expensive	Cheaper
Power consumption	Higher (per bit) but no refresh power	Lower per bit but needs refresh power
Density (storage capacity)	Low density	High density
Chip size	Larger	Smaller
Typical use	Cache memory (L1, L2, L3)	Main memory (RAM in computers)
Data retention	Holds data as long as power is ON	Loses data quickly without refresh

Cache Memory

Cache Memory

Cache memory is a small, very high-speed memory located near the CPU. It stores frequently used data and instructions to reduce access time to RAM. It improves CPU performance by reducing waiting time.

Working:

CPU first checks cache memory

Cache Hit → Data found (fast access)

Cache Miss → Data fetched from RAM

Levels:

L1 Cache → Fastest, smallest (inside CPU core)

L2 Cache → Medium speed and size

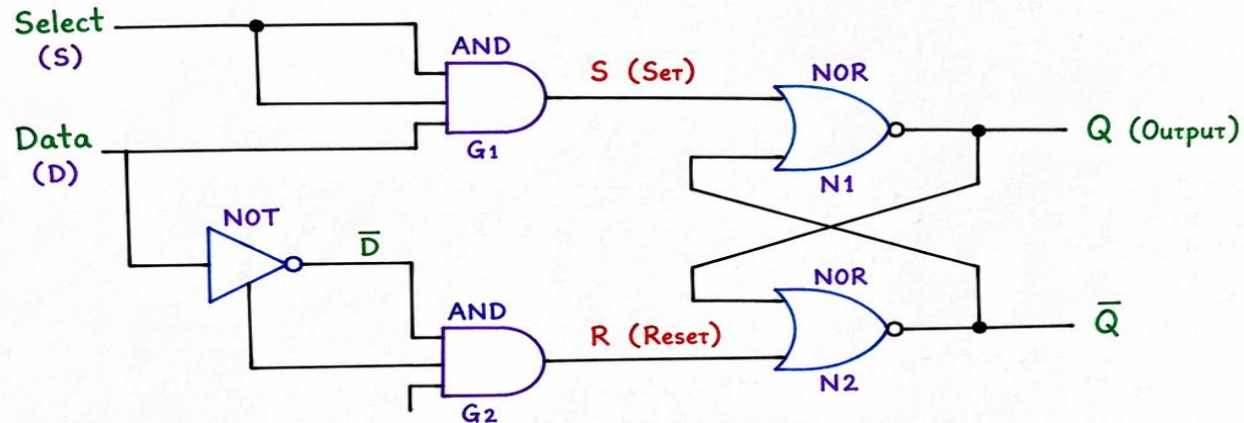
L3 Cache → Largest, shared cache

Memory Hierarchy:

CPU → Cache → RAM → Storage

RAM CELL USING SR LATCH

LOGIC DIAGRAM :



TRUTH TABLE :

Select (S)	Data (D)	S	R	Q (Next)
0	X	0	0	No Change (Memory Hold)
1	0	0	1	0
1	1	1	0	1

WORKING :

- When Select (S) = 0 :
S = 0 and R = 0, so the SR latch keeps its previous state.
Hence, the cell holds the data.
- When Select (S) = 1 :
If Data (D) = 1 $\rightarrow$ S = 1, R = 0
SR latch is SET, so Q = 1.
If Data (D) = 0 $\rightarrow$ S = 0, R = 1
SR latch is RESET, so Q = 0.

KEY POINT :

A RAM cell using SR latch stores one bit of data. The Select line enables the cell for write operation, and when not selected, the latch retains (memorizes) the data.



UNIT 2 – An Overview of Boolean Algebra

- Boolean algebra and Boolean theorems, Logic gates, and implementation of Boolean functions with various logic gates. Circuit equivalence, Venn diagram, Simplification of the logic up to four variables using the K-map method.



Digital Logic circuits

- Digital Logic circuits operate in the binary mode where each input and output is either 0 or 1.
- The 0 and 1 do not represent actual numbers but instead represent the state of the voltage variable or logic level.
- So in the digital logic field, several other terms are used synonymously with 0 and 1.

LOGIC 1	LOGIC 0
HIGH	LOW
TRUE	FALSE
ON	OFF
+5V	0V



Logic Gates

- The most basic digital devices are called gates.
- Gates got their name from their function of allowing or blocking (gating) the flow of digital information.
- A gate has one or more inputs and produces an output depending on the input(s).
- A gate is called a combinational circuit.
- Three most important gates are: AND, OR, NOT





What is Logic Gate?

- Digital gate is a Digital Device used to perform the logic operation
- Logic gates (or simply gates) are the fundamental building blocks of digital circuitry
- Electronic gates require a power supply.
Gate INPUTS are driven by voltages having two nominal values,
e.g. 0V and 5V representing logic 0 and logic 1 respectively.

The OUTPUT of a gate provides two nominal values of voltage only,
e.g. 0V and 5V representing logic 0 and logic 1 respectively.

- In general, there is only one output to a logic gate except in some special cases.





Truth table :

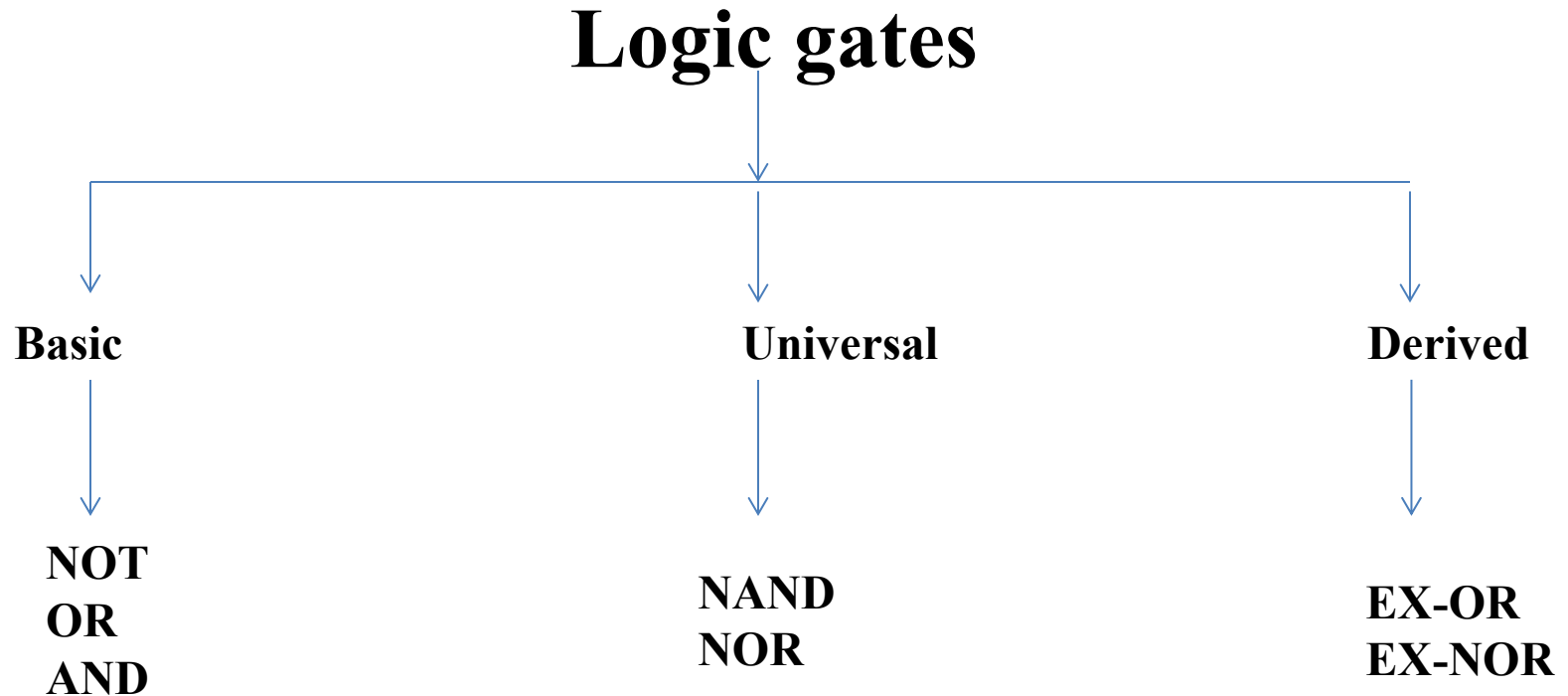
- The operation of a logic gate or a logic circuit can be best understood with the help of a table called "Truth Table". The truth table consists of all the possible combinations of the inputs and the corresponding state of output produced by that logic gate or logic circuit.

Boolean expression :

- The relation between the inputs and the outputs of a gate can be expressed mathematically by means of the Boolean Expression.

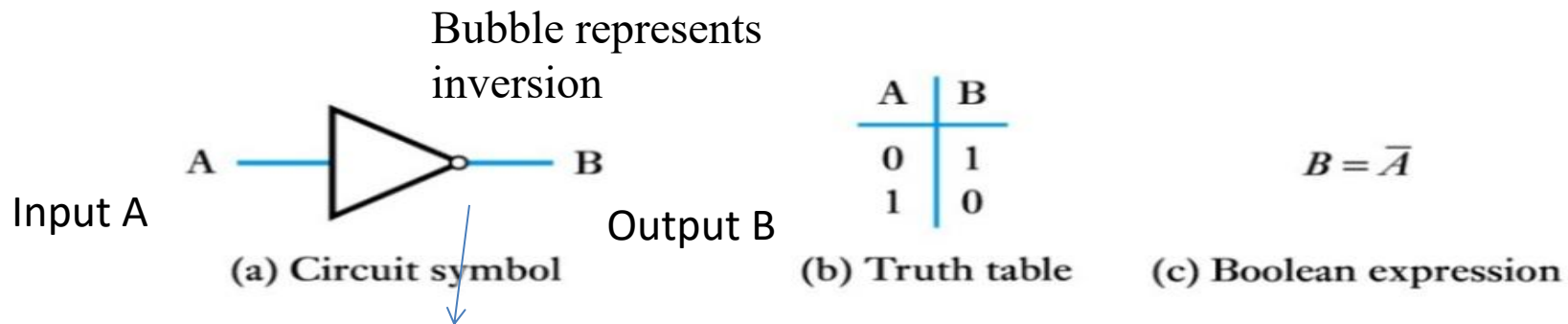


Classification of Logic Gates



Basic gates with their symbols and truth table

- 1) The NOT gate or Inverter is a logic gate having one input (A) and one output (Y). Its symbol and truth table are shown in Fig. below
- 2) The NOT gate is also known as an "Inverter" because its output is the inverted version or "complement" of its input. This is shown in the truth table of NOT gate.
- 3) The bubble (o) in the symbol of a NOT gate indicates the inversion operation



AND gate with its symbol and truth table

- 1) AND is one of the logic operators. It performs the logical multiplication on its inputs.
- 2) The output is high ($Y = 1$) if and only if all the inputs to the AND gate are high (1). The output is low (0), if any one or more inputs are low (0). AND gate can have two or more inputs and only one output.
- 3) The logical symbol of a two input AND gate is as shown below
- 4) Boolean expression : The expression relating the inputs (A, B) and output (Y) of a gate is called as the expression. The Boolean expression for an AND gate is,

$$Y = A \cdot B$$

where the "dot" between A and B represents multiplication.



(a) Circuit symbol

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

(b) Truth table

$$C = A \cdot B$$

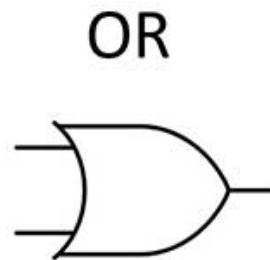
(c) Boolean expression



OR gate with its symbol and truth table

- 1) OR is one of the logic operators. It performs the logical addition on its inputs.
- 2) The output is high ($Y = 1$) if any one or all the inputs are high (1). The output is low (0), if both the inputs are low. The logical symbol of a two input OR gate is as shown below
- 3) Boolean expression : The expression relating the inputs (A, B) and output (Y) of a gate is called as the expression. The Boolean expression for an AND gate is,

$$Y = A + B$$



A	B	Output
0	0	0
1	0	1
0	1	1
1	1	1

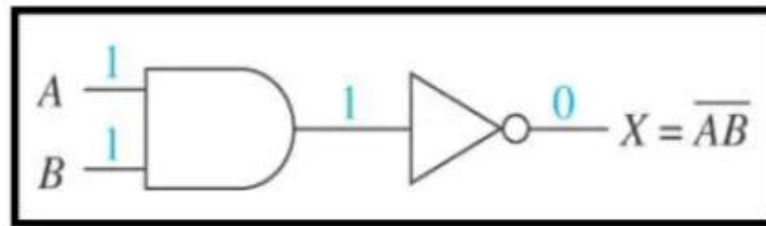
Universal Gates

What are Universal gates ?

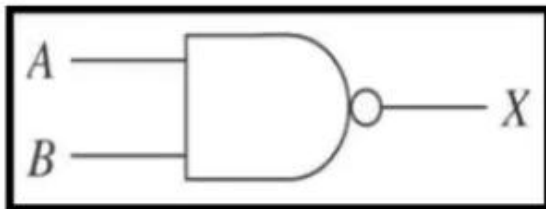
1. The **NAND** and **NOR** gates are called as “**Universal gates**” because **it is possible to implement any Boolean expression** with the help of only NAND or NOR gates.
2. Therefore **a user can build any logical circuit** with the help of **only NAND or only NOR gates**.
3. This is a great advantage because a user will have to make a stock of only NAND or NOR gates ICs with him.



What happens when you add a NOT to an AND gate?



“Not AND” = NAND



A	B	X
0	0	1
0	1	1
1	0	1
1	1	0



- 1) The term NAND can be split as NOT-AND which means that the NAND operation can be implemented with the combination of an AND gate and a NOT gate i.e. inverter.
- 2) Thus a NAND gate is equivalent to an AND gate followed by an inverter as shown in Fig. below
- 3) The truth table of a two input NAND gate is shown below which shows that the output is low (0) if and only if both the inputs are high (1) simultaneously. For all other input combinations the output voltage will be high (1).
- 4) A NAND gate is called as "Universal Gate" because we can construct AND, OR and NOT gates using only NAND gates.

5)

- **The NAND gate**



(a) Circuit symbol

A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

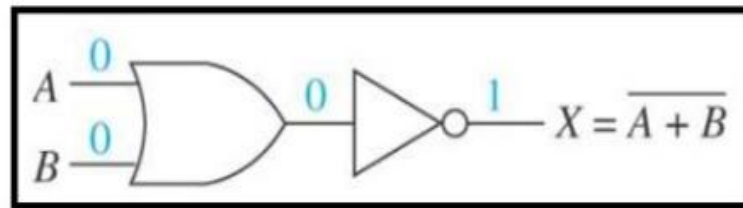
(b) Truth table

$$C = \overline{A \cdot B}$$

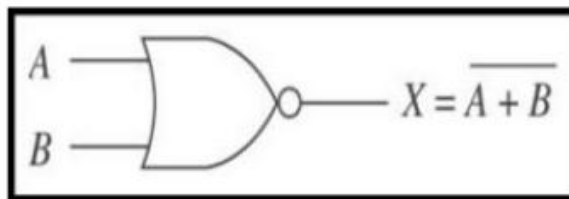
(c) Boolean expression



What happens when you add a NOT to an OR gate?



“Not OR” = NOR



A	B	X
0	0	1
0	1	0
1	0	0



- 1) This is another universal gate.
- 2) The word NOR can be split as NOT-OR which means that a NOR operation can be implemented with the combination of an OR gate and a NOT gate, i.e. inverter. Thus a NOR gate is equivalent to an OR gate followed by an inverter as shown in Fig.
- 3) The symbol of a two input NOR gate is shown in Fig. 3.10.1(a), where a bubble (o) on the output side represents inversion. The truth table of a two input NOR gate is shown in
- 4) Fig. shows that "the output of a NOR gate is high (1) if and only if all its inputs are low (0) simultaneously". The output of a two input NOR gate is low (0) if any one or all the inputs are at high (1) level.



(a) Circuit symbol

A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

(b) Truth table

$$C = \overline{A + B}$$

(c) Boolean expression



EX-OR and EX-NOR Gates :

- EX-OR and EX-NOR gates are special type of gates. They can be used for applications such as half adder, full adder and subtractors.
- These gates are also called as Derived gates



1. The exclusive-OR gate is abbreviated as EX-OR gate or sometimes as X-OR gate.
2. An EX-OR gate having two input terminals and one output terminal is shown in Fig.
3. The truth table of a two input EX-OR gate is shown in Fig, which shows that, when both the inputs are at identical logic levels ($A = B$), the output is low (0) i.e. $Y = 0$ for $A = B = 0$ or $A = B = 1$, and the output is high (1) when A and B are not equal.

11

• The Exclusive OR gate

- Exclusive OR gate are true if either input is true but not both.
- The **Symbol** and **Truth Table** is shown below.
- **Output will be high for different inputs**



(a) Circuit symbol

A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

(b) Truth table

$$C = A \oplus B$$

(c) Boolean expression





- 1) The word EX-NOR is a short form of exclusive-NOR. Exclusive-NOR means NOT-exclusive OR, so EX-NOR gate is equivalent to an EX-OR gate followed by a NOT gate.
- 2) The symbol for a two input EX-NOR gate is as shown in Fig. and its truth table is given in Fig. which shows that the output of an EX-NOR gate is high (1) if both the inputs are identical ($A = B$) and the output is low (0) if the inputs are not identical ($A \neq B$).
- 3) The Boolean expression for an EX-NOR gate is given by,

• The Exclusive NOR gate

- The **Symbol** and **Truth Table** is shown below.
- **Output will be high for same inputs**



(a) Circuit symbol

A	B	C
0	0	1
0	1	0
1	0	0
1	1	1

(b) Truth table

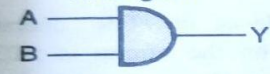
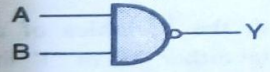
$$C = \overline{A \oplus B}$$

(c) Boolean expression



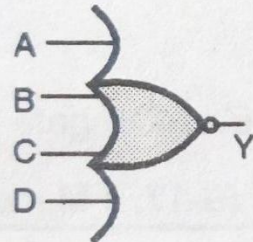


Q. Draw symbols and logic expressions for all gates

Sr. No.	Name of gate	Boolean expression	Logical operation
1.	NOT gate or inverter 	$Y = \bar{A}$	Inversion
2.	AND gate 	$Y = AB$	Logical multiplication
3.	OR gate 	$Y = A + B$	Logical addition
4.	NAND gate 	$Y = \overline{AB}$	NOT AND
5.	NOR gate 	$Y = \overline{A + B}$	NOT OR
6.	Exclusive OR 	$Y = A \oplus B$	Addition / Subtraction
7.	Exclusive NOR 	$Y = \overline{A \oplus B}$	NOT EXOR



Symbols :



Truth table

Inputs			Output
A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Output is low
if at least one
input is 1

(a)

(b) Three input NOR gate

(B-472) Fig. 3.14.4 : Symbols, Boolean equations and truth table for multiple input NOR gate

Boolean equations :

$$Y = \overline{A + B + C}$$

$$Y = \overline{A + B + C + D}$$



NAND Gate as Universal gate

- NOT gate using NAND gate

1. NOT gate (Inverter) using NAND gate :

- Fig. 3.15.1(a) shows the realization of a NOT gate (inverter) using a two input NAND gate.
- As both the inputs of the NAND are connected together, we can write that,

$$\text{Input} = A = B = A$$

- So output is given by,

$$Y = \overline{A \cdot B} = \overline{A \cdot A} \quad \dots \text{since } A = B = A$$

$$\text{But } A \cdot A = A \quad \therefore Y = \bar{A}$$

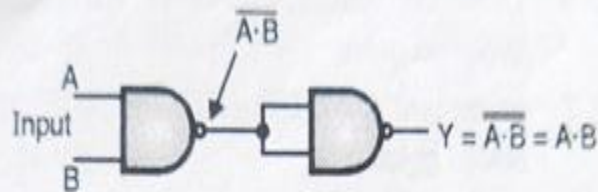


(B-505) Fig. 3.15.1(a) : NOT using NAND

2. AND gate using NAND :

$$Y = A \cdot B \quad \dots \text{AND gate}$$

$$Y = \overline{\overline{A \cdot B}} \quad \dots \text{Double inversion}$$



(B-506) Fig. 3.15.1(b) : AND gate using NAND

$$\therefore Y = A \cdot B = \overline{\overline{A \cdot B}} \quad \dots (3.15.1)$$

- Equation (3.15.1) can be realized using only NAND gates as shown in Fig. 3.15.1(b).

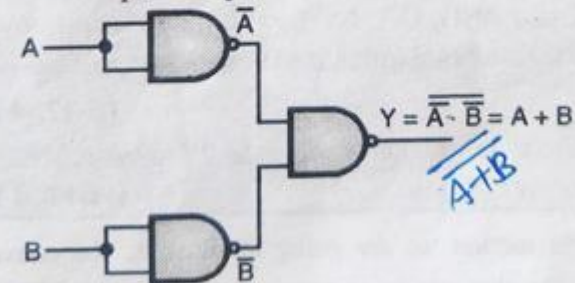
3. OR gate using NAND :

$$Y = A + B \quad \dots \text{OR gate}$$

$$Y = \overline{\overline{A + B}} \quad \dots \text{Double inversion}$$

$$\therefore Y = \overline{\overline{A} \cdot \overline{B}} \quad \dots \text{De Morgan's law}$$

This is the required expression.



(B-507) Fig. 3.15.1(c) : OR gate using NAND gates

- So $Y = A + B = \overline{\overline{A} \cdot \overline{B}}$ and Fig. 3.15.1(c) shows the realization.

NOR Gate as Universal gate

1. NOT gate (Inverter) using NOR :

- Fig. 3.15.2(a) shows the realization of a NOT gate using only NOR gate.
- As both the inputs of the NOR gate are connected together, we can write that,

$$A = B = A$$

- So output of NOR is given by,

$$Y = \overline{A + B} = \overline{A + A}$$

- But $A + A = A$

$$\therefore Y = \overline{A}$$



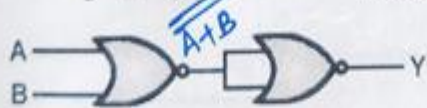
(B-511) Fig. 3.15.2(a) : NOT gate using NOR gate

- This is the Boolean expression of an inverter. So Fig. 3.15.2(a) indeed represents an inverter.

2. OR using NOR :

$$Y = A + B \quad \dots \text{OR gate}$$

$$Y = \overline{\overline{A + B}} \quad \dots \text{Double inversion}$$



(B-512) Fig. 3.15.2(b) : OR using only NOR gates

This is the required expression.

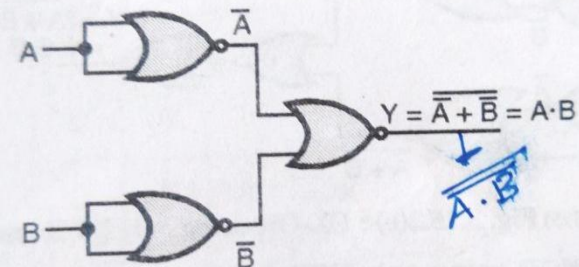
- Fig. 3.15.2(b) shows the realization of OR gate using only NOR gates.

3. AND using only NOR gates :

$$Y = A \cdot B \quad \dots \text{AND gate}$$

$$Y = \overline{\overline{A \cdot B}} \quad \dots \text{Double inversion}$$

- This is the required expression, Fig. 3.15.2(c) shows the realization of AND gate using NOR gates only.



(B-513) Fig. 3.15.2(c) : AND gate using only NOR gates

NAND gate using only NOR gates :

- Boolean expression for a NAND gate is,

$$Y = \overline{A \cdot B} = \overline{A} + \overline{B} \quad \dots \text{using De Morgan's theorem}$$

- Take double inversion of RHS to get,

$$Y = \overline{\overline{\overline{A} + \overline{B}}}$$

- This is the required expression. Fig. 3.15.2(d) shows the realization of NAND gate using only NOR gates.



Boolean Laws

Boolean laws used to reduce Boolean Expression

1. Commutative law.
2. Associative law.
3. Distributive law.
4. OR law.
5. AND law
6. INVERSION law





- Boolean Algebra is used to analyze and simplify the digital (logic) circuits. Since it uses only the binary numbers i.e. 0 and 1 it is also called as "Binary Algebra", or "Logical Algebra".
- It was invented by George Boole in the year 1854.

Rules in boolean algebra :

- There are some rules to be followed while using a Boolean algebra, these are:
 1. Variables used can have only two values. Binary 1 for HIGH and Binary 0 for LOW.
 2. Complement of a variable is represented by a overbar (-). Thus complement of variable B is represented as $\overline{B}$. Thus if $B=0$ then $\overline{B} = 1$ and if $B = 1$ then $\overline{B} = 0$.
 3. ORing of the variables is represented by a plus (+) sign between them. For example ORing of A, B, C is represented as $A + B + C$.
 4. Logical ANDing of the two or more variables is represented by writing a dot between them such as A B C.D.E. Sometimes the dot may be omitted like ABCDE.





Commutative Law:

- Any binary operation which satisfies the following expression is known as commutative operation :
 1. $A.B=B.A$
 2. $A+B=B+A$
- Thus the commutative law states that we can change the sequence of the variables (inputs) without having any effect on the output of a logic circuit.



Associative Law:

- This law states that the order in which the logic operations are performed is not at all important ultimate outcome is the same.

i.e. $(A.B) C = A (B.C)$ and
 $(A+B) + C = A + (B+C)$





Distributive Law:

- The distributive law states that,
$$A (B+C) = AB+ AC$$



AND Laws :

- These laws are related to the AND operation therefore they are called as "AND" laws. The AND laws are as follows:

1. $A \cdot 0 = 0$:

That means if one input of an AND operation is permanently kept at the LOW (0) level, then the output is zero irrespective of the other variable.

2. $A \cdot 1 = A$:

- That means if one input of an AND operation is HIGH (1) permanently then the output is always equal to the other variable.

3. $A \cdot A = A$

- That means if both the inputs in an AND operation have the same value either "0" or "1" then the output will also have the same value as that of the input.

4. $A \cdot \bar{A} = 0$

This law states that the result of an "AND" operation on a variable (A) and its complement ($\bar{A}$) is always LOW (0)



OR laws

- These laws use the OR operation. Therefore they are called as OR laws. The OR laws are as follows:

1. $A + 0 = A$:

- That means if one variable of an "OR" operation is LOW (0) permanently, then the output is always equal to the other variable.

2. $A + 1 = 1$:

- That means if one variable of an "OR" operation is HIGH (1) permanently, then the output is HIGH (1) permanently irrespective of the value of the other variable.

3. $A + A = A$:

- This law states that if both the variables of an OR operation have the same value either "0" or "1" then the output also will be equal to the input i.e. 0 or 1 respectively.

4. $A + \overline{A} = 1$:

- This law states that the result of an "OR" operation on a variable and its complement is always 1 (HIGH).





INVERSION Law

- This law uses the “NOT” operation. The inversion law states that if a variable is subjected to a double inversion then it will result in the original variable itself.

$$\overline{\overline{A}} = A$$



Other Important Rules

1. $A + AB = A$

$$\text{LHS} = A + AB$$

$$= A(1 + B)$$

$$\text{But } 1 + B = 1$$

$$\therefore \text{LHS} = A \cdot 1 = A$$

$$\text{Since } A \cdot 1 = A$$

$$\therefore A + AB = A$$

Hence proved.

2. $A + \overline{AB} = A + B$

$$\text{LHS} = A + \overline{AB}$$

$$\text{Substitute } A = A + AB$$

$$\text{LHS} = A + AB + \overline{AB} = A + B(A + \overline{A})$$

$$\text{But } A + \overline{A} = 1$$

$$\therefore \text{LHS} = A + B(1) = A + B$$

$$\therefore A + \overline{AB} = A + B$$

Hence proved.



- 3. $(A + B) (A + C) = A + BC :$

$$\begin{aligned}\text{LHS} &= (A + B) (A + C) \\ &= A . A + A . C + B . A + B . C\end{aligned}$$

$$\text{But } A . A = A \text{ and } B . A = A . B$$

$$\therefore \text{LHS} = A + AC + AB + BC$$

$$\text{But } A + AB = A$$


$$\therefore \text{LHS} = A + AC + BC = A (1 + C) + BC$$

$$\text{But } (1 + C) = 1$$


$$\therefore \text{LHS} = A + BC$$

$$\therefore (A + B) (A + C) = A + BC$$

...Proved.



Duality theorem.

*State and explain Duality Theorem****

1. According to the duality theorem the following conversions are possible in a given Boolean expression :
 1. **Change each AND operation to an OR operation.**
 2. **Change each OR operation to an AND operation.**
 3. **Complement any 1 or 0 appearing in the expression.**
2. Duality theorem is sometimes useful in creating new expressions from the given Boolean expressions.
3. For example if the given expression is $A + 1 = 1$ then replace the OR (+) operation by AND () operation and take complement of 1 to write the dual of the given relation as,
 $A.0 = 0$
4. The dual of $A.(B+C) = AB + AC$ is given by,
 $A+(B.C) = (A + B).(A + C)$



De-Morgan's Theorem

*Q. State and Prove De-Morgans Theorem******

Theorem 1:

$$\overline{A.B} = \overline{A} + \overline{B}$$

1. This theorem states that the complement of the product is equal to the addition of the complement of each term.
2. This can be shown as follows. The left hand side of this theorem represents a NAND gate with inputs A and B whereas The Right Hand Side represents OR gate with inverted inputs.
3. This theorem can be verified by writing a truth table as follows

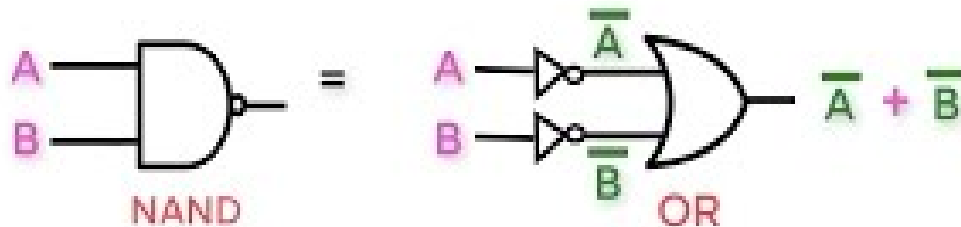


Illustration of De-Morgans First Theorem

A	B	$\overline{A.B}$	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

Truth table



- Theorem 2:**

$$\overline{A+B} = \overline{A} \cdot \overline{B}$$

1. This theorem states that the complement of the addition is equal to the product of the complement of each term.
2. This can be shown as follows. The left hand side of this theorem represents a NOR gate with inputs A and B whereas The Right Hand Side represents AND gate with inverted inputs.
3. This theorem can be verified by writing a truth table as follows

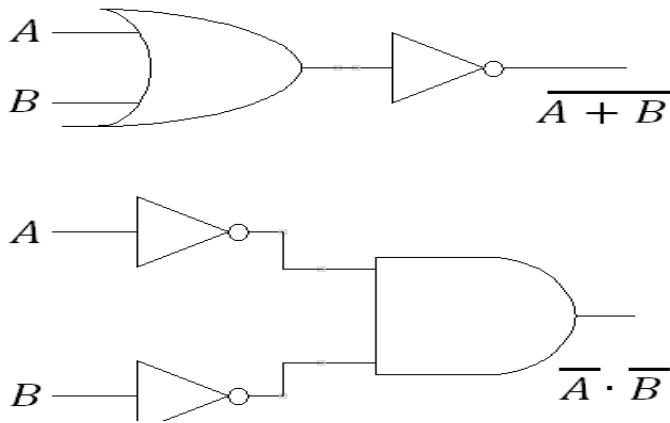


Illustration of De-Morgans First Theorem

A	B	$\overline{A+B}$	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1	1	1
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	0

Truth table



Example 1

Problem: Simplify the Boolean expression $A \cdot B + A \cdot \overline{B}$.

Example 2

Problem: Simplify the Boolean expression $A + A \cdot B$.

Example 3

Problem: Simplify the Boolean expression $A \cdot \overline{A}$.

Example 4

Problem: Simplify the Boolean expression $(A + B) \cdot (A + \overline{B})$.

Problem: Simplify the Boolean expression $A + \overline{A} \cdot B$.

Example 8

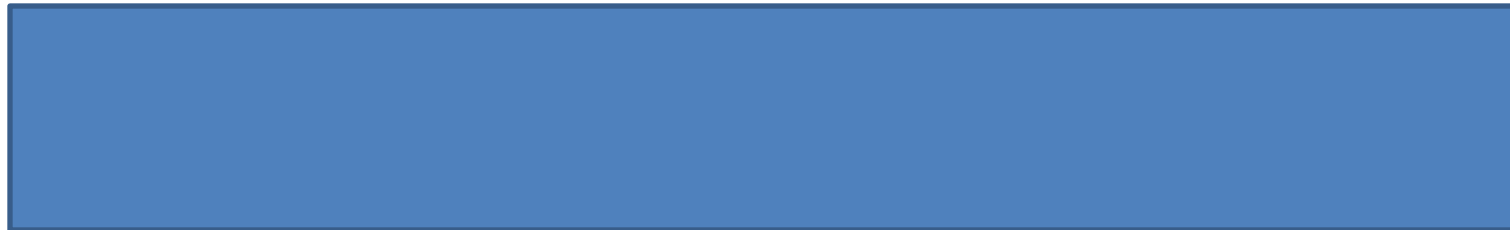
Problem: Simplify the Boolean expression $A \cdot (A + B)$.

Example 9



Example 5

Problem: Simplify the Boolean expression $(A \cdot B) + (\bar{A} \cdot B)$.



Example 6

Problem: Simplify the Boolean expression $A \cdot B + \bar{A} \cdot \bar{B} + A \cdot \bar{B} + \bar{A} \cdot B$.





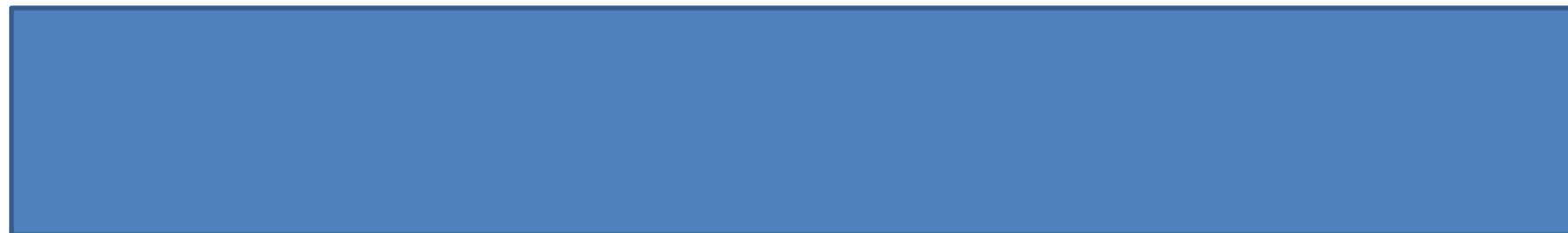
Example 10

Problem: Simplify the Boolean expression $A \cdot (A + \overline{A} \cdot B)$.



Example 11

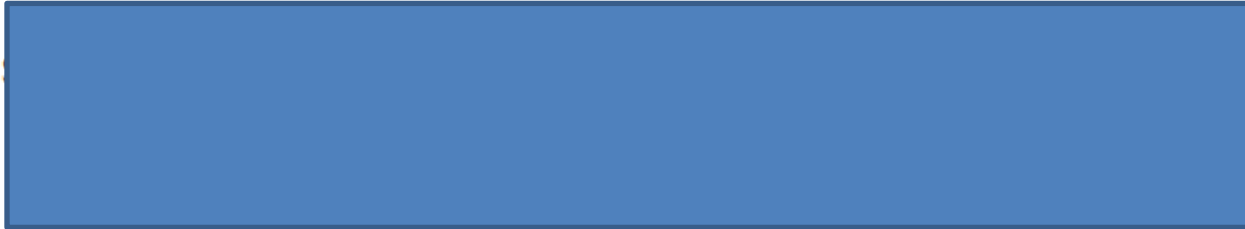
Problem: Simplify the Boolean expression $(A \cdot B) + (\overline{A} \cdot \overline{B})$.





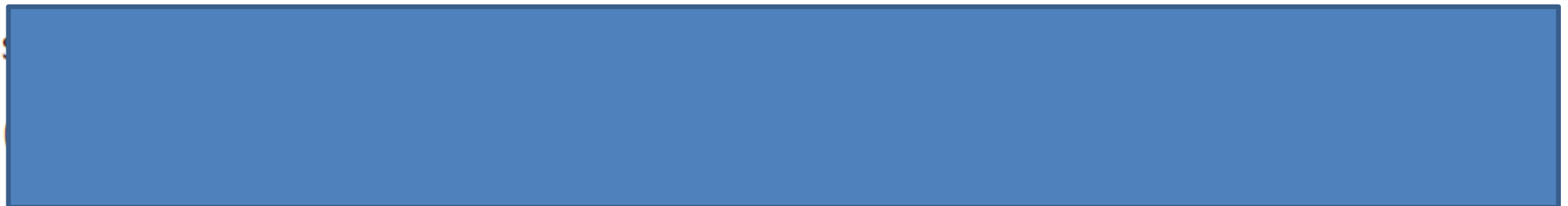
Example 12

Problem: Simplify the Boolean expression $A \cdot (B + \overline{B}) \cdot (C + \overline{C})$.



Example 13

Problem: Simplify the Boolean expression $(A + B) \cdot (A + \overline{B}) \cdot (A + C)$.





Example 14

Problem: Simplify the Boolean expression $A \cdot B + A \cdot C + B \cdot C$.



Example 15

Problem: Simplify the Boolean expression $A \cdot \overline{B} + \overline{A} \cdot B \cdot C$.





Example 1: Simplify the Boolean Expression

Expression: $A \cdot \overline{A} + B \cdot (A + \overline{A})$

Solution:





Expression: $(A + \overline{B}) \cdot (A + B)$



Final Simplified Expression: A





Example 3: Simplify the Boolean Expression

Expression: $\overline{A \cdot B} + A$

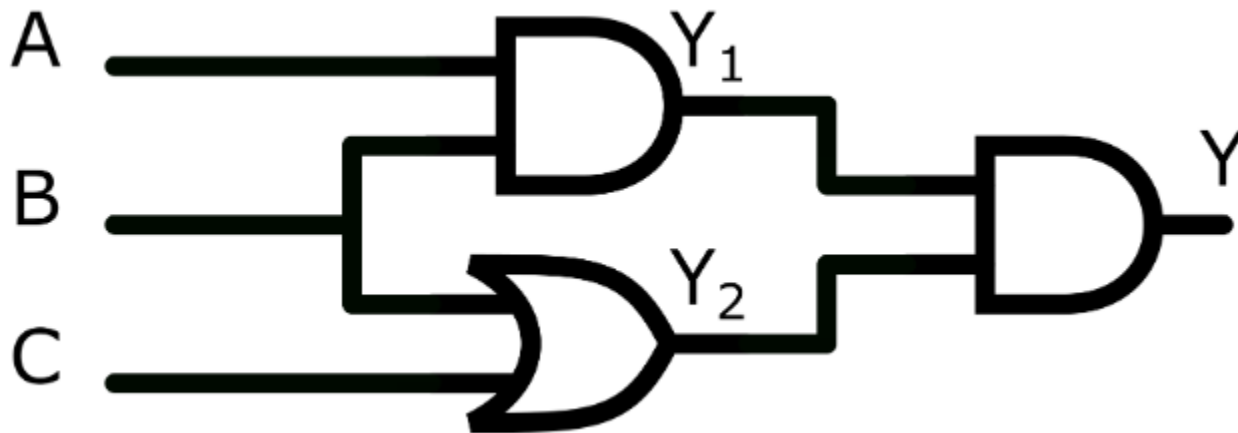
Solution:

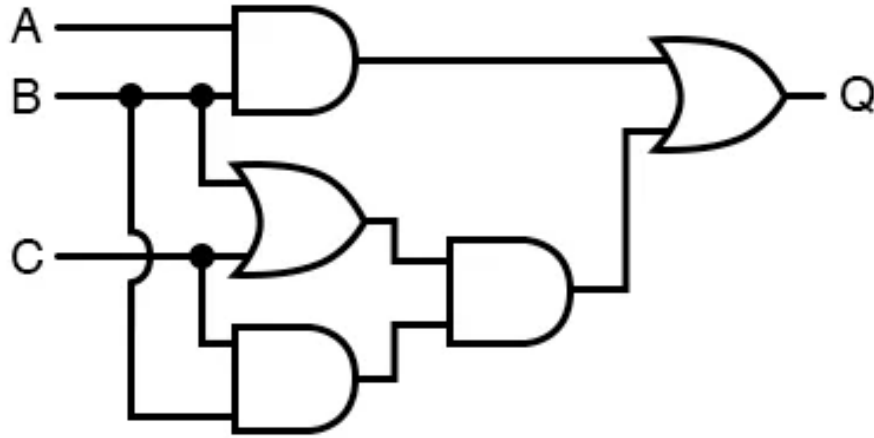


Final Simplified Expression: 1

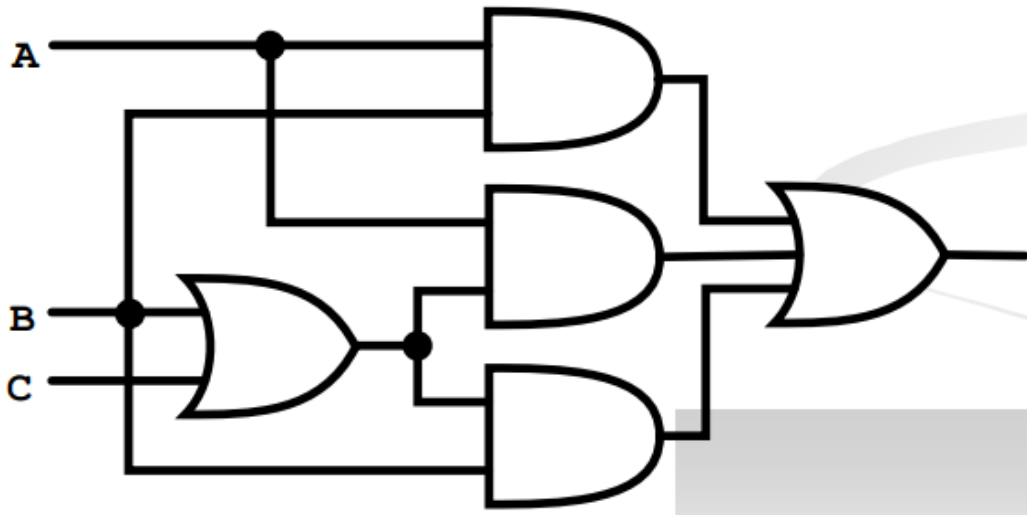


Write the Boolean expression and
simplify it





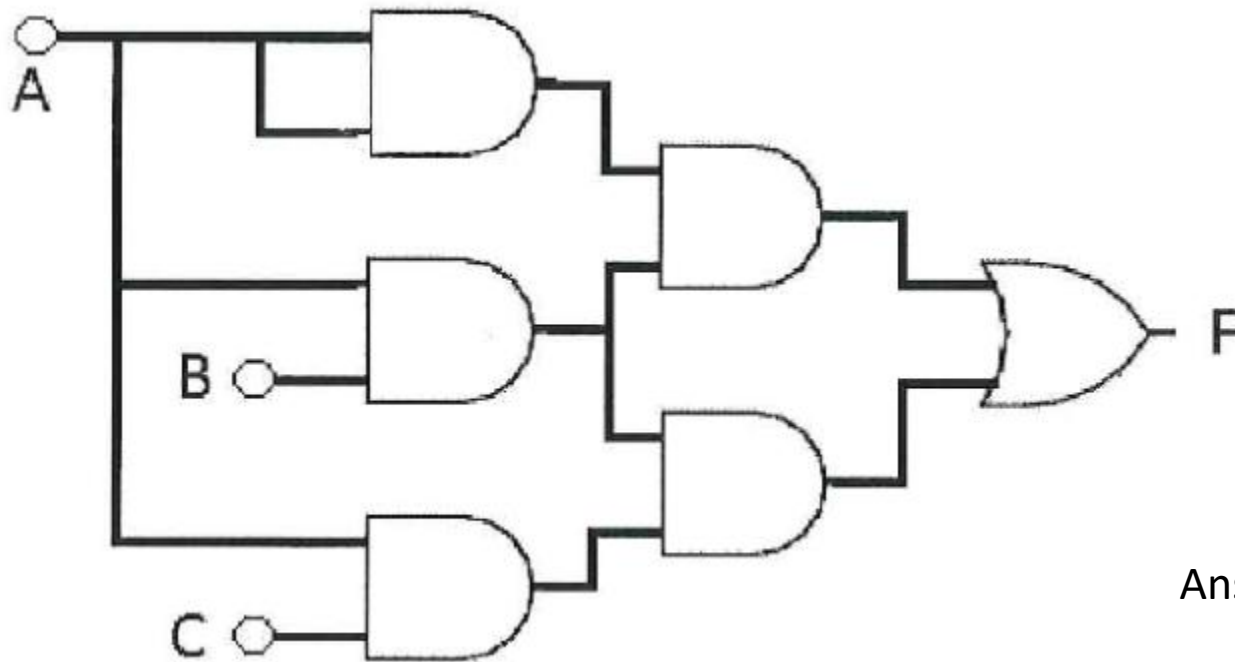
$$\text{Ans} = B(A+C)$$



$$\text{Ans} = B + (A.C)$$



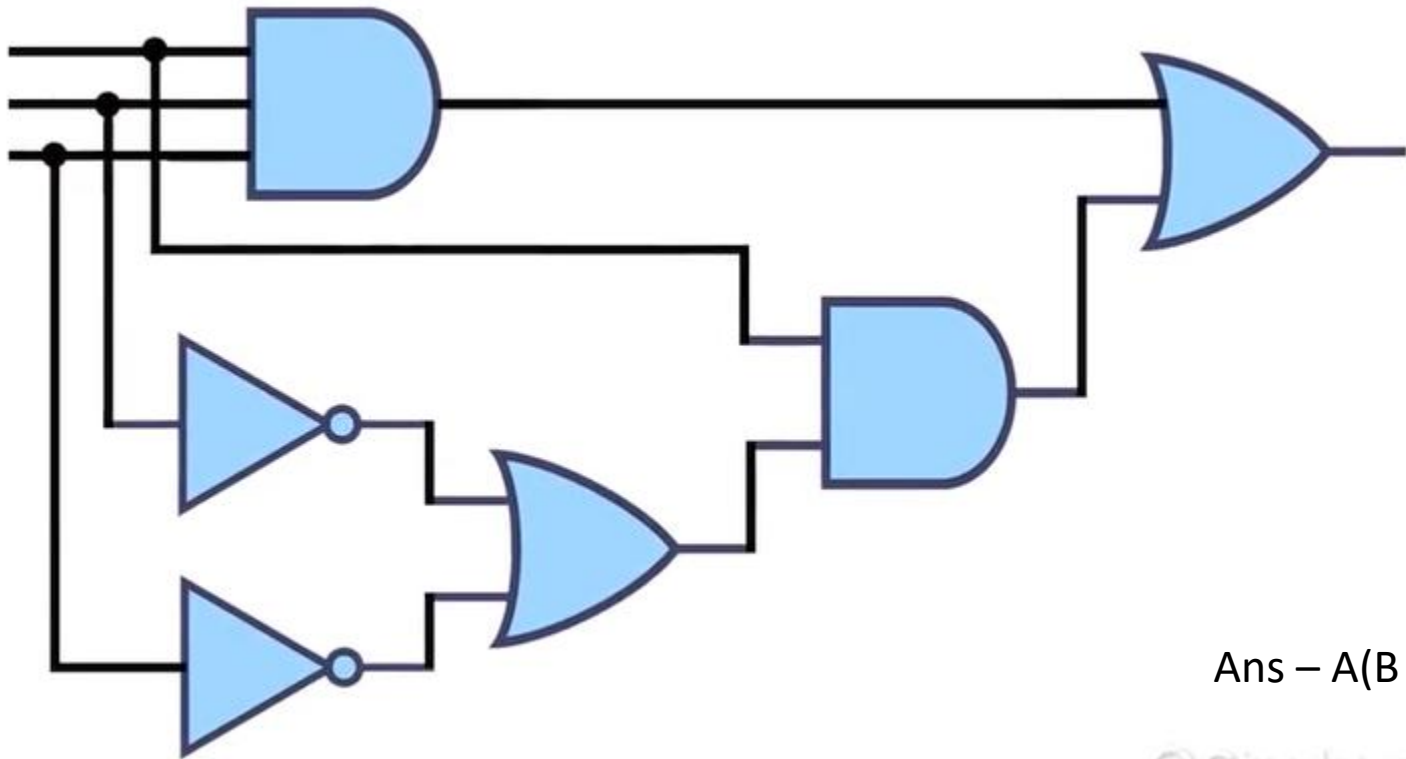
Simplify the following



Ans = $A.B$



Simplify the following



Ans – $A(B + C)$



SOP and POS Representations for Logic Expressions

Consider that the logic expression given to us is as follows:

$$Y = AC + BC$$

- In this Boolean expression, **Y is the result or output** and A, B, C are called as **literals**.
- Then it can be realized using basic gates.

Any logic expression can be expressed in the following two standard forms:

- **Sum-of-products form (SOP)** {Used when output is Logic-1}
- **Product-of-sums form (POS)** {Used when output is Logic-0}

These two forms are suitable for **reducing the given logic expression to its simplest form**.

Cont...

Sum-of-products form (SOP)

$$Y = ABC + BCD + ABD$$

$$A = XY + XY + X\tilde{Y}$$

$$Y = PO + POR + POR$$

Product-of-Sums form (POS)

$$Y = (A+B+C).(A + B).(A + C)$$

$$A = (X+Y).(X+Y+Z)$$

$$Y = (P+R).(P+Q).(P+R)$$

$$Z = (A+B).(C+D)$$

Cont...

- **Standard Form-In** this form each term may contain one, two or any number of literals. It is not necessary that each term should contain all the literals
- **Canonical Form-This** rule states that each term used in an equation **must contain all the available input variables.**

Sr.No	Expression	Type
1	$Y=AB+ \overline{A}BC + A\overline{B}C$	
2	$Y=AB+\overline{A}B+\overline{A}\overline{B}$	
3	$Y=(A+B).\overline{(A+B)}.(A+B)$	
4	$Y=(A+B).(A+B+C)$	



Cont...

⊗ Distribution Th^m :-

$$A + BC = (A + B)(A + C)$$

① ② ③
① + ② ① + ③

$$A + \bar{A}B = (A + \bar{A})(A + B) = (A + B)$$

$$A + \bar{A}\bar{B} = (A + \bar{B})$$

$$\bar{A} + AB = \bar{A} + B$$

$$\bar{A} + A\bar{B} = \bar{A} + \bar{B}$$

⊗ $(A + B)(A + C)$ $\Rightarrow$ $A + BC$ ~~A~~.

Multiply

e.g.:- $AB + A\bar{B} + \bar{A}B$

$$A(B + \bar{B}) + \bar{A}B = A + \bar{A}B = \boxed{A + B} \quad \text{A}$$

⊗ $AB + \bar{A}C + A\bar{B} = A(B + \bar{B}) + \bar{A}C$

$$A + \bar{A}C = \boxed{A + C} \quad \text{A}$$



SOP Form

Example :- Truth Table is given. Find out SOP form.

	A	B	Y	
(m ₀)	0	0	1	→ $\bar{A}\bar{B}$
(m ₁)	0	1	0	
(m ₂)	1	0	1	→ $A\bar{B}$
(m ₃)	1	1	1	→ AB

→ Canonical SOP.

$$Y(A, B) = \underbrace{\bar{A}\bar{B}}_{(m_0)} + \underbrace{A\bar{B}}_{(m_2)} + \underbrace{AB}_{(m_3)} = \sum m(0, 2, 3) \rightarrow \left\{ \text{Standard SOP} \right\}$$

$$= \bar{A}\bar{B} + A(\bar{B} + B) \Rightarrow A + \bar{A}\bar{B}$$

$$= (A + \bar{A})(A + \bar{B}) \Rightarrow \boxed{A + \bar{B}} \cdot A \quad \left\{ \text{Minimal SOP} \right\}$$



Cont...

Example:- $F(A, B, C) = \bar{A} + BC$ (Minimal Term SOP)

Now 1st, we write canonical form OK

$$= \bar{A} \cdot 1 \cdot 1 + 1 \cdot B \cdot C$$

$$= \bar{A} (B + \bar{B}) (C + \bar{C}) + (A + \bar{A}) BC$$

$$= (\bar{A}B + \bar{A}\bar{B}) (C + \bar{C}) + ABC + \bar{A}BC$$

$$= \underbrace{\bar{A}BC}_{(m_3)} + \underbrace{\bar{A}B\bar{C}}_{(m_2)} + \underbrace{\bar{A}\bar{B}C}_{(m_1)} + \underbrace{\bar{A}\bar{B}\bar{C}}_{(m_0)} + \underbrace{ABC}_{(m_7)} + \cancel{\underbrace{\bar{A}BC}_{(m_3) \text{ (Repeat)}}}$$

↗ Canonical SOP.

$$= \boxed{\sum m(0, 1, 2, 3, 7)} \quad \text{Ans. Standard SOP.}$$



POS Form

Example :- POS - Product of Sum
- When o/p is logic 0

$$\begin{array}{ccc} \downarrow & & \downarrow \\ M_7 & \text{Maxterm} & M_5 \\ (A+B+C) \cdot (A+\bar{B}+C) \cdot (\bar{A}+B+\bar{C}) & & \downarrow \\ & & M_2 \end{array}$$

Given :-

	A	B	Y	
M_0	0	0	1	$\rightarrow A+B$
M_1	0	1	0	$\rightarrow A+\bar{B}$
M_2	1	0	1	$\rightarrow \bar{A}+B$
M_3	1	1	0	$\rightarrow \bar{A}+\bar{B}$

$$Y = \underbrace{(A+\bar{B})}_{(M_1)} \underbrace{(\bar{A}+\bar{B})}_{(M_3)} \rightarrow \text{Cannonical POS.}$$

$$= \boxed{\Pi M(1, 3)} \rightarrow \text{Standard POS.}$$



Cont...

Note

$$Y(A, B) = \prod M(1, 3) = \overline{B}$$
$$Y(A, B) = \sum m(0, 2) = \overline{B}$$

$$\begin{aligned} \textcircled{1} \quad POS &= (A + \overline{B})(\overline{A} + \overline{B}) = \cancel{A \cdot \overline{A}} + A\overline{B} + \overline{B}\overline{A} + \overline{B}\overline{B} \\ &= \overline{B}(A + \overline{A}) + \overline{B} \\ &= \overline{B} + \overline{B} = \boxed{\overline{B}} \end{aligned}$$

$$\begin{aligned} \textcircled{2} \quad SOP &= \overline{A}\overline{B} + A\overline{B} = \overline{B}(A + \overline{A}) \\ &= \boxed{\overline{B}} \end{aligned}$$

$$\begin{aligned} \sum m(0, 2) &= \prod M(1, 3) \\ \text{or} \\ POS &= SOP \end{aligned}$$

Cont...

Q. Design a combinational logic circuit whose output is high (1) only when majority of inputs (A,B,C,D) are low(0).

Truth Table

Majority-LOW (for 4 inputs): output 1 iff at most one input is 1 (i.e., at least three 0s).

Minterms: $m(0, 1, 2, 4, 8)$.

Minimal SOP:

$$Y = A'B'C' + A'B'D' + A'C'D' + B'C'D'$$

(Each term represents "three particular inputs are 0".)

Equivalent POS (shows the constraint 'no two 1s at once'):

$$Y = (A' + B')(A' + C')(A' + D')(B' + C')(B' + D')(C' + D').$$

A	B	C	D	Out-put
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Cont...

Q. Write the Boolean equation and draw logic diagram for the logic that will have output as Y and inputs A,B,C.

The logic performs the following operation:

- **Y=1**
when **A=B=C=1**
and when **A=B=0** and **C=1**
- **Y=0** for all other cases.

Minterms where Y=1:

- $A = B = C = 1 \Rightarrow m_7$
- $A = B = 0, C = 1 \Rightarrow m_1$

Canonical SOP:

$$Y = \sum m(1, 7)$$

Minimal SOP (simplified):

$$Y = A'B'C + ABC = C(A'B' + AB) = C \cdot \overline{A \oplus B} = C(A \text{ XNOR } B)$$

Concept of Minterm and Maxterm

- **Minterm**- Each individual term in the canonical SOP Form is called as Minterm.
- **Maxterm**- Each individual term in the canonical POS form is called as maxterm.
- Canonical SOP – $Y = ABC + A\bar{B}C + A\bar{B}\bar{C}$ each individual term is **minterm**

$\downarrow$
m0

$\downarrow$
m1

$\downarrow$
m2
- Canonical POS – $Y = (A+B).(A+\bar{B})$

$\downarrow$
M0

$\downarrow$
M1



Conversion from Standard SOP to Canonical SOP form

Steps to be followed:

Step 1: For each term in the given standard SOP expression find the missing literal.

Step 2: Then AND this term with the term formed by ORing the missing literal and its complement.

Step 3: Simplify the expression to get the canonical SOP expression.

Convert the expression into the canonical SOP form

Soln. :

Step 1: Find the missing literal for each term :

$$Y = AB + \overline{A}\overline{C} + BC$$


 C


 B


 A

Missing literal →

Step 2: AND each term with (Missing literal + Its C(complement)) :

$$Y = AB.(C + \overline{C}) + \overline{A}\overline{C}.(B + \overline{B}) + BC.(A + \overline{A})$$





Original product term

Step 3: Simplify the expression to get the canonical SOP :

$$Y = AB(C + \overline{C}) + \overline{A}\overline{C}(B + \overline{B}) + BC(A + \overline{A})$$

$$\begin{aligned}
 &= ABC + \overline{A}BC + \overline{A}B\overline{C} + \overline{A}\overline{C}\overline{B} + \overline{A}\overline{C}B + \overline{A}BC \\
 &= (ABC + \overline{A}BC) + (\overline{A}B\overline{C} + \overline{A}\overline{C}\overline{B}) + \overline{A}\overline{C}B + \overline{A}BC
 \end{aligned}$$

But $A + \overline{A} = 1$ & $(\overline{A}B\overline{C} + \overline{A}\overline{C}\overline{B}) = \overline{A}\overline{C}(B + \overline{B})$ and $(\overline{A}\overline{C}B + \overline{A}BC) = \overline{A}\overline{C}(B + \overline{B})$

$$\therefore Y = ABC + \overline{A}BC + \overline{A}B\overline{C} + \overline{A}\overline{C}\overline{B}$$

Canonical SOP form


 Each term contains all the literals



Cont...

Soln. :

1. Given expression is,

$$Y = \bar{A} + B\bar{C}\bar{D}$$

$$\therefore Y = \bar{A} (B + \bar{B}) (C + \bar{C}) (D + \bar{D}) + B\bar{C}\bar{D} (A + \bar{A})$$

$$= \bar{A} BCD + \bar{A} BC\bar{D} + \bar{A} B\bar{C}D + \bar{A} B\bar{C}\bar{D}$$

$$+ \bar{A}\bar{B}CD + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D}$$

$$+ AB\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D}$$

$$\therefore Y = \bar{A} BCD + \bar{A} BC\bar{D} + \bar{A} B\bar{C}D + \bar{A} B\bar{C}\bar{D} + \bar{A}\bar{B}CD$$

$$+ \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D} + AB\bar{C}\bar{D}$$

$$\dots (\because A + A = A)$$

This is the required expression in the standard SOP form.



Cont...

1. Convert following equation into standard SOP form (canonical) :

$$Y = \bar{A} + BCD$$
2. Convert following equation into standard POS form (canonical) :

$$Y = (\bar{A} + B)(B + \bar{C})(\bar{A} + \bar{C})$$

Soln. :

1. Given expression is,

$$Y = \bar{A} + BCD$$

$$\begin{aligned} \therefore Y &= \bar{A} (B + \bar{B})(C + \bar{C})(D + \bar{D}) + BCD(\bar{A} + A) \\ &= \bar{A}BCD + \bar{A}BC\bar{D} + \bar{A}B\bar{C}D + \bar{A}B\bar{C}\bar{D} + \bar{A}\bar{B}CD \\ &\quad + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D} \\ &\quad + AB\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D} \\ \therefore Y &= \bar{A}BCD + \bar{A}BC\bar{D} + \bar{A}B\bar{C}D \\ &\quad + \bar{A}B\bar{C}\bar{D} + \bar{A}\bar{B}CD + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}D \\ &\quad + \bar{A}\bar{B}\bar{C}\bar{D} + AB\bar{C}\bar{D} \quad \dots (\because A + A = A) \end{aligned}$$

This is the required expression in the canonical SOP form.

$$\begin{aligned} 2. \quad Y &= (\bar{A} + B)(B + \bar{C})(\bar{A} + \bar{C}) \\ &= (\bar{A} + B + CC) \cdot (B + \bar{C} + AA) \cdot (\bar{A} + \bar{C} + BB) \end{aligned}$$

But $A + BC = (A + B)(A + C)$

$$\begin{aligned} \therefore Y &= (\bar{A} + B + C)(\bar{A} + B + \bar{C}) \cdot (B + \bar{C} + A)(B + \bar{C} + \bar{A}) \\ &\quad \cdot (\bar{A} + \bar{C} + B)(\bar{A} + \bar{C} + \bar{B}) \end{aligned}$$

$$\begin{aligned} \therefore Y &= (\bar{A} + B + C)(\bar{A} + B + \bar{C}) \cdot (A + B + \bar{C})(\bar{A} + B + \bar{C}) \\ &\quad \cdot (\bar{A} + B + \bar{C})(\bar{A} + B + C) \end{aligned}$$

...Ans.



Cont...

Ex. 4.2.6 : Convert following equation to canonical SOP form : $Y = (A + B\bar{C})(B + AC)$

S-09, 2 Marks

Soln. :

$$\begin{aligned} Y &= (A + B\bar{C})(B + AC) \\ &= AB + AAC + BB\bar{C} + ABC\bar{C} \end{aligned}$$

$$= AB + AC + B\bar{C}$$

$$[\because A \cdot A = A, B \cdot B = B \text{ and } C \cdot \bar{C} = 0]$$

$$= AB(C + \bar{C}) + AC(B + \bar{B}) + B\bar{C}(A + \bar{A})$$

$$= ABC + AB\bar{C} + ABC + A\bar{B}C + AB\bar{C} + \bar{A}B\bar{C}$$

$$Y = ABC + AB\bar{C} + A\bar{B}C + \bar{A}B\bar{C}$$

This is required canonical SOP form.

Ex. 4.2.7 : Convert following expression into canonical SOP form : $Y = A + B$

W-09, 2 Marks

Soln. :

$$Y = A + B$$

$$Y = A(B + \bar{B}) + B(A + \bar{A}) = AB + A\bar{B} + AB + \bar{A}B$$

$$Y = AB + A\bar{B} + \bar{A}B$$

This is canonical SOP form.

Conversion from standard POS to canonical POS form

Steps to be followed:

Step 1: For each term find the missing literal

Step 2: Then OR each term with the term formed by ANDing the missing literal in that term with its complement.

Step 3: Simplify the expression to get the canonical POS



Cont...

Ex. 4.2.8: Convert the following into canonical SOP :

1. $AC + CD + BC$ 2. $\bar{A}(B + \bar{C})$

W-12, 4 Marks

Soln.:

1. $AC + CD + BC$:

$$\begin{aligned} \therefore Y &= AC(B + \bar{B})(D + \bar{D}) + CD(A + \bar{A})(B + \bar{B}) \\ &\quad + BC(A + \bar{A})(D + \bar{D}) \\ &= ABCD + ABC\bar{D} + A\bar{B}CD + A\bar{B}C\bar{D} + ABCD \\ &\quad + \bar{A}BCD + \bar{A}BC\bar{D} + \bar{A}\bar{B}CD + \bar{A}\bar{B}C\bar{D} + ABCD \\ &\quad + \bar{A}BCD + \bar{A}BC\bar{D} \\ &= ABCD + ABC\bar{D} + A\bar{B}CD + A\bar{B}C\bar{D} + \bar{A}BCD \\ &\quad + \bar{A}BC\bar{D} + \bar{A}\bar{B}CD + \bar{A}\bar{B}C\bar{D} \end{aligned}$$

2. $\bar{A}(B + \bar{C})$:

$$\begin{aligned} \therefore Y &= \bar{A}B + \bar{A}\bar{C} \\ &= \bar{A}B(C + \bar{C}) + \bar{A}\bar{C}(B + \bar{B}) \\ &= \bar{A}BC + \bar{A}B\bar{C} + \bar{A}\bar{B}C + \bar{A}\bar{B}\bar{C} \end{aligned}$$

Convert the following SOP equation into standard SOP equation.

$$Y = AB + \bar{A}B + A\bar{B}\bar{C}$$

S-15, 2 Marks

$$Y = AB + \bar{A}B + A\bar{B}\bar{C}$$

$$= AB(C + \bar{C}) + \bar{A}B(C + \bar{C}) + A\bar{B}\bar{C}$$

$$Y = ABC + ABC\bar{C} + \bar{A}BC + \bar{A}B\bar{C} + A\bar{B}\bar{C}$$

$$Y = ABC + AB\bar{C} + \bar{A}BC + \bar{A}B\bar{C} + A\bar{B}\bar{C} \dots \text{Ans}$$



Cont...

Ex. 4.2.11: Convert the following expression into its standard forms:

$$1. Y = \bar{A}BC + AC + \bar{B}$$

$$2. Y = (B + \bar{C}) \times (A + D) \times (\bar{B} + \bar{D})$$

W-16, 4 Marks

Soln.:

$$1. Y = \bar{A}BC + AC + \bar{B}$$

$$= \bar{A}BC + AC(B + \bar{B}) + \bar{B}(A + \bar{A})(C + \bar{C})$$

$$= \bar{A}BC + ABC + \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}\bar{B}C$$

$$Y = \bar{A}BC + ABC + \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C}$$

$$Y = (B + \bar{C}) \cdot (A + D) \cdot (\bar{B} + \bar{D})$$

$$= (B + \bar{C} + A\bar{A} + D\bar{D}) \cdot (A + D + B\bar{B} + C\bar{C}) \cdot (\bar{B} + \bar{D} + A\bar{A} + C\bar{C})$$

$$= (B + \bar{C} + A + D) (B + \bar{C} + A + \bar{D}) (B + \bar{C} + \bar{A} + D)$$

$$(B + \bar{C} + \bar{A} + \bar{D}) (A + D + B + C) (A + D + B + \bar{C})$$

$$(A + D + \bar{B} + C) (A + D + \bar{B} + \bar{C}) (\bar{B} + \bar{D} + A + C)$$

$$(\bar{B} + \bar{D} + A + \bar{C}) (\bar{B} + \bar{D} + \bar{A} + C) (\bar{B} + \bar{D} + \bar{A} + \bar{C})$$

$$= (A + B + \bar{C} + D) (A + B + \bar{C} + \bar{D}) (\bar{A} + \bar{B} + \bar{C} + D)$$

$$(\bar{A} + B + \bar{C} + \bar{D}) (A + B + C + D) (A + \bar{B} + C + D)$$

$$(A + \bar{B} + \bar{C} + D) (A + \bar{B} + C + \bar{D}) (A + \bar{B} + \bar{C} + \bar{D})$$

$$(\bar{A} + \bar{B} + C + \bar{D}) (\bar{A} + \bar{B} + \bar{C} + D)$$



Cont...

Ex. 4.4.3: Convert the following boolean expression into its standard forms :

$$1. \quad Y = A\bar{B} + AC + \bar{B}C$$

$$2. \quad Y = (A + \bar{B}) \cdot (A + C) \cdot (B + \bar{C})$$

S-14, 4 Marks

Soln. :

$$1. \quad Y = A\bar{B} + AC + \bar{B}C$$

$$= AB(C + \bar{C}) + AC(B + \bar{B}) + \bar{B}C(A + \bar{A})$$

$$= ABC + AB\bar{C} + ABC + A\bar{B}C + A\bar{B}\bar{C} + \bar{A}\bar{B}C$$

$$Y = ABC + AB\bar{C} + A\bar{B}C + \bar{A}\bar{B}C$$

$$\dots(\because ABC + ABC = ABC, A\bar{B}C + \bar{A}\bar{B}C = A\bar{B}C)$$

This is required standard SOP form.

$$2. \quad Y = (A + \bar{B})(A + C)(B + \bar{C})$$

$$= (A + \bar{B} + C\bar{C})(A + C + B\bar{B})(B + \bar{C} + A\bar{A})$$

$$= (A + \bar{B} + C)(A + \bar{B} + \bar{C})(A + B + C)(A + \bar{B} + C)$$

$$(A + B + \bar{C})(\bar{A} + B + \bar{C})$$

$$= (A + \bar{B} + C)(A + \bar{B} + \bar{C})(A + B + C)$$

$$(A + B + \bar{C})(\bar{A} + B + \bar{C})$$

$$\dots(\because (A + \bar{B} + C)(A + \bar{B} + \bar{C}) = A + \bar{B} + C)$$

This is required standard POS form.



SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR

K-Map



**Karnaugh
Map(K-Map)**



What is K-map ? Explain the rules to simplify

- This is another **simplification technique to reduce the Boolean equation &** it overcomes all the disadvantages of the algebraic simplification technique.
- K-map (short form of Karnaugh map) **is a graphical method of simplifying a Boolean equation.**
- K-maps can be written **for 2, 3, 4 ... upto 6 variables.** Beyond that the **K-map technique becomes very cumbersome.**
- K-map is ideally suitable for designing the combinational logic circuits using either a SOP method or a POS method.

K-Map Structure

2 variable k map

A. SOP: -

A \ B	$\bar{B}$ 0	B 1
$\bar{A}$ 0	$\bar{A}.\bar{B}$	$\bar{A}.B$
A 1	$A.\bar{B}$	$A.B$

4 variable k map

A \ B	CD			
	00	01	11	10
00				
01				
11				
10				

3 variable k map

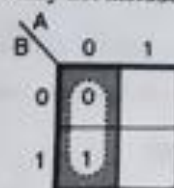
2 elements in one group

A \ BC	00	01	11	10
0	0 0	1 1	0 3	1 2
1	1 4	1 5	0 7	0 6

	C'D'	C'D	CD	CD'
A'B'	0	1	3	2
A'B	4	5	7	6
AB	12	13	15	14
AB'	8	9	11	10

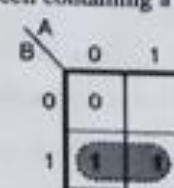
Rules followed for K-Map Simplification

1. Groups may not include any cell containing a zero.



Wrong X

(a)



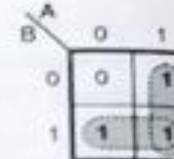
Right ✓

(b)

2. Groups may be horizontal or vertical, but not diagonal.



Wrong X

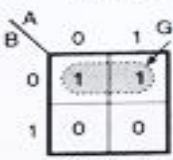


Right ✓

3. Groups must contain 1, 2, 4, 8, or in general 2^n cells. Identify the largest possible group first.

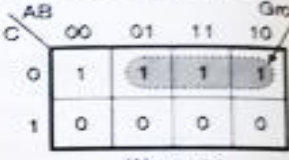
That is if $n = 1$, a group will contain two 1's since $2^1 = 2$.

If $n = 2$, a group will contain four 1's since $2^2 = 4$.



Right ✓

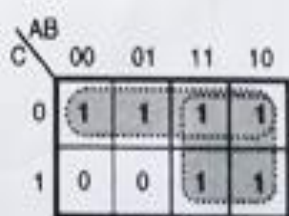
(a)



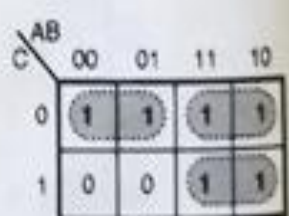
Wrong X

(b)

4. Each group should be as large as possible :



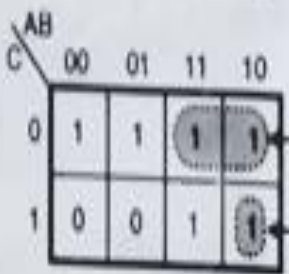
Right ✓



Wrong X

(Note that no Boolean law broken, but not sufficiently minimized)

5. Each cell containing a one must be in at least one group.



Group I

Group II

1 Present in at least one group



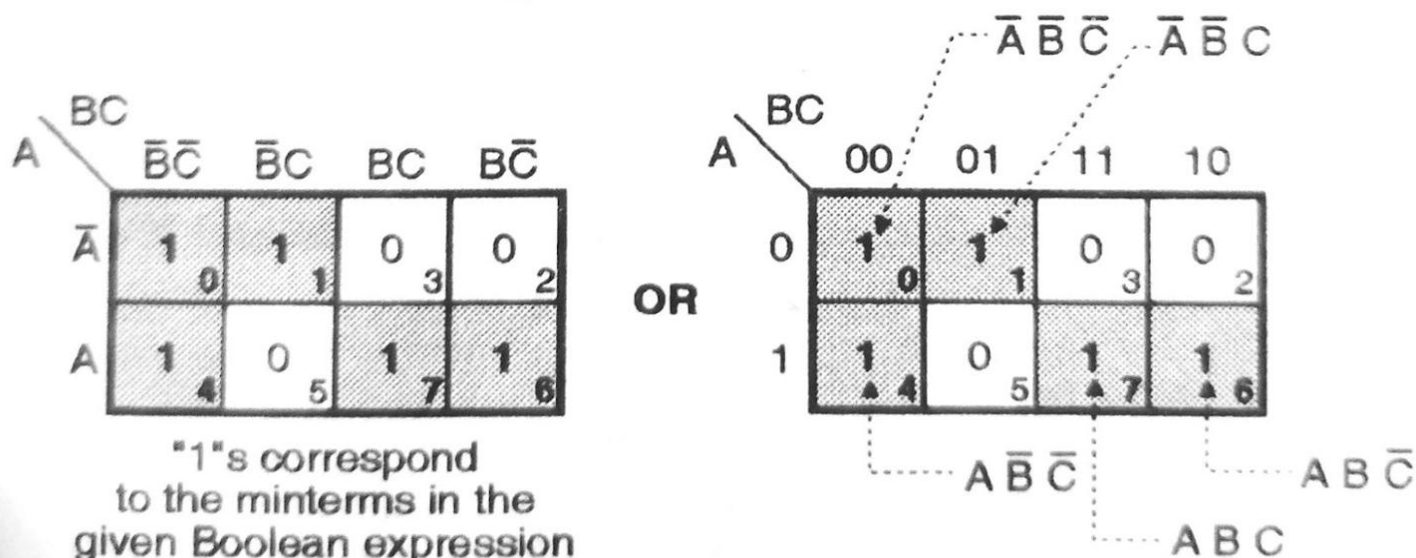
Cont....

Ex. 4.5.1 : Represent the equation given below on Karnaugh map :

$$Y = \bar{A} \bar{B} \bar{C} + \bar{A} \bar{B} C + A \bar{B} \bar{C} + A \bar{B} C + ABC.$$

Soln. : The given expression is in the standard SOP form. Each term represents a minterm.

We have to enter 1's in the boxes corresponding to each minterm, as shown in Fig. P. 4.5.1.

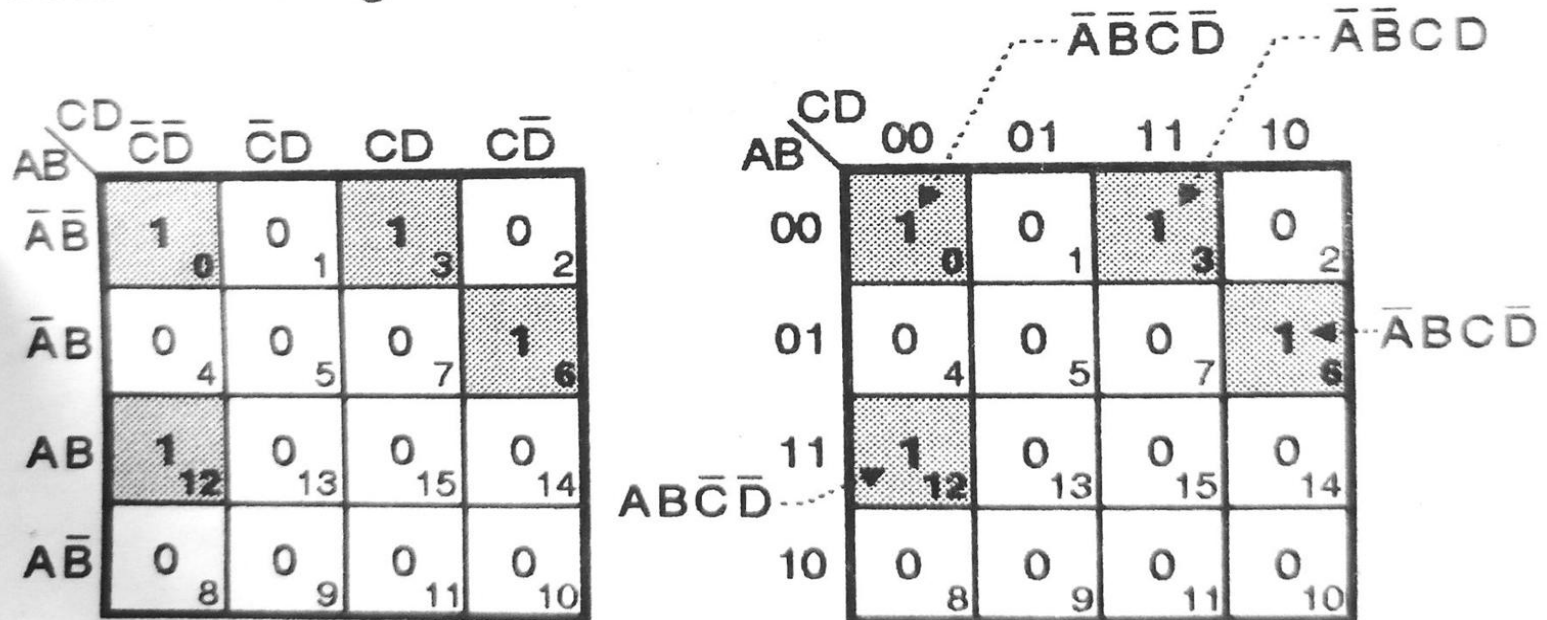


Cont....

Ex. 4.5.2 : Plot the following Boolean expression on K-map :

$$Y = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}CD + \bar{A}BC\bar{D} + AB\bar{C}\bar{D}$$

Soln. : Refer Fig. P. 4.5.2.



"1"s correspond to the minterms in the given Boolean expression

Way of Grouping (Pairs, Quads and Octets)

1. While grouping, we **should group most number of 1's** (or 0's).
2. The grouping follows the binary rule i.e. we can group 1, 2, 4, 8, 16, 32 number of 1's or 0's. We cannot group 3, 5, 7, number of 1's or 0's.
 - **Pairs:** A group of two adjacent 1's or 0's is called as a pair.
 - **Quad:** A group of four adjacent 1's or 0's is called as a quad.
 - **Octet:** A group of eight adjacent 1's or 0's is called as octet.

Grouping of 2 Adjacent Ones

Given K-map :

	BC	$\bar{B}\bar{C}$	$\bar{B}C$	$B\bar{C}$
$\bar{A}$	0	0	1	1
A	0	0	0	0

Group of adjacent 1's
i.e. a pair

(C-228) Fig. 4.6.3

Simplification :

$$Y = \bar{A}BC + \bar{A}B\bar{C} = \bar{A}B(C + \bar{C})$$

$$= \bar{A}B \quad \dots \text{since } C + \bar{C} = 1$$

Thus C is eliminated

Given K-map :

	BC	$\bar{B}\bar{C}$	$\bar{B}C$	$B\bar{C}$
$\bar{A}$	0	1	0	0
A	0	1	0	0

Group of adjacent 1's
i.e. a pair

Given K-map :

	BC	$\bar{B}\bar{C}$	$\bar{B}C$	$B\bar{C}$
$\bar{A}$	0	0	0	0
A	1	0	0	1

Pair

(C-228)

Simplification :

$$Y = A\bar{B}\bar{C} + A\bar{B}C$$

$$= A\bar{B}(\bar{C} + C)$$

$$Y = A\bar{B}$$

Thus A is eliminated

Given K-map :

	BC	$\bar{B}\bar{C}$	$\bar{B}C$	$B\bar{C}$
$\bar{A}$	0	0	1	1
A	0	1	1	0

Pair 1 : $\bar{A}BC + \bar{A}B\bar{C} = \bar{A}B(C + \bar{C}) = \bar{A}B$

Pair 2 : $A\bar{B}\bar{C} + A\bar{B}C = A\bar{B}(\bar{C} + C) = A\bar{B}$

Pair 3 : This pair is not required

Add the two product terms to get,

$$Y = \bar{A}B + A\bar{B}$$

Grouping of 2 Adjacent Ones

Given K-map :

		CD			
		$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
AB	$\bar{A}\bar{B}$	0	1	0	0
	$\bar{A}B$	0	0	0	0
AB	$A\bar{B}$	0	0	0	0
	AB	0	1	0	0

(C-228)

Simplification :

$$Y = \bar{A}\bar{B}\bar{C}D + A\bar{B}\bar{C}D = \bar{B}\bar{C}D(\bar{A} + A)$$

$$\therefore Y = \bar{B}\bar{C}D$$

Conclusion : A is eliminated.

Example of overlap :

Given K-map :

		B	
		$\bar{B}$	B
A	$\bar{A}$	1	1
	A	1	0

Pair 1 : $\bar{A}\bar{B} + \bar{A}B$
 $= \bar{A}(\bar{B} + B) = \bar{A}$

Note that B is eliminated

Pair 2 : $\bar{A}\bar{B} + A\bar{B} = \bar{B}$

Note that A is eliminated

$\therefore$ Final expression $Y = \bar{A} + \bar{B}$

Note that in order to cover all the 1's, we have to overlap two pairs as shown.



Grouping of 4 Adjacent Ones

Given K-map :

CD \ AB	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	1	0	0
$\bar{A}B$	0	1	0	0
AB	0	1	0	0
$A\bar{B}$	0	1	0	0

→ $Y = \bar{C}D$

(C-231)

Simplification :

$$\begin{aligned}
 Y &= \bar{A}\bar{B}\bar{C}D + \bar{A}B\bar{C}D + A\bar{B}\bar{C}D + A\bar{B}C\bar{D} \\
 &= \bar{C}D[\bar{A}\bar{B} + \bar{A}B + A\bar{B} + A\bar{B}] \\
 &= \bar{C}D[\bar{A}(\bar{B} + B) + A(\bar{B} + B)] \\
 &= \bar{C}D[\bar{A} + A] = \bar{C}D
 \end{aligned}$$

Thus A and B are eliminated.

Given K-map :

CD \ AB	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	0	0	0	0
AB	0	0	0	0
$A\bar{B}$	1	1	1	1

→ $Y = A\bar{B}$

(C-230)

Simplification :

$$\begin{aligned}
 Y &= A\bar{B}\bar{C}\bar{D} + A\bar{B}\bar{C}D + A\bar{B}C\bar{D} + A\bar{B}CD \\
 &= A\bar{B}[\bar{C}\bar{D} + \bar{C}D + C\bar{D} + CD] \\
 &= A\bar{B}[\bar{C}(\bar{D} + D) + C(\bar{D} + D)] \\
 \therefore Y &= A\bar{B}[\bar{C} + C] = A\bar{B}
 \end{aligned}$$

Thus C and D are eliminated.



Cont....

Four adjacent ones forming a square

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	1	1	0	0
AB	1	1	0	0
$A\bar{B}$	0	0	0	0

(C-231(a))

→ $Y = B\bar{C}$

$$\begin{aligned}
 Y &= \bar{A} B \bar{C} \bar{D} + A B \bar{C} \bar{D} + \bar{A} B \bar{C} D + A B \bar{C} D \\
 &= B \bar{C} \bar{D} (\bar{A} + A) + B \bar{C} D (\bar{A} + A) \\
 &= B \bar{C} \bar{D} + B \bar{C} D = B \bar{C} (\bar{D} + D)
 \end{aligned}$$

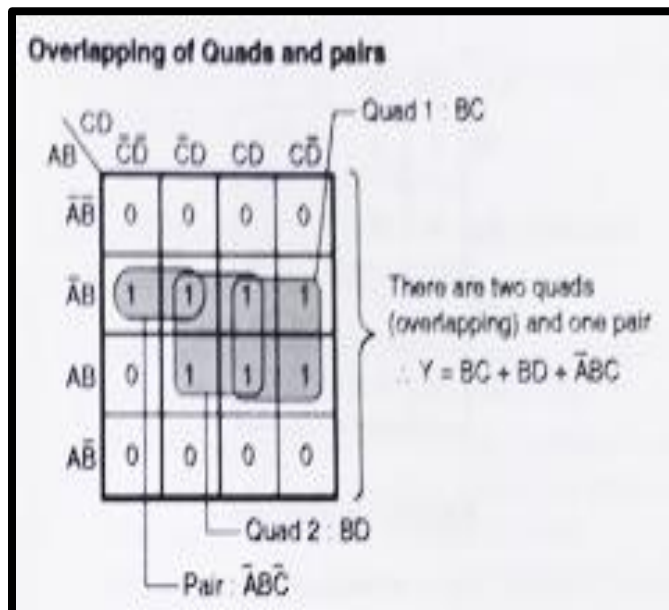
$$\therefore Y = B\bar{C}$$

Thus A and D are eliminated

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	1	1	0
$\bar{A}B$	0	0	0	0
AB	0	0	0	0
$A\bar{B}$	0	1	1	0

→ $Y = \bar{B}D$

Cont....



CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	1	0	0	1
AB	1	0	0	1
AB	0	0	0	0

(C-231(c))

$$Y = B\bar{D}$$

$$Y = \bar{A}\bar{B}\bar{C}\bar{D} + A\bar{B}\bar{C}\bar{D} + \bar{A}BC\bar{D} + ABC\bar{D}$$

$$= \bar{B}\bar{C}\bar{D}(\bar{A} + A) + B\bar{C}\bar{D}(\bar{A} + A)$$

$$= \bar{B}\bar{C}\bar{D} + B\bar{C}\bar{D}$$

$$Y = \bar{B}\bar{D}(\bar{C} + C) = \bar{B}\bar{D}$$

Thus A and C are eliminated

1's corresponding to corners forming a Quad :

CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	1	0	0	1
$\bar{A}B$	0	0	0	0
AB	0	0	0	0
AB	1	0	0	1

$$Y = \bar{B}\bar{D}$$

(C-232)

$$Y = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D}$$

$$= \bar{B}\bar{C}\bar{D}(\bar{A} + A) + \bar{B}C\bar{D}(\bar{A} + A)$$

$$= \bar{B}\bar{C}\bar{D} + \bar{B}C\bar{D} = \bar{B}\bar{D}(\bar{C} + C)$$

$$\therefore Y = \bar{B}\bar{D} \quad \text{Thus A and C are eliminated}$$

Grouping of 8 Adjacent Ones

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	1	1
$\bar{A}B$	0	0	1	1
AB	0	0	1	1
$A\bar{B}$	0	0	1	1

$Y = C$

A, B and D are eliminated

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	1	1	1	1
$\bar{A}B$	0	0	0	0
AB	0	0	0	0
$A\bar{B}$	1	1	1	1

Octet $Y = \bar{B}$

A, C and D are eliminated

(C-234(b)) (c)

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	1	0	0	1
$\bar{A}B$	1	0	0	1
AB	1	0	0	1
$A\bar{B}$	1	0	0	1

Octet $Y = \bar{D}$

A, B and C are eliminated



Cont....

AB \ CD	CD			
	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	1	1	1	1
$\bar{A}B$	1	1	1	1
$A\bar{B}$	0	0	0	0
AB	0	0	0	0

→ $Y = \bar{A}$

B, C and D are eliminated

(C-234) Fig. 4.6.6(a)

$$Y = \bar{A} \bar{B} \bar{C} \bar{D} + \bar{A} \bar{B} \bar{C} D + \bar{A} \bar{B} C \bar{D} + \bar{A} \bar{B} C D \\ + \bar{A} B \bar{C} \bar{D} + \bar{A} B \bar{C} D + \bar{A} B C \bar{D} + \bar{A} B C D$$

$$\therefore Y = \bar{A} \bar{B} \bar{C} (\bar{D} + D) + \bar{A} \bar{B} C$$

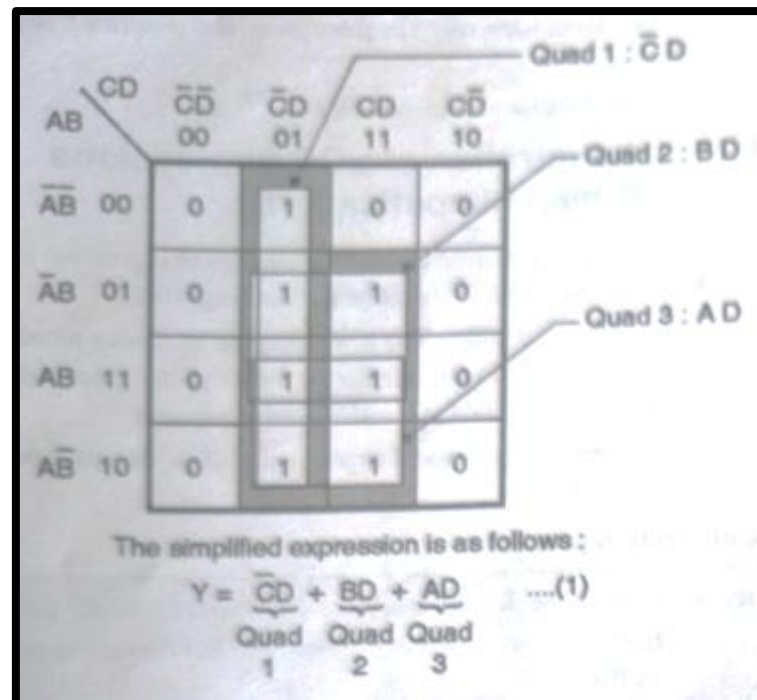
$$(\bar{D} + D) + \bar{A} B \bar{C} (\bar{D} + D) + \bar{A} B C (\bar{D} + D)$$

$$\therefore Y = \bar{A} \bar{B} (\bar{C} + C) + \bar{A} B (\bar{C} + C)$$

$$\therefore Y = \bar{A} (\bar{B} + B) = \bar{A}$$

Example

- For the logical expression given below draw the K-map and obtain the simplified logical expression: $Y = \sum m(1, 5, 7, 9, 11, 13, 15)$. Realize the minimized expression using the basic gates.





Example

2. Using K-map realize the following expression using minimum number of gates : $\Sigma m (1, 3, 4, 5, 7, 9, 11, 13, 15)$
3. Simplify the following expressions using K-map :
 - a. $f(A, B, C) = \Sigma m (0, 1, 3, 4, 6)$
 - b. $f(A, B, C, D) = \Sigma m (0, 1, 2, 4)$



Cont...

1. Simplify the Boolean function using a 3-variable K-map:

$$F(A,B,C)=\Sigma m(1,2,6)+d(0,5)$$

2. Obtain the minimized expression:

$$F(A,B,C)=\Sigma m(3,5,7)+d(1,2)$$

3. Minimize the function using K-map:

$$F(A,B,C)=\Sigma m(0,4,6)+d(2,7)$$

4. Simplify the Boolean function:

$$F(A,B,C,D)=\Sigma m(1,3,7,11,15)+d(0,2,5)$$

5. Reduce the given function using K-map:

$$F(A,B,C,D)=\Sigma m(4,6,8,10,12,14)+d(0,2)$$

6. A digital system has 4 inputs (A,B,C,D). The output is 1 **when at least three inputs are HIGH**. Assume the cases where exactly one input is HIGH are “don’t care”. Write the SOP function and minimize it using a 4-variable K-map.

Cont...

Application-Based 4-Variable K-map SOP Questions

1. A digital system has 4 inputs (A,B,C,D). The output should be 1 if **at least three inputs are HIGH**. Write the SOP expression from the truth table and minimize it using a 4-variable K-map.
2. A circuit has 4 inputs (A,B,C,D). The output is 1 **only when exactly two inputs are HIGH**. Derive the Boolean expression in SOP form and minimize it using a 4-variable K-map.
3. A system has 4 inputs (A,B,C,D). The output should be 1 if **the number of HIGH inputs is even (0, 2, or 4)**. Construct the SOP expression and simplify it using a K-map.
4. A digital lock has 4 switches (A,B,C,D). The lock opens (output = 1) **only if input A is HIGH and at least one of the other three inputs (B, C, D) is HIGH**. Write the SOP expression and minimize it using a 4-variable K-map.
5. Simplify using a 4-variable K-map:
$$F(A,B,C,D)=\Sigma m(0,2,5,7,8,10,13,15).$$
6. Reduce the Boolean expression:
$$F(A,B,C,D)=\Sigma m(1,3,4,5,11,13,14,15)$$



SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR

Cont...

Cont...

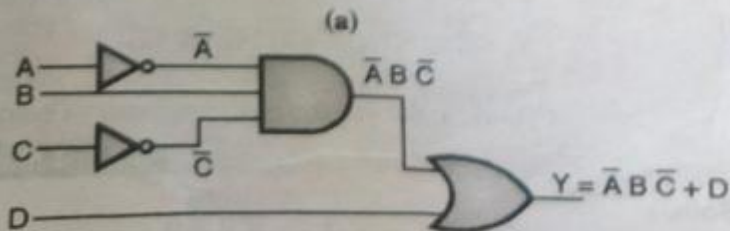
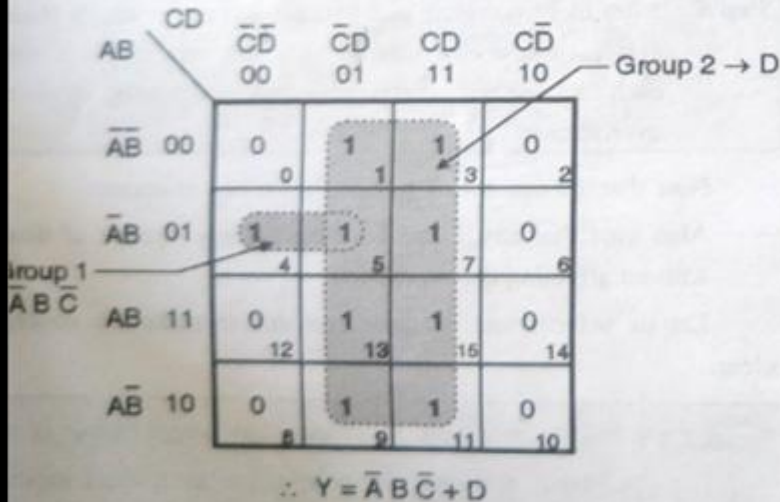
Ex. 4.7.2 : Using K-map realize the following expression using minimum number of gates :

$$Y = \sum m(1, 3, 4, 5, 7, 9, 11, 13, 15)$$

S-09. 4 Marks

Soln. :

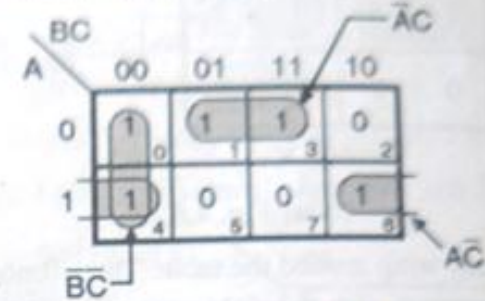
The required K-map is shown in Fig. P. 4.7.2(a) and the realization using minimum gates is shown in Fig. P. 4.7.2(b).



(b) Realization with minimum gates

1. $f(A,B,C) = \sum m(0,1,3,4,6)$

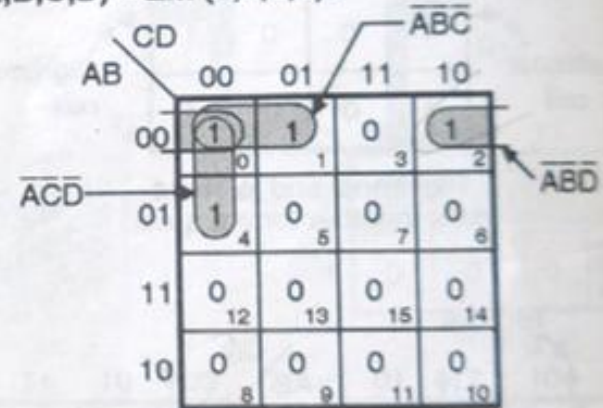
Simplification using K-map :



(C-2600) Fig. P. 4.7.3(a)

$$\therefore f(A,B,C) = \bar{A}\bar{C} + \bar{A}C + BC$$

2. $f(A,B,C,D) = \sum m(0,1,2,4)$



(C-2601) Fig. P. 4.7.3(b)

$$\therefore f(A,B,C,D) = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}D + \bar{A}C\bar{D}$$

Cont...

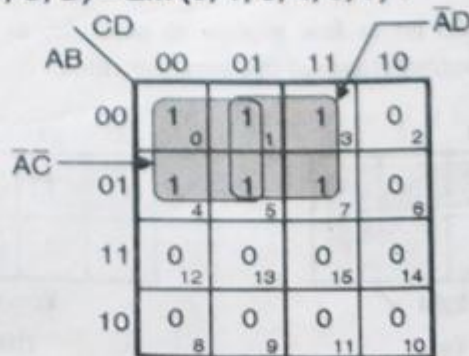
Ex. 4.7.4 : Solve the following SOP expression with K-map :

1. $f(A, B, C, D) = \sum m(0, 1, 3, 4, 5, 7)$
2. $f(A, B, C) = \sum m(0, 1, 4, 5, 6, 7)$

W-11, W-17, 4 Marks

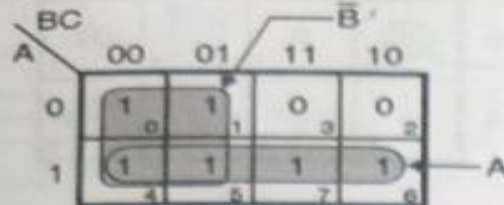
Soln. :

1. $f(A, B, C, D) = \sum m(0, 1, 3, 4, 5, 7) :$



$$f(A, B, C, D) = \bar{A}D + \bar{A}\bar{C}$$

2. $f(A, B, C) = \sum m(0, 1, 4, 5, 6, 7) :$



(C-2603) Fig. P. 4.7.4(b)

$$f(A, B, C) = A + \bar{A}$$

Ex. 4.7.5 : Convert the following functions into SOP form and plot the K map :
 $Y = AB + AC + BC.$

W-11

Soln. :

$$Y = AB + AC + BC$$

$$= AB(C + \bar{C}) + AC(B + \bar{B}) + BC(A + \bar{A})$$

$$= ABC + A\bar{B}\bar{C} + ABC + A\bar{B}C + ABC + \bar{A}BC$$

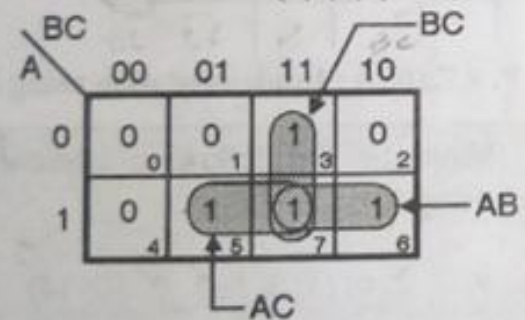
$$\therefore Y = ABC + A\bar{B}\bar{C} + A\bar{B}C + \bar{A}BC$$

$$\therefore Y = \sum m(7, 6, 5, 3)$$

This is the required standard SOP form (canonical)

Simplification using K-map :

$$Y = \sum m(3, 5, 6, 7)$$



(C-2604) Fig. P. 4.7.5

$$Y = AB + BC + AC$$

Cont...

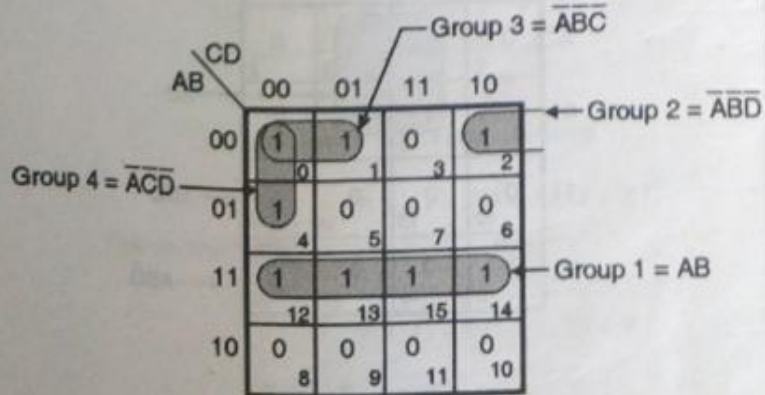
Ex. 4.7.6 : Solve the following Boolean expression using K-map :

$$Y = \sum m (m_0, m_1, m_2, m_4, m_{12}, m_{13}, m_{14}, m_{15})$$

S-12, 4 Marks

Soln. :

Simplification using K-map :



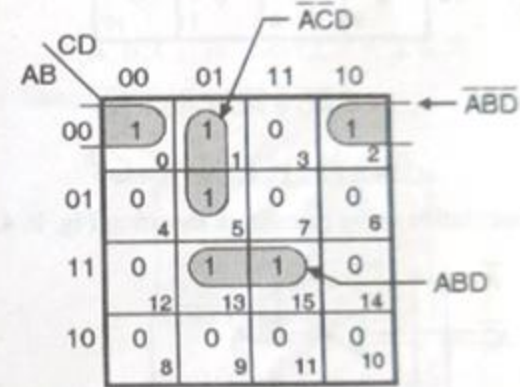
Ex. 4.7.8 : Simplify the following using K-map and realize using NAND – NAND gates.

$$f(A, B, C, D) = \sum m (0, 1, 2, 5, 13, 15)$$

W-12, 4 Marks

Soln. :

Step 1 : Minimization using K-map :



(C-3105) Fig. P. 4.7.8 (a)

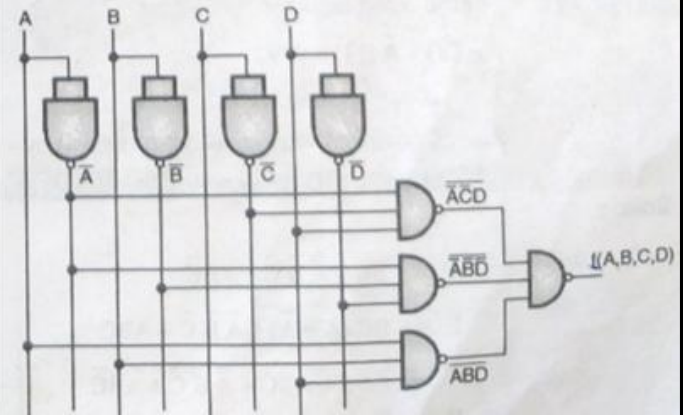
Step 2 : Realization using NAND gates :

$$f(A, B, C, D) = \overline{\overline{A} \overline{C} D} + \overline{\overline{A} B \overline{D}} + \overline{ABD}$$

...Take double inversion

$$= \overline{\overline{A} \overline{C} D} \cdot \overline{\overline{A} B \overline{D}} \cdot \overline{ABD}$$

...using De Morgan's laws.





Cont...

Digi. Tech. and ...

7.9 : Minimize the following Boolean expression using K-map.

$$Y = \sum m(0, 1, 3, 4, 5, 6, 7, 13, 15)$$

Draw the logical circuits diagram of minimized expression using basic gates. **W-13. 4 Marks**

n. :

$$Y = \sum m(0, 1, 3, 4, 5, 6, 7, 13, 15)$$

The required K-map is as shown in Fig. P. 4.7.9.

AB \ CD	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	1	1	0
10	0	0	0	0

$\therefore Y = \bar{A}\bar{C} + \bar{A}\bar{D} + \bar{A}\bar{B} + BD$

Cont...

➤ Minimize the following expressions using K-map :

- $Y = \sum m (1, 5, 6, 7, 11, 12, 13, 15)$
- $F = \sum m (0, 1, 2, 3, 11, 12, 14, 15)$

Solution: Simplification using K-Map is as shown below

A \ B	CD			
	00	01	11	10
00				
01				
11				
10				

$$\begin{aligned}
 F &= \sum (0, 2, 3, 6, 7) + d (8, 10, 11, 15) \\
 &= \pi (1, 4, 5, 9, 12, 13, 14) + d (8, 10, 11, 15)
 \end{aligned}$$

$$\therefore F = (C + \bar{D}) \cdot (\bar{B} + C) \cdot \bar{A}$$

AB \ CD	C+D C+ $\bar{D}$ $\bar{C}$ + $\bar{D}$ $\bar{C}$ +D			
	C+D	C+ $\bar{D}$	$\bar{C}$ + $\bar{D}$	$\bar{C}$ +D
A+B		0		
A+ $\bar{B}$	0	0		
$\bar{A}$ + $\bar{B}$	0	0	X	0
$\bar{A}$ +B	X	0	X	X



संयुक्त प्रौद्योगिकी

SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR

*Thank
you*



Unit-4 Syllabus

➤ Introduction

➤ Latches

➤ Flip-Flop

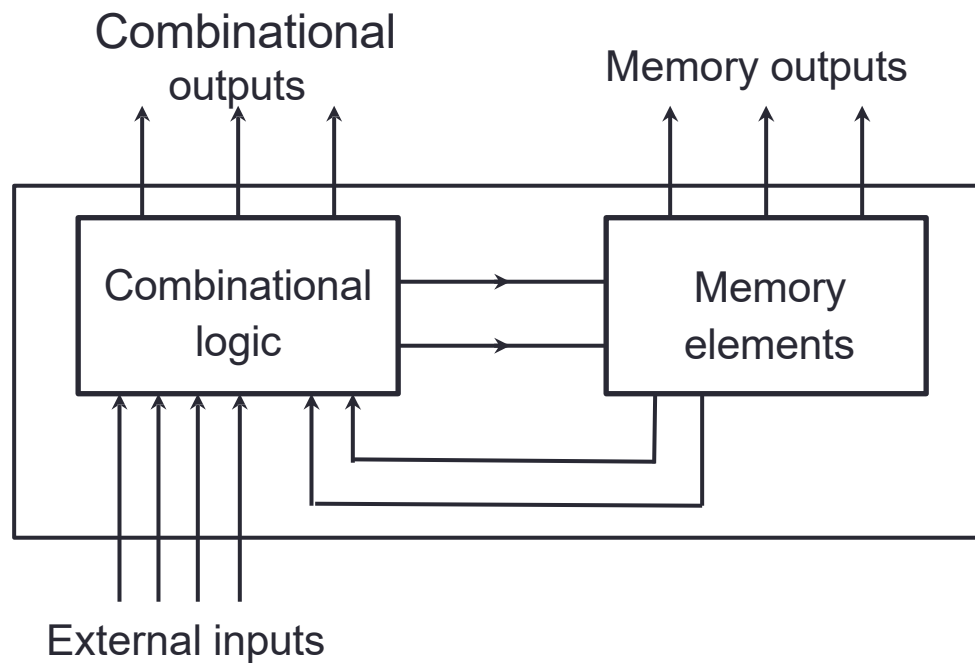
➤ Registers

➤ Counters



Introduction

A **sequential circuit** consists of a *feedback path*, and employs some *memory elements*.



Sequential Circuit = **Combinational logic** + **Memory Elements**



Introduction

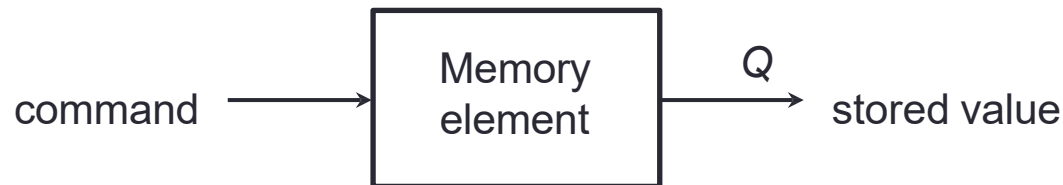
- There are two types of sequential circuits:
 - *Synchronous*: outputs change only at specific time
 - *Asynchronous*: outputs change at any time
- *Multivibrator*: A class of sequential circuits. They can be:
 - *Bistable* (2 stable states)
 - *Monostable* or *one-shot* (1 stable state)
 - *Astable* (no stable state)
- Bistable logic devices: *latches* and *flip-flops*.
- Latches and flip-flops differ in the method used for changing their state.





Memory Elements

Memory Element: A device which can remember value indefinitely, or change value on command from its inputs.



Characteristic Table:

Command (at time t)	$Q(t)$	$Q(t+1)$
Set	X	1
Reset	X	0
Memorise / No Change	0	0
	1	1

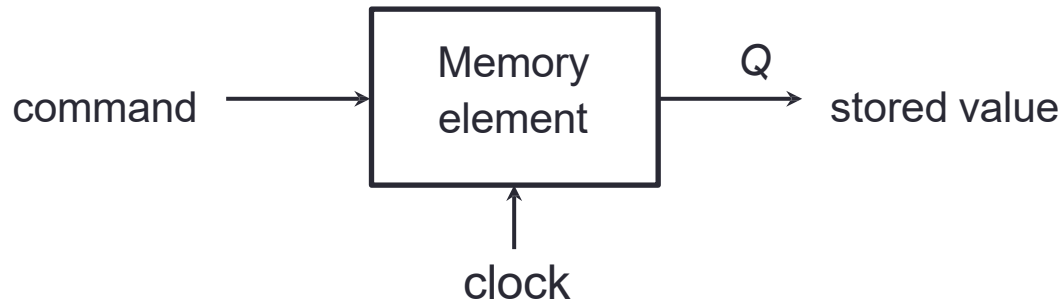
$Q(t)$: Current State

$Q(t+1)$ or Q^+ : Next State

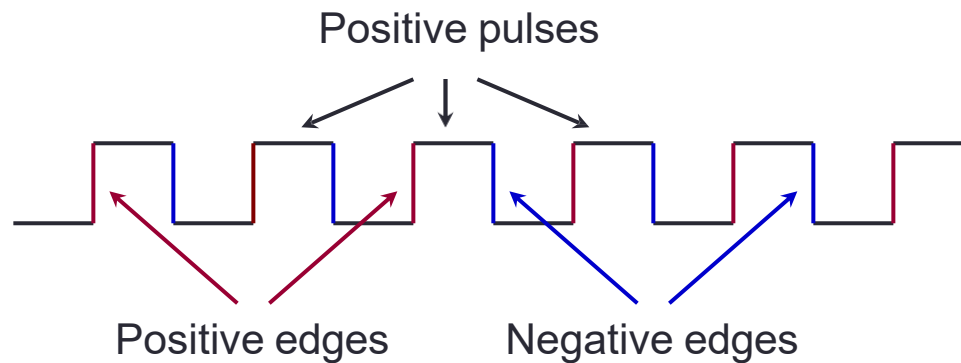


Memory Element with Clock

Flip-flops are memory elements that change state on clock signals.

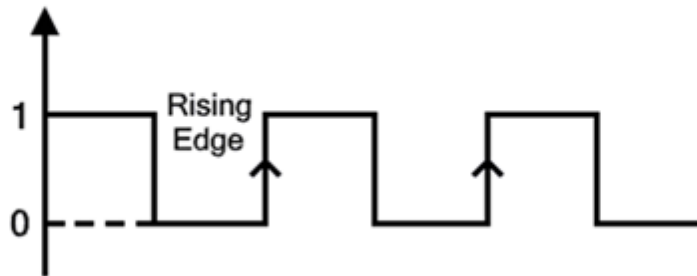


Clock is usually a square wave.

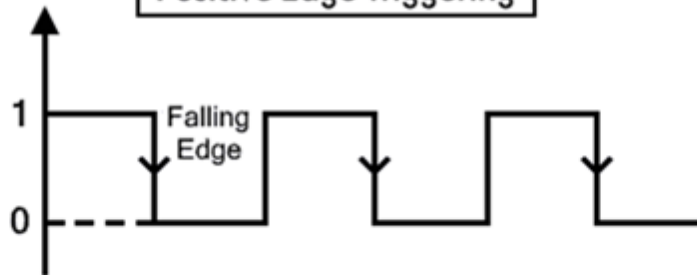


Cont...

Difference Between Edge Triggering & Level Triggering

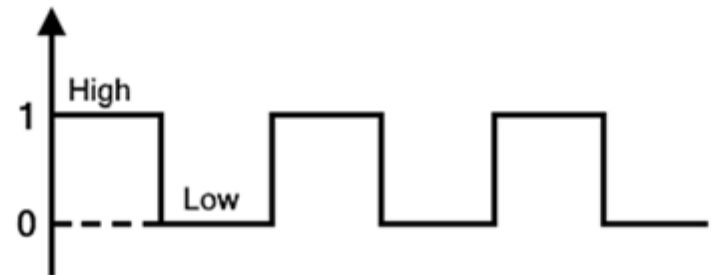


Positive Edge Triggering



Negative Edge Triggering

Edge Triggering

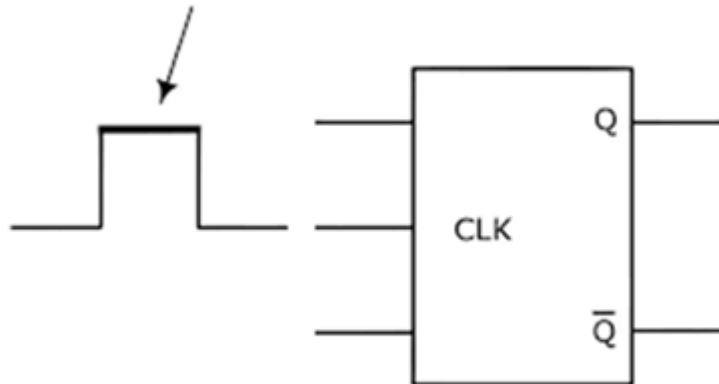


Level Triggering



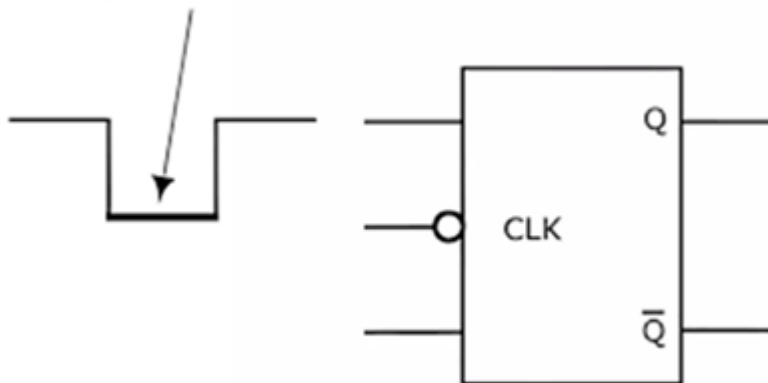
Cont...

Triggers on high clock level



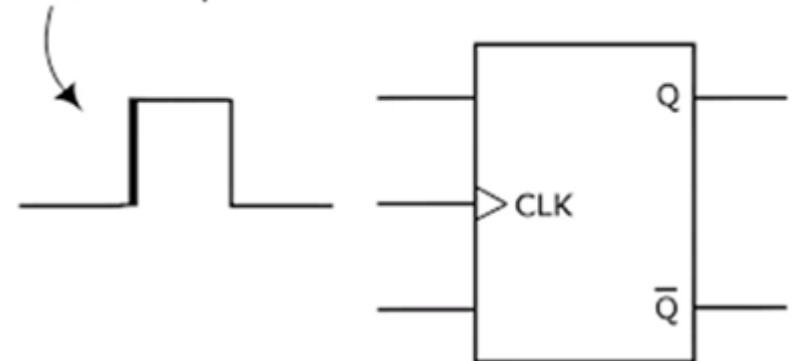
High Level Triggering

Triggers on low clock level



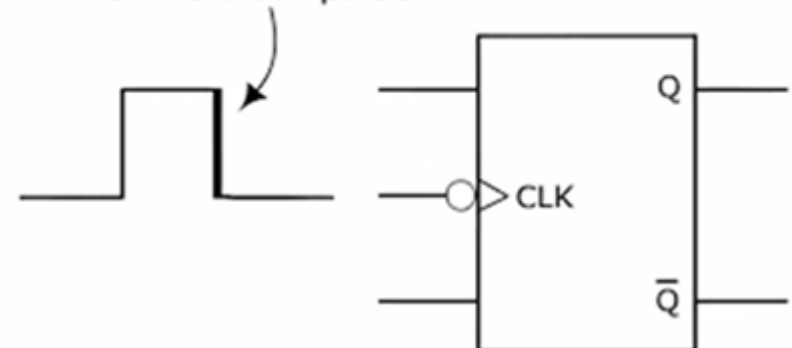
Low Level Triggering

Triggers on this edge of the clock pulse



Positive Edge Triggering

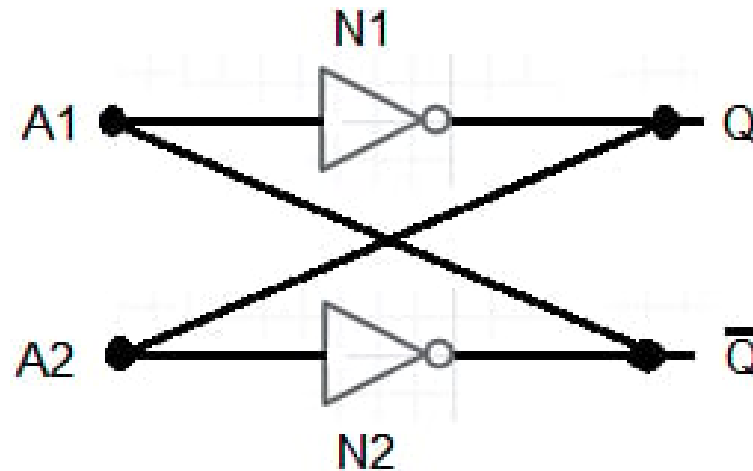
Triggers on this edge of the clock pulse



Negative Edge Triggering

Latch Using NOT Gate

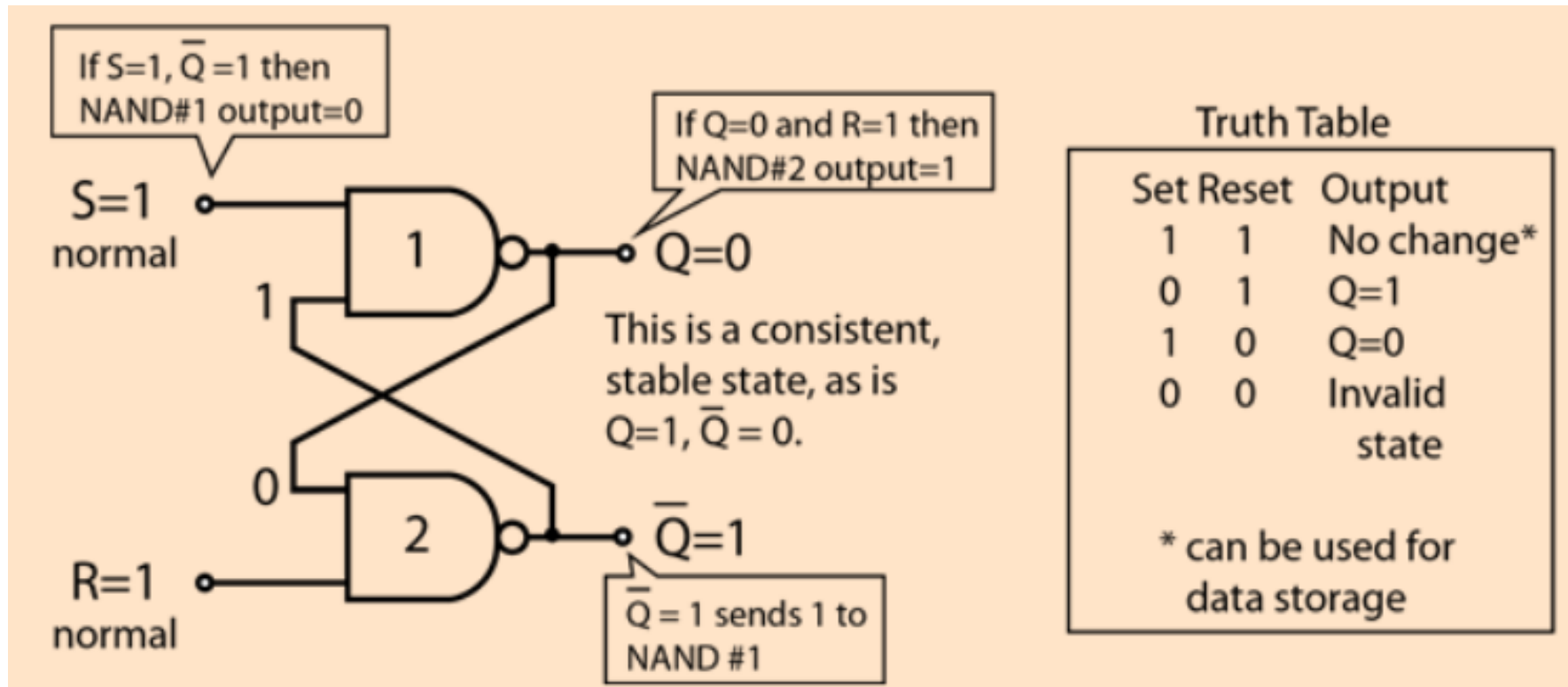
- Cross-coupling two NOT circuits N1 and N2.
- Connecting each gate's output to the other gate's input, we get a feedback combination commonly referred to as a latch.
- A latch is a memory element characterized by having only two stable logical states at its output .
- The latch is a bistable circuit with two complementary outputs.



Drawback: We can not change the state.

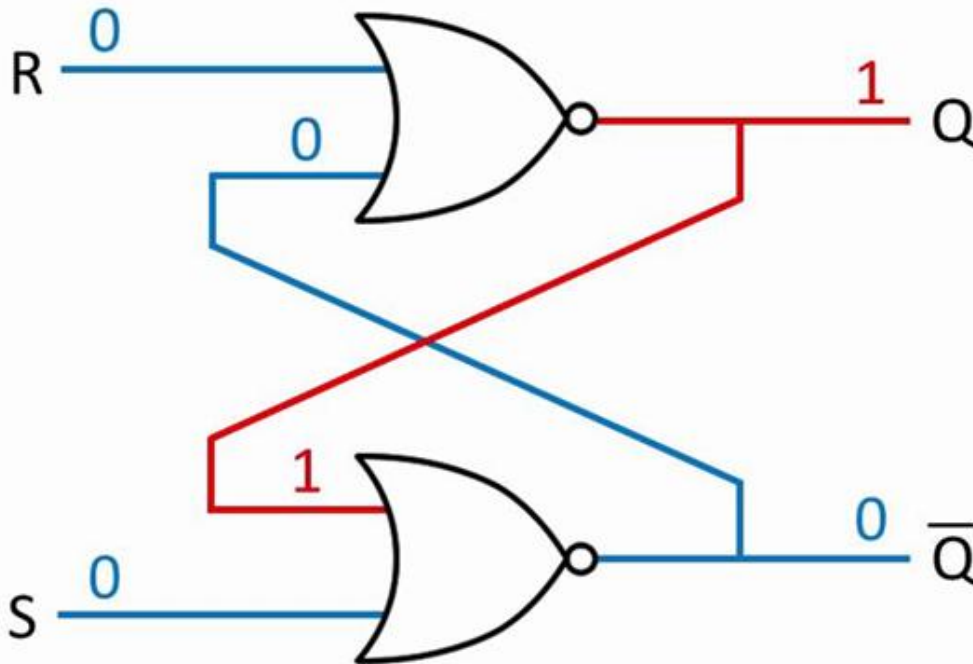
Latch Using NAND Gate

For *active-LOW* input S'-R' latch (also known as NAND gate latch)



Latch Using NOR Gate

- Complementary outputs: **Q** and **Q'** .
- When **Q** is **HIGH**, the latch is in **SET** state.
- When **Q** is **LOW**, the latch is in **RESET** state.
- For *active-HIGH input S-R latch* (also known as NOR gate latch)



Truth Table

S	R	Q	$\bar{Q}$
0	0	1	0
0	0	0	1
0	1	0	1
1	0	1	0
1	1	0	0

Drawbacks

- Latch circuits are **not suitable in synchronous logic circuits**.
- When the enable signal is active, the excitation inputs are gated directly to the output Q. Thus, any change in the excitation input immediately causes a change in the latch output.
- The problem is solved by using a special timing control **signal called a clock** to restrict the times at which the states of the memory elements may change.
- This leads us to the **edge-triggered memory elements called *flip-flops***.





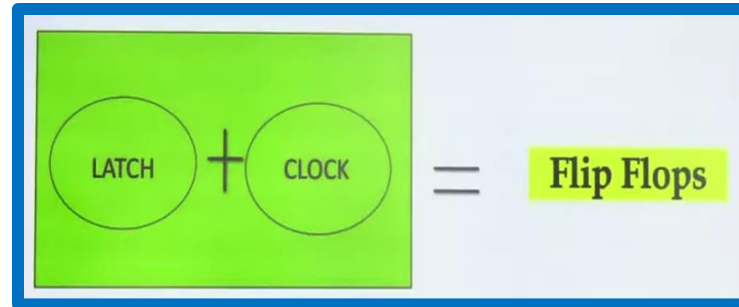
संयुक्त प्रौद्योगिकी

SYMBIOSIS

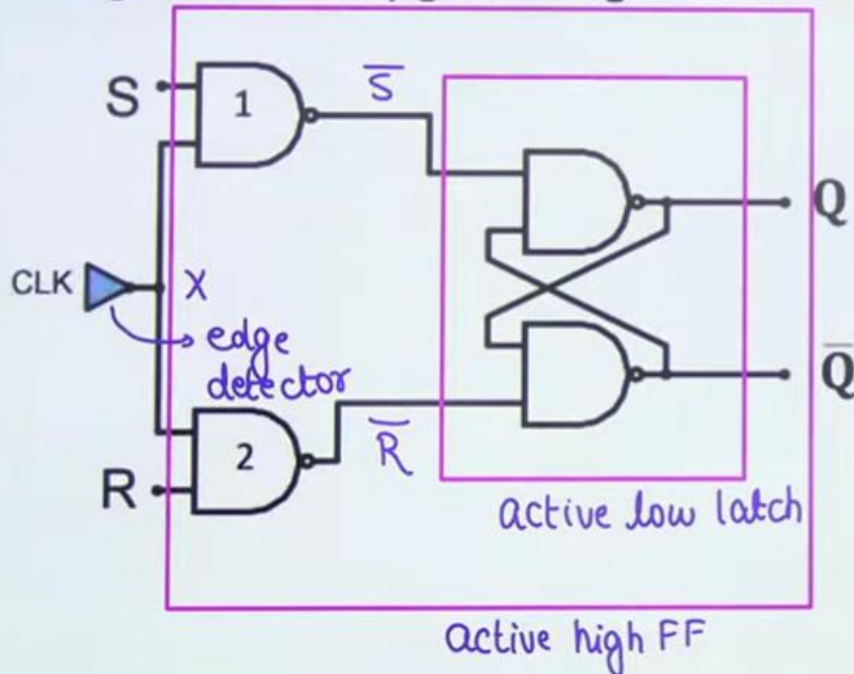
INSTITUTE OF TECHNOLOGY, NAGPUR

Cont...

SR Flip-Flop



Nand gates 1 & 2 are used here to hold the state of Nand later when the clock is 0 Nand gate is the only gate that gives 1 when one of its i/p is 0



latch (NAND)

CLOCK	S	R	$\bar{S}$	$\bar{R}$	Q_{n+1}	State
no edge	X	X	X	X	Q_n	HOLD
↑	0	0	1	1	Q_n	HOLD
↑	0	1	1	0	0	RESET
↑	1	0	0	1	1	SET
↑	1	1	0	0	1	INVALID



Characteristics Table

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

$Q_{n+1} = Q_n$

Hold

Reset

irrespective of Q_n

$Q_{n+1} = 0$

Set irrespective of Q

$Q_{n+1} = 1$

Invalid

In char. table, we also consider present state as an input.

→ very important to derive logical expression for next state.

$Q_{n+1} = \sum m(1, 4, 5, 6, 7)$

$$Q_{n+1} = \sum m(1, 4, 5) + \sum d(6, 7)$$

Characteristics Equation

RQ	00	01	11	10
0		1		
1	1	1	X	X

$S + \bar{R}Q_n$ with $S = R \neq 1$
 Or $S \cdot R \neq 1$

$$Q_{n+1} = S + \bar{R}Q_n$$

based on knowledge of
 inputs & present state
 we can judge the output

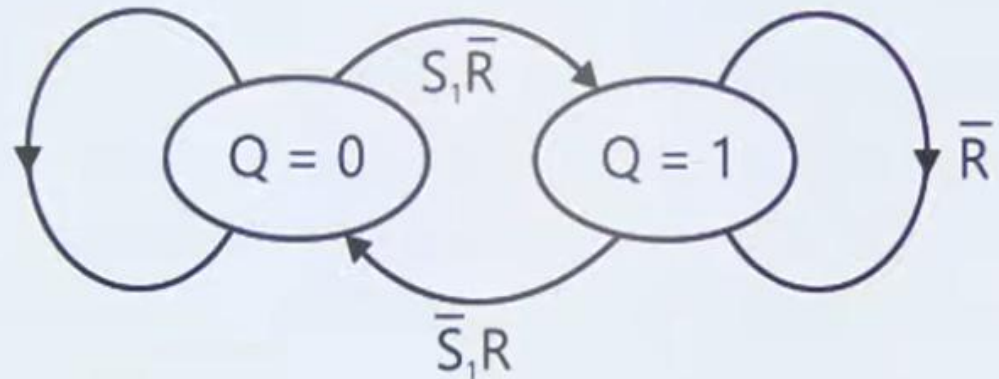
Drawback

4th State is Invalid, to
 solve this problem, we
 will use a **JK Flip-Flop**.

Excitation Table

Outputs		Inputs	
Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

State Diagram





Cont...

Q1 A flip-flop is an _____

- A. An Edge sensitive device
- B. Synchronous device
- C. Both a and b
- D. None of the above

Ans-1: C

When $S=0$, $R=0$, $CLK=X$ then the output will be _____

- A. No change
- B. Set
- C. Reset
- D. Invalid

Ans-2: A

The flip flop is a _____ device

- A. Unstable
- B. Bi-stable
- C. Both a and b
- D. None of the above

Ans-3: C

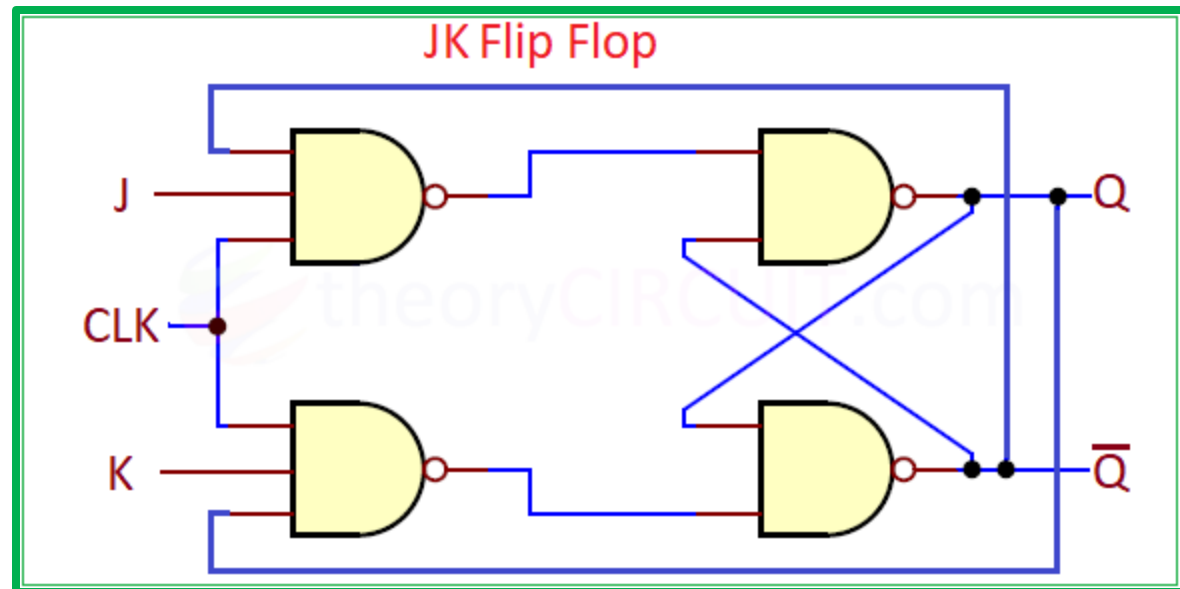
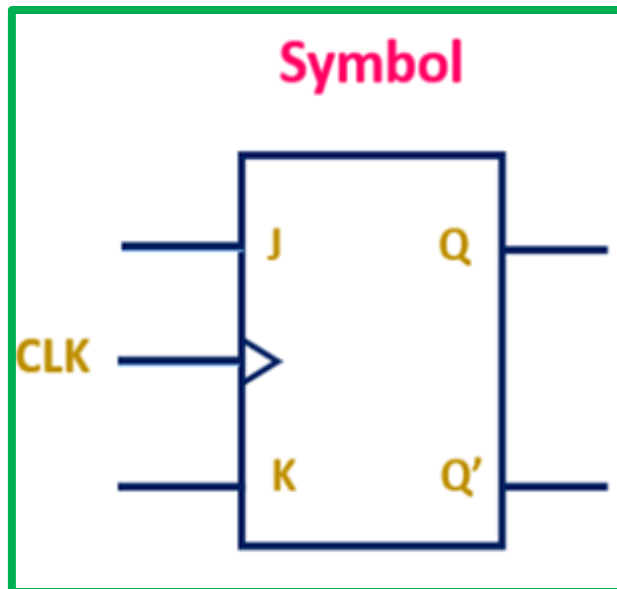
Q.5 In a clocked S - R flip-flop made with NAND gates, the input combinations $S=1$, $R=1$, are not permitted because it leads to

- (a) reset set of the Flip-Flop
- (b) both the outputs are zero simultaneously
- (c) an unpredictable output
- (d) the output becomes available before the clock goes low

Ans-4: C

J-K Flip-Flop

- A JK flip-flop is a sequential logic circuit used for storing a single bit of information, **named after its inventor, Jack Kilby**.
- It has two inputs, J and K, which, along with a clock signal, determine the next state of the output (Q).
- It can **set, reset, or toggle its output**, making it a versatile building block for counters and other digital circuits.





Cont...

Truth Table

J	K	Q_{n+1}
0	0	Q_n (No change)
0	1	0 (Reset)
1	0	1 (Set)
1	1	$\overline{Q_n}$ (toggle)

Excitation Table

CLK	J	K	Q_n	Q_{n+1}	Q_{n+1}
0	X	X	0/1	0/1	Q_n
↑	0 0	0 0	0 1	0 1	Q_n
↑	0 0	1 1	0 1	0 0	0
↑	1 1	0 0	0 1	1 1	1
↑	1 1	1 1	0 1	1 0	Q_n'

$$Q_{n+1} = \Sigma m(1, 4, 5, 6)$$



Characteristics Equation

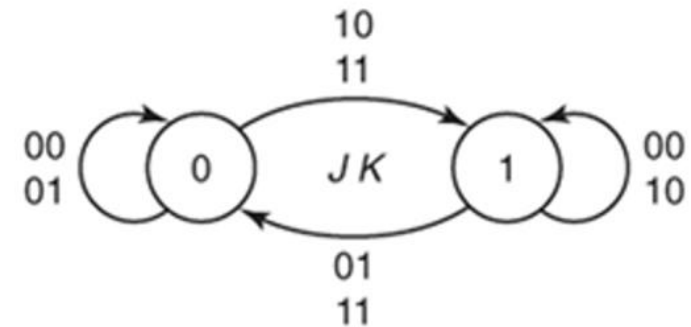
JK Q _n	00		01		11		10	
	0	1	2	3	4	5	6	7
0			1	1				
1	1						1	

$$Q_{n+1} = JQ_n' + K'Q_n$$

Excitation Table

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

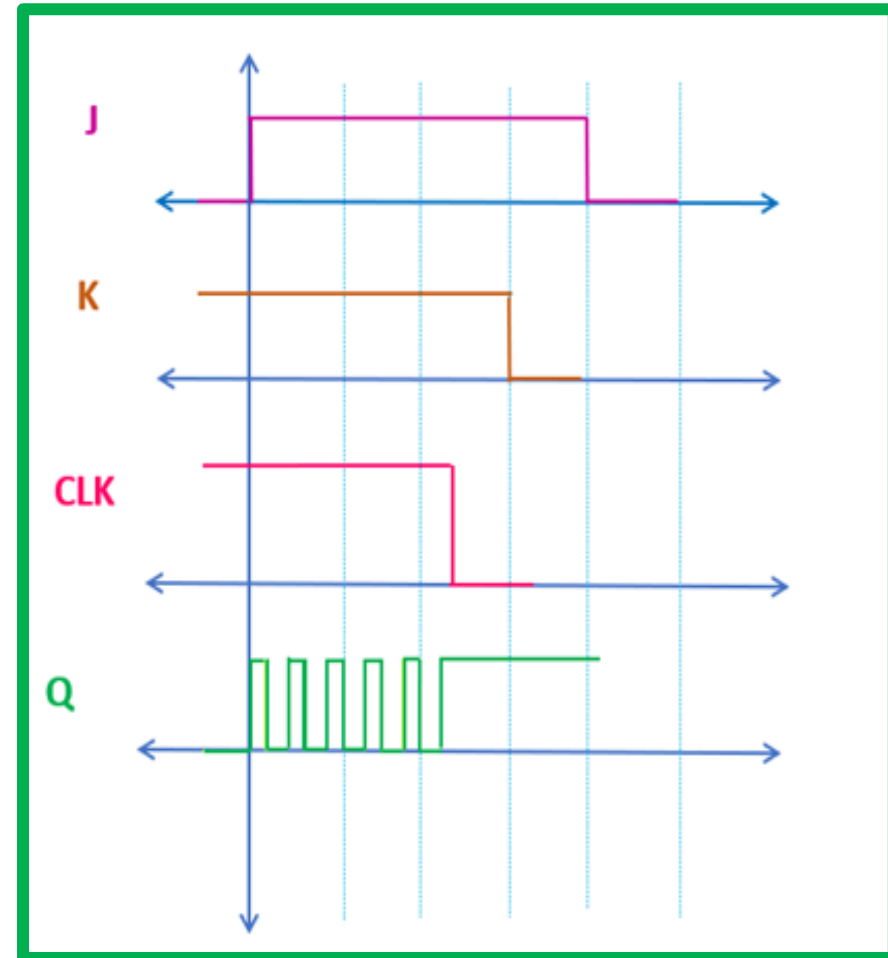
State Diagram



‘X’ represents that the value of input variable can be either ‘0’ or ‘1’.

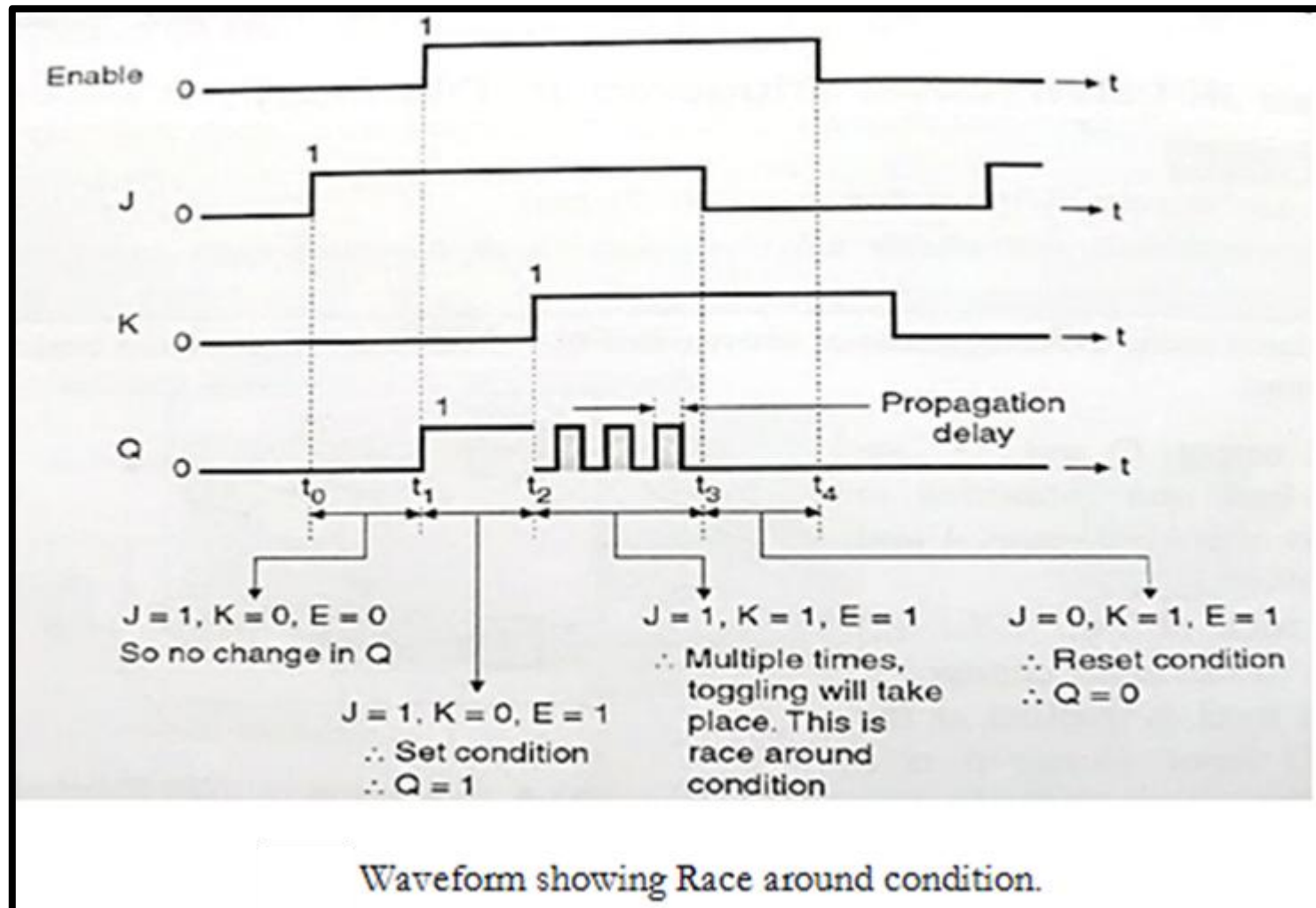
Race-Around Condition

- In the level triggered JK Flip-Flop, when $J=K=1$, the ON time of the clock is more than the propagation delay of the JK Flip-Flop then the output of the flip-flop may toggle continuously between '1' and '0'. This condition is known as the Race Around condition.
- The issue of Race Around Condition can be resolved using the Master-Slave Flip-Flop.

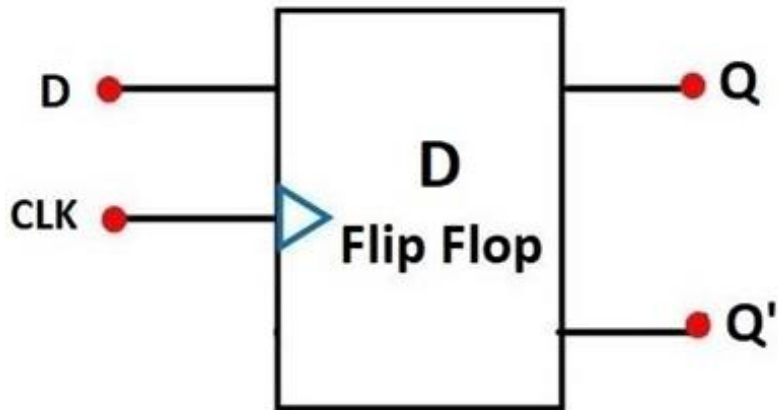




Cont...



D Flip-Flop

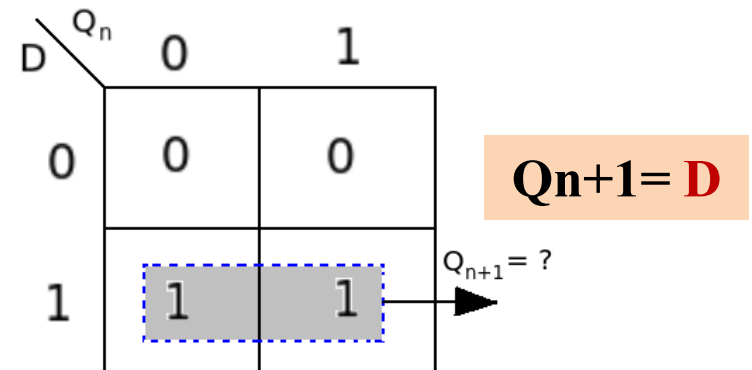


Truth Table

D Flip Flop		
Input	Output	
D	Q	$Q^{\wedge}$
0	0	1
1	1	0

Characteristics Table

D	Q_n	Q_{n+1}	
0	0	0	Reset
0	1	0	
1	0	1	Set
1	1	1	



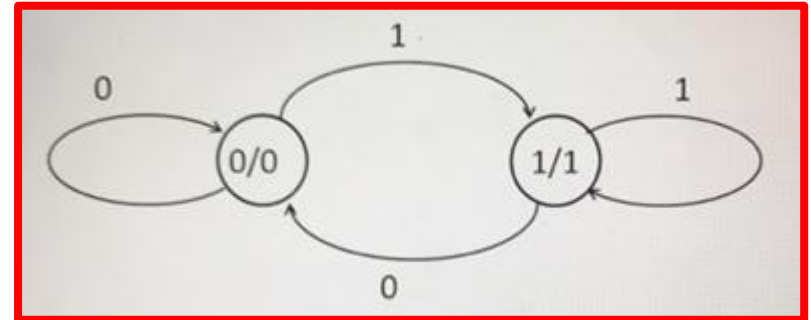


Cont...

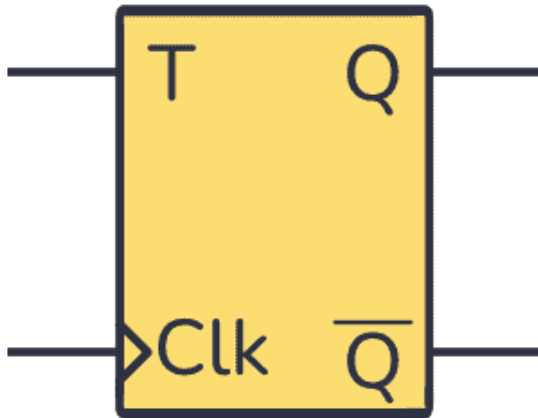
Excitation Table

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

State Diagram



T Flip-Flop

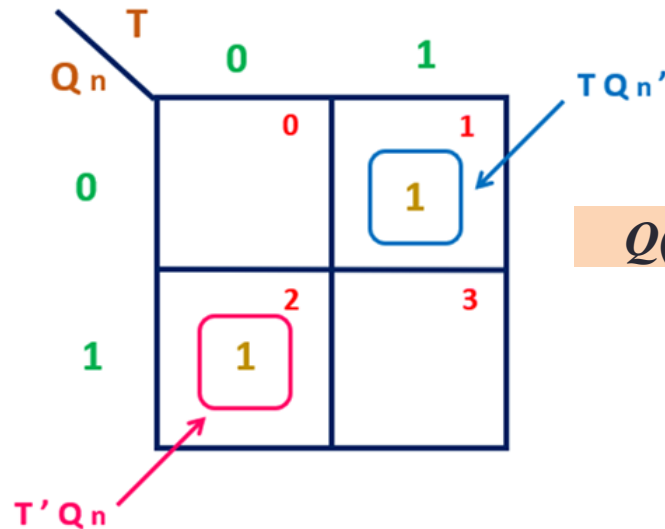


Truth Table

T	CLK	$Q(t+1)$	Comments
0	↑	$Q(t)$	No change
1	↑	$Q(t)'$	Toggle

Characteristics Table

Q	T	$Q(t+1)$
0	0	0
0	1	1
1	0	1
1	1	0



$$Q(t+1) = T.Q' + T'.Q$$

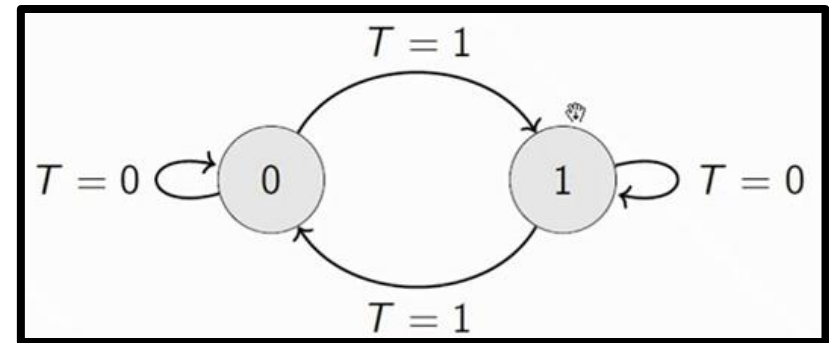
Cont...

Excitation Table

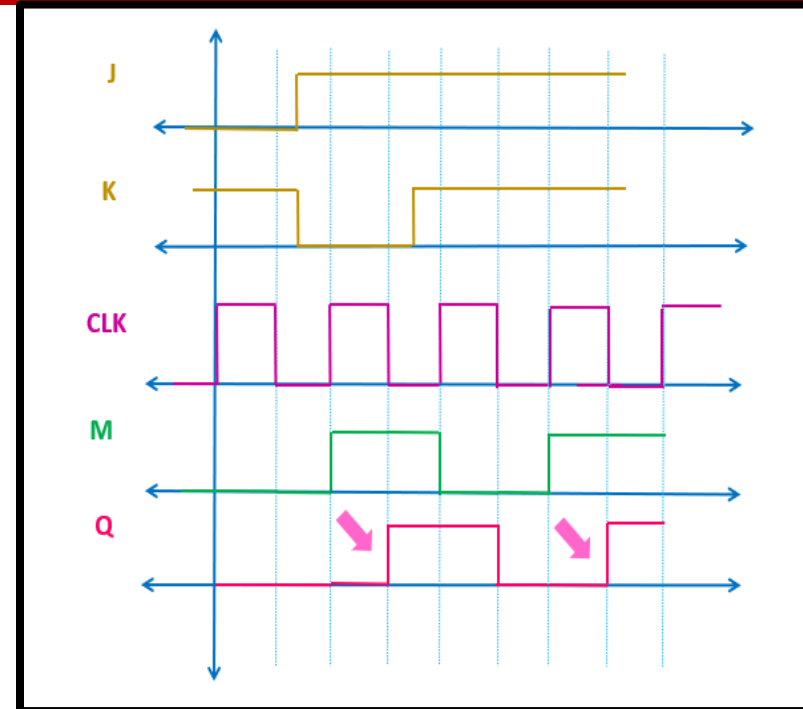
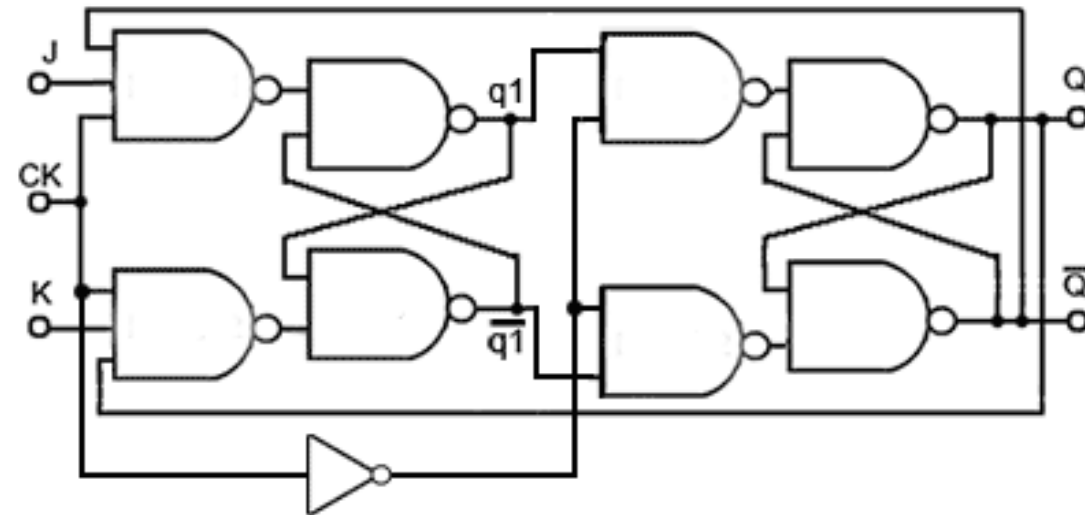
Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Excitation table of T flip flop

State Diagram



Master Slave JK-Flip-Flop





Cont...

Digital Electronics Introduction to Registers

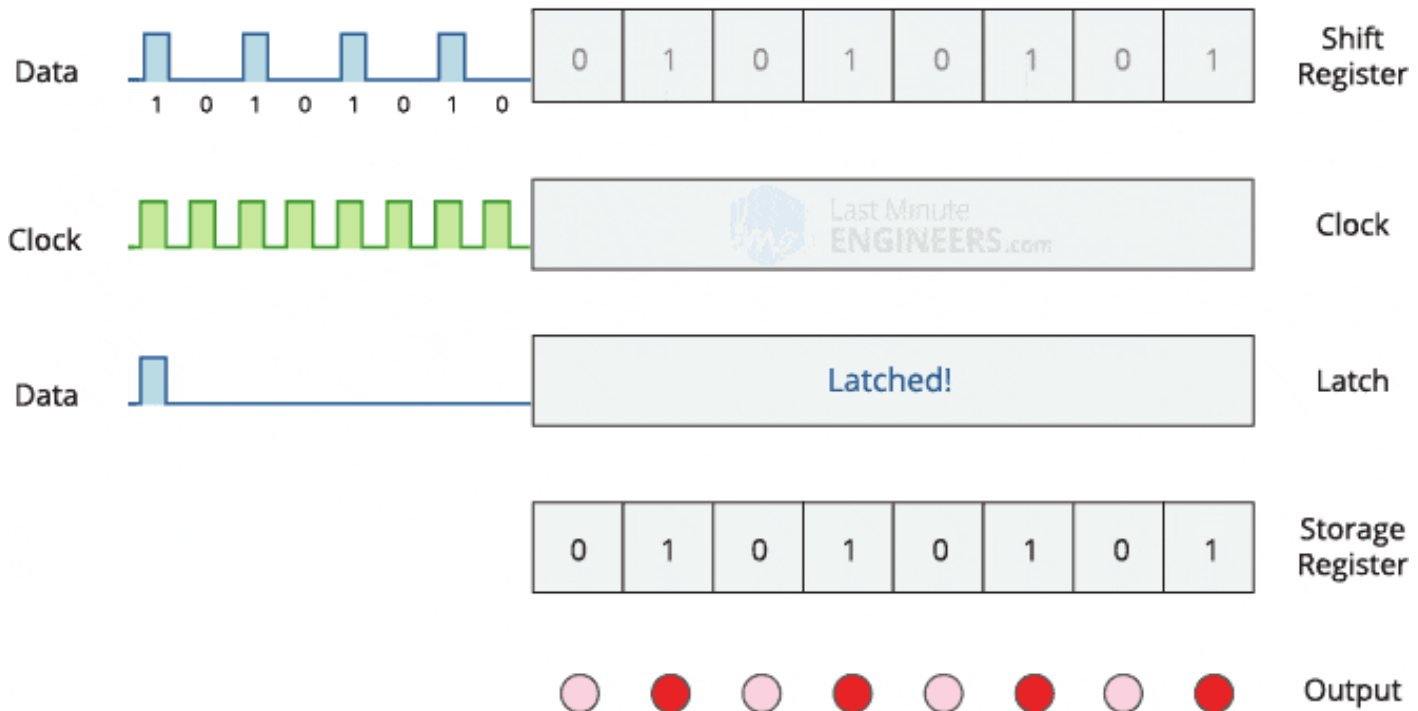


Introduction to Register

- **Flip-flop is a 1 bit memory cell** which can be used for storing the digital data.
- **To increase the storage capacity** in terms of number of bits, **we have to use a group of flip-flop**. **Such a group of flip-flop is known as a Register**.
- The **n-bit register will consist of n number of flip-flop** and it is capable of storing an n-bit word.
- A register is a digital circuit with **two basic functions**: **data storage** and **data movement**.
- The **information stored within the registers can be transferred with the help of shift registers**.
- **Shift Register is a group of flip flops used to store multiple bits of data**.

Cont...

- The **bits stored in such registers can be made to move within the registers** and in/out of the registers **by applying clock pulses**.
- The **registers which will shift the bits to left** are called **"Shift left registers"**.
- The **registers which will shift the bits to right** are called **"Shift right registers"**.



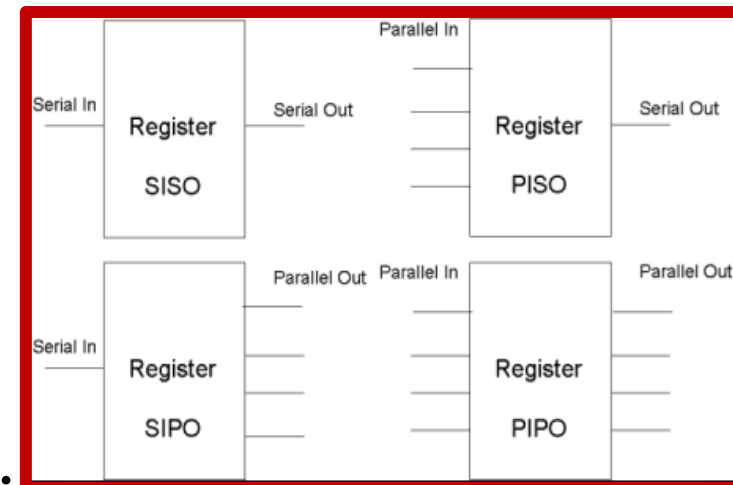
Cont...

Shift Register

- A register which is capable of shifting the binary information hold in each Flip-flop to its next Flip-flop in a selected direction (either Left or Right is know as **Universal Shift Register**) is called **shift register**.
- A **shift register** which can **shift the data in both directions** is called a **bi-directional shift register**.

Type of Shift Register

- 1) Serial in and serial out Shift register (**SISO**).
- 2) Serial in and Parallel out Shift register (**SIPO**).
- 3) Parallel in and serial out Shift register (**PISO**).
- 4) Parallel in and Parallel out Shift register (**PIPO**).





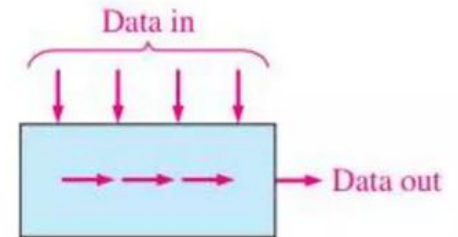
Data Movement in Shift Registers



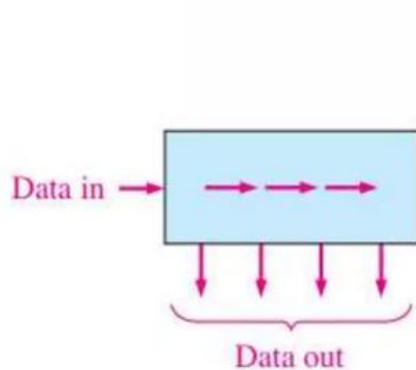
(a) Serial in/shift right/serial out



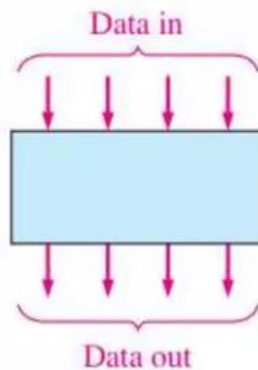
(b) Serial in/shift left/serial out



(c) Parallel in/serial out



(d) Serial in/parallel out



(e) Parallel in/parallel out



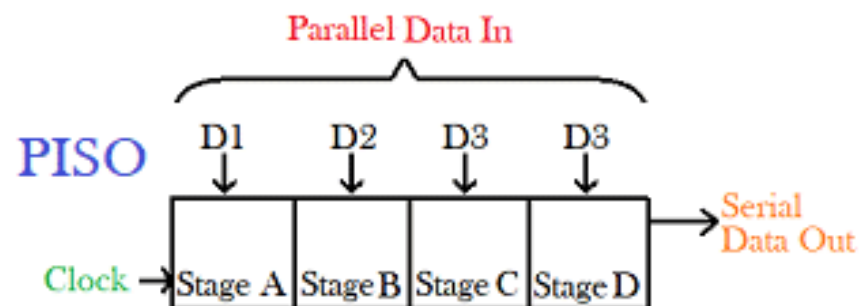
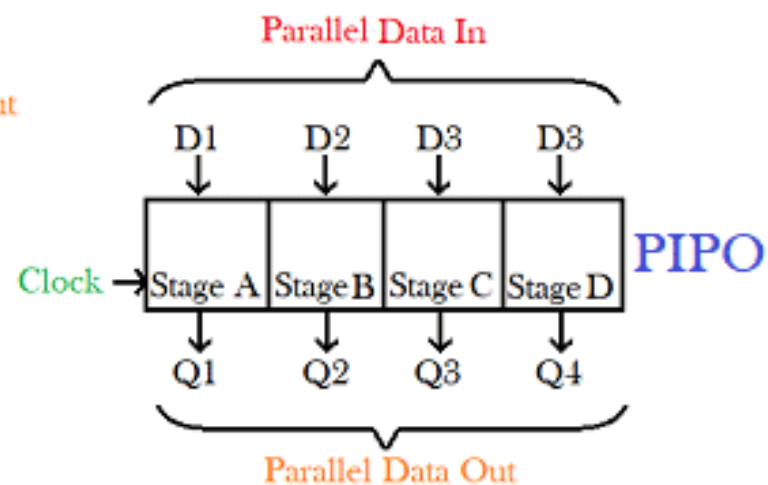
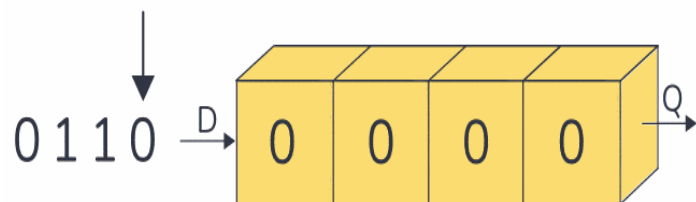
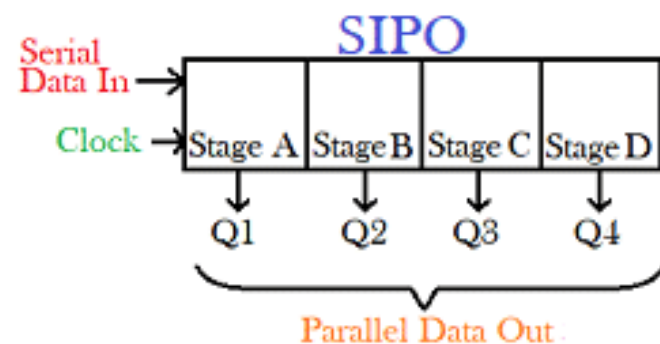
(f) Rotate right



(g) Rotate left



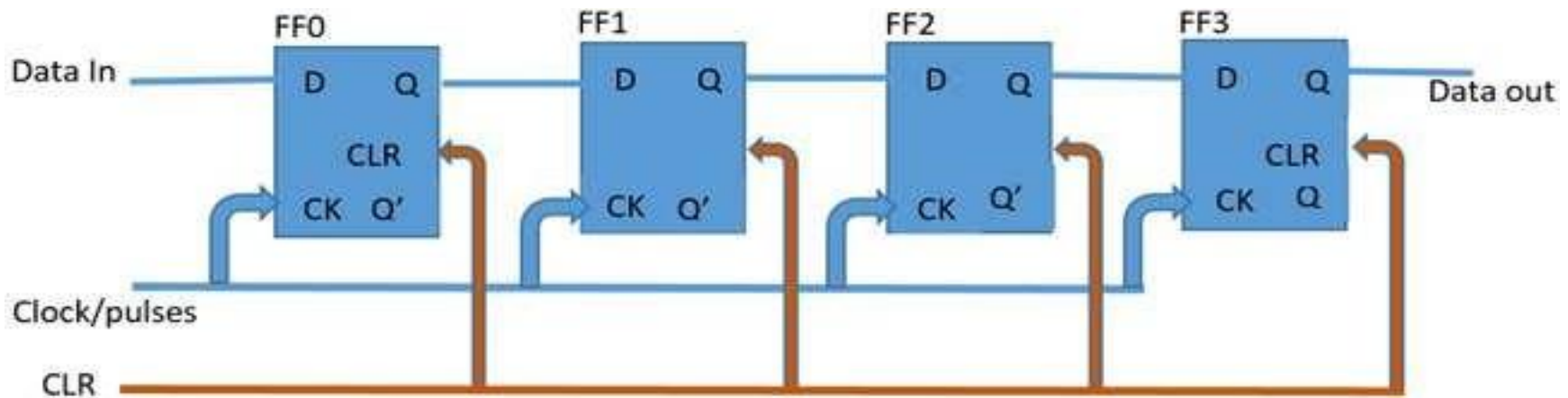
Cont...



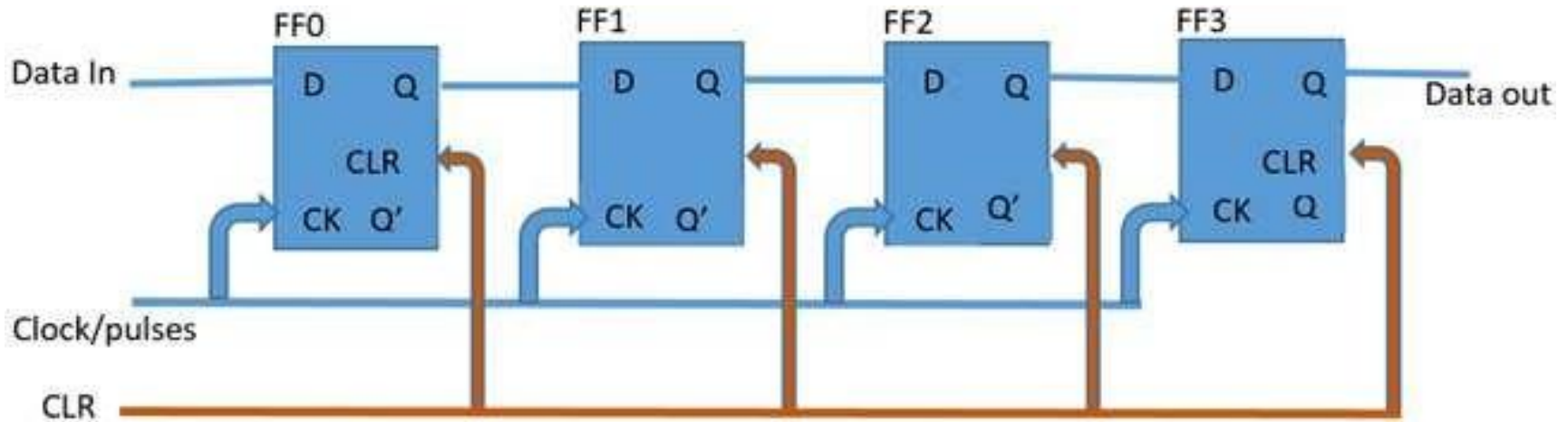


(SISO)

- **Serial in – Serial out** shift registers are shift registers that **streams in data serially (one bit per clock cycle)** and **streams out data too in the same way, one after the other.**
- A simple **serial in – serial Out 4-bit shift register** is shown in figure, the register consists of 4 flip flops.
- On startup, the **shift register is first cleared, forcing the outputs of all flip flops to zero**, **the input data is then applied to the input serially, one bit at a time.**



Cont...



Input: 1101

First clock cycle:

Second clock cycle:

Third clock cycle:

Fourth clock cycle:

FF0	FF1	FF2	FF3
1	0	0	0

FF0	FF1	FF2	FF3
0	1	0	0

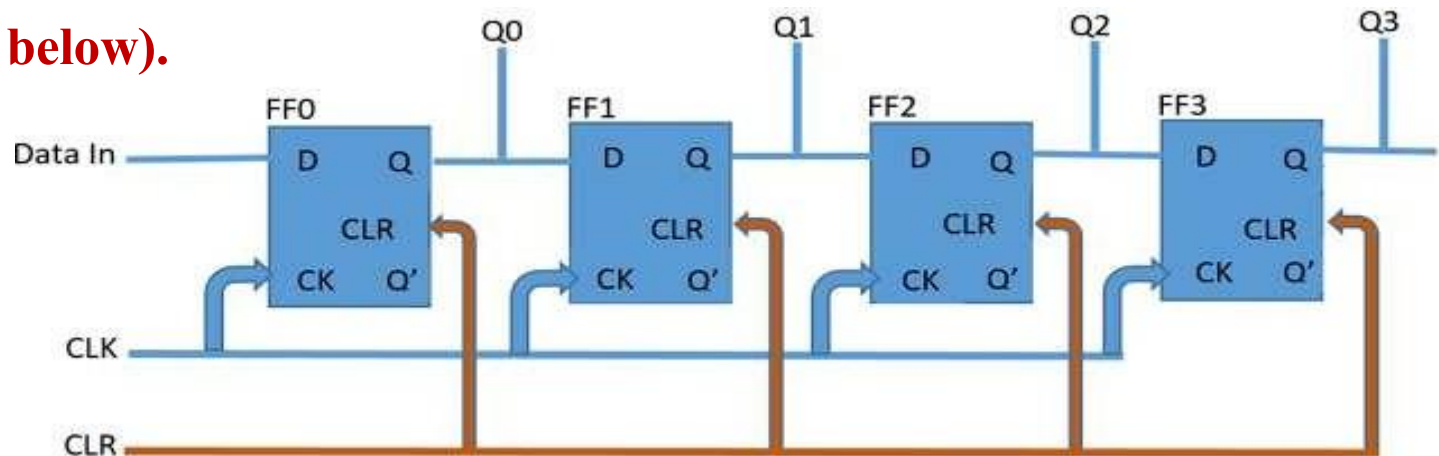
FF0	FF1	FF2	FF3
1	0	1	0

FF0	FF1	FF2	FF3
1	1	0	1



(SIPO)

- The **second type of shift register** we will be considering is the **Serial in – Parallel out shift register**.
- These types of shift registers are **used for the conversion of data from serial to parallel**.
- The data comes in one after the other per clock cycle and can either be shifted and replaced or be **read off at each output**.
- This means **when the data is read, each read in bit becomes available simultaneously on their respective output line (Q0 – Q3 for the 4-bit shift register shown below)**.





Cont...

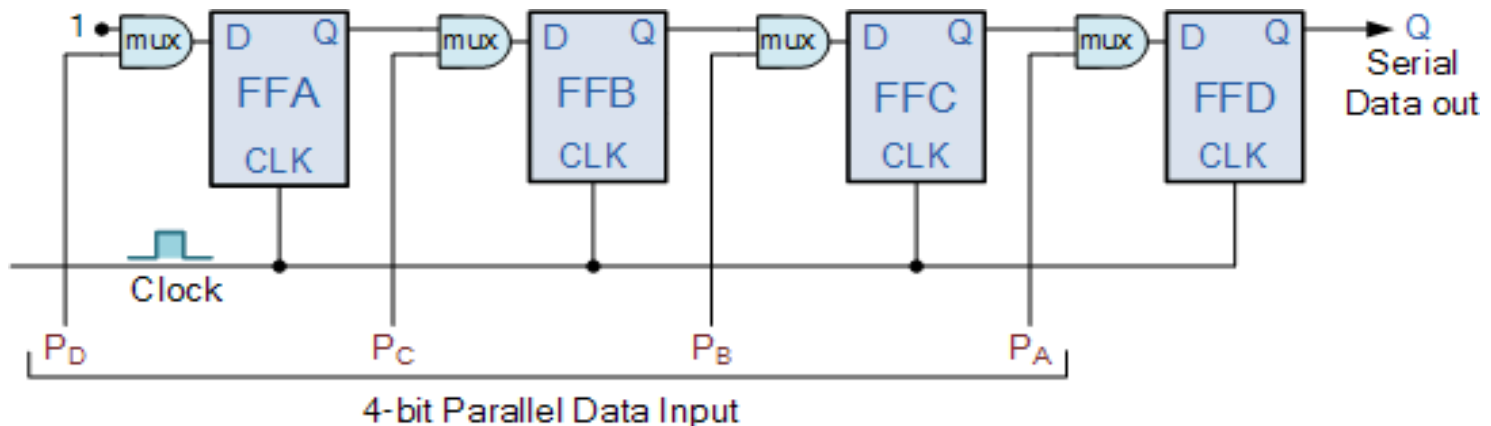
A table showing how data gets shifted out of serial in – parallel out 4 bit shift register is shown below, with the data in as 1001.

Clear	FF0	FF1	FF2	FF3
1001	0	0	0	0
Clk-1	1	0	0	0
Clk-2	0	1	0	0
Clk-3	0	0	1	0
Clk-4	1	0	0	1

A good example of the serial in – parallel out shift register is the 74HC164 shift register, which is an 8-bit shift register.

(PISO)

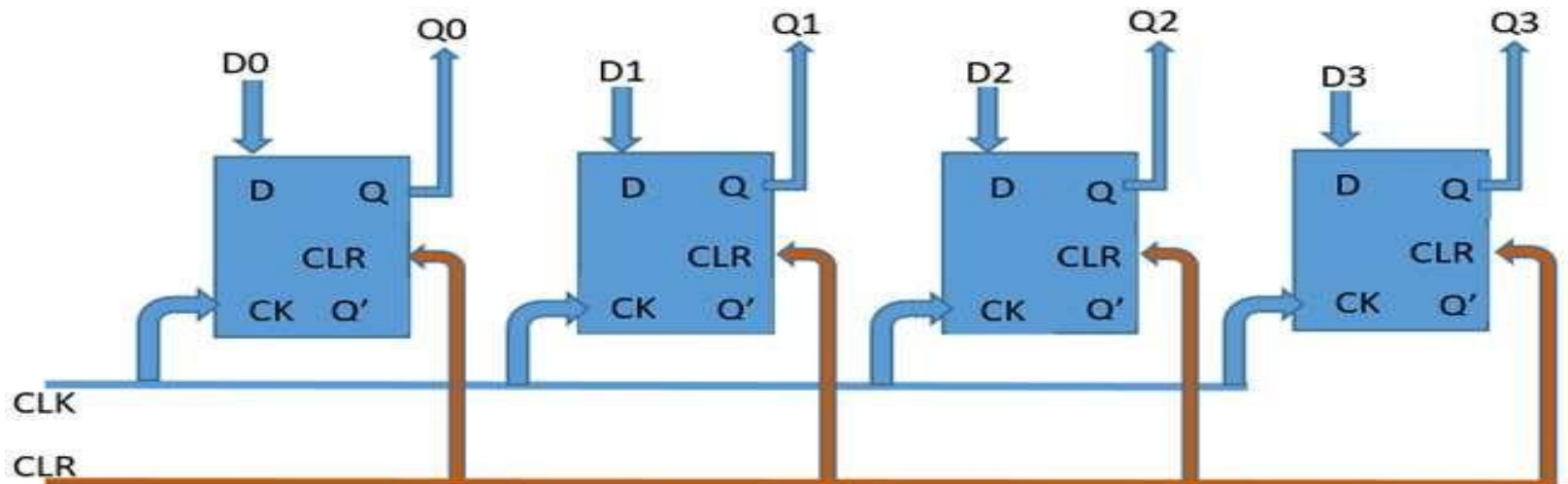
- This register can be used to store and shift a 4-bit word, **with the write/shift (WS) control input controlling the mode of operation of the shift register.**
- When the WS control line is low (Write Mode), **data can be written and clocked in via D0 to D3.**
- To shift the data out serially, the WS control line is brought HIGH (Shift mode), **the register then shifts the data out on clock input.**
- **A good example of a parallel in – serial out shift register is the 74HC165 8-bit shift register** although it **can also be operated as a serial in – serial out shift register.**





(PIPO)

- The input data at each of the **input pins from D0 to D3** are read in at the same time when **the device is clocked and at the same time**, the data read in from each of the inputs is passed out at the corresponding output (from Q0 to Q3).
- The **74HC195 shift register** is a multipurpose shift register which is capable of working in most of the modes described by all the types we have discussed so far **especially as a parallel in – parallel out shift register**.

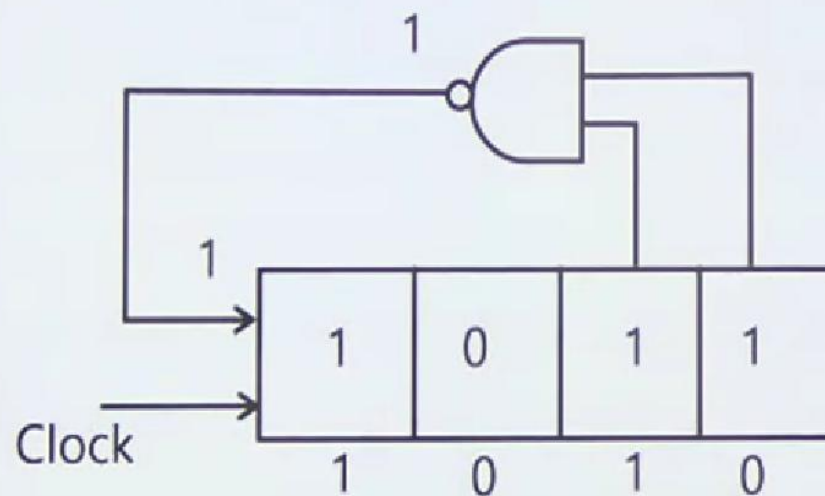




Cont...

Q. CKT ^{shown} ~~strain~~ in fig is a 4 bit SIPO reg. initially loaded with 1010 than content of reg after 3 clock pulse will be

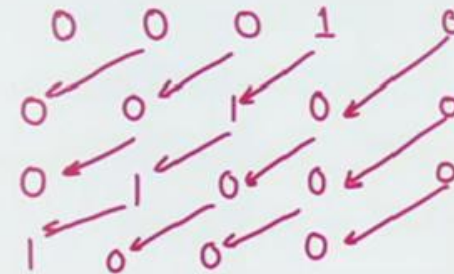
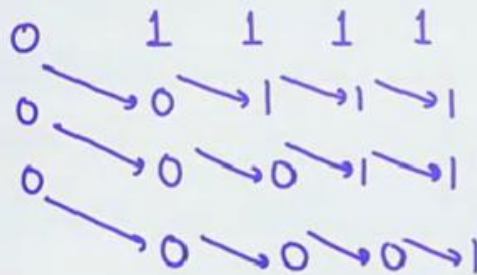
	Q_3	Q_2	Q_1	Q_0
0	1	0	1	0
1	1	1	0	1
2	1	1	1	0
3	1	1	1	1



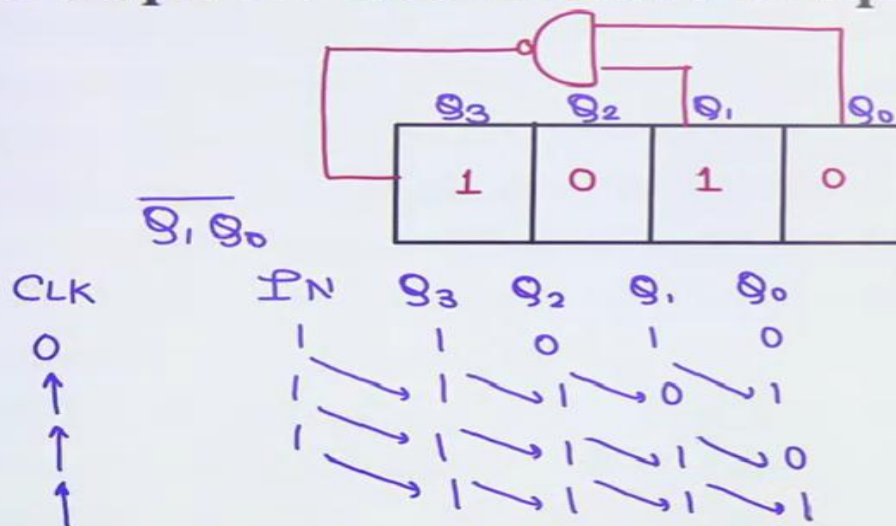


Problems

Q.1 A 4 Bit SISO Register, is initially Loded with Bit pattern 1111
The its shifted 3 counts to right & 3 counts to left the data stored will be C
assume both right shift & left shift is done through 0



If it N bit SIPO Reg N Clock Pulses are required for serial input & for parallel output no Clock Pulses are required.



right shift
data after 3 clock pulses

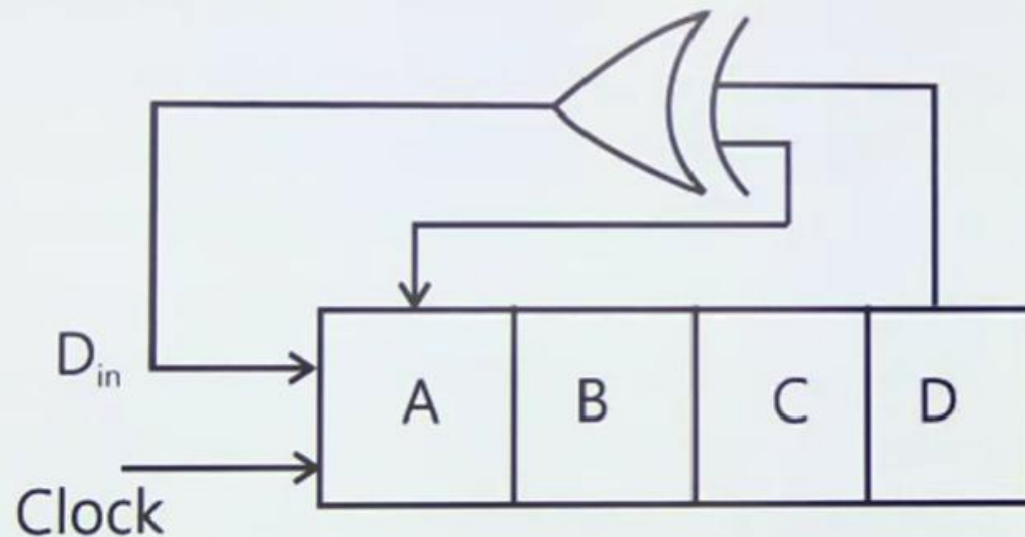


Cont...

A 4 bit shift reg. configured for right, shift, operation like

$O_{in} \rightarrow A, A \rightarrow B, B \rightarrow C, C \rightarrow D$

If the present state of shift reg. is ABCD $\rightarrow$ 1101 Then the No of clock cycles req. to reach the state ABCD -1111 is

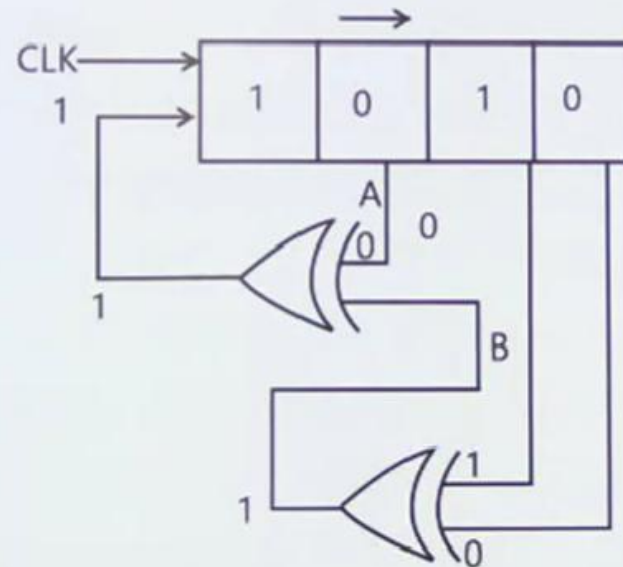




Cont...

Q. The shift Reg. Shown in given fig. is initially loaded with bit pattern 1010 then the clock is applied & with each clock pulse shifted by 1 bit position to the right. The content.

Right with each shift the bit at serial I/P is pushed to MSB position after how many clock pulses will the content of reg become 1010 again.



Counter

- **Introduction**
- **Building Blocks**
- **Classification/Types**

Asynchronous (Ripple) Counters

Synchronous (Parallel) Counters

Up/Down Counters

Decade (BCD), Ring, and Johnson Counters

- **Applications**

Introduction

- A **counter** is a **sequential logic circuit** that **counts the number of input pulses** and generates output sequences in a specific order.
- It is widely used in **digital systems** for applications such as **event counting, frequency division, time measurement, and sequence generation.**
- In simple terms, a counter **advances its state (binary value)** by one each time it receives a clock pulse.
- The **number of unique states** it can represent depends on the **number of flip-flops used.**

Cont...

Basic Functions

➤ Counting Pulses:

It records the number of clock pulses or external events.

➤ Generating Sequences:

Produces a predefined binary or coded sequence (e.g., $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow \dots$).

➤ Frequency Division:

Each flip-flop in the counter divides the input clock frequency by 2, thus a counter can act as a **frequency divider**.



Building Blocks

- Counters are **sequential logic circuits**, meaning their output depends on both **present input** and **previous states**.
- They are mainly composed of **flip-flops** (typically JK, T, or D flip-flops).
- Each flip-flop represents **one binary bit (0 or 1)**.
- Flip-flops are connected in a specific configuration **(either asynchronous/ripple or synchronous)** to form the complete counter circuit.

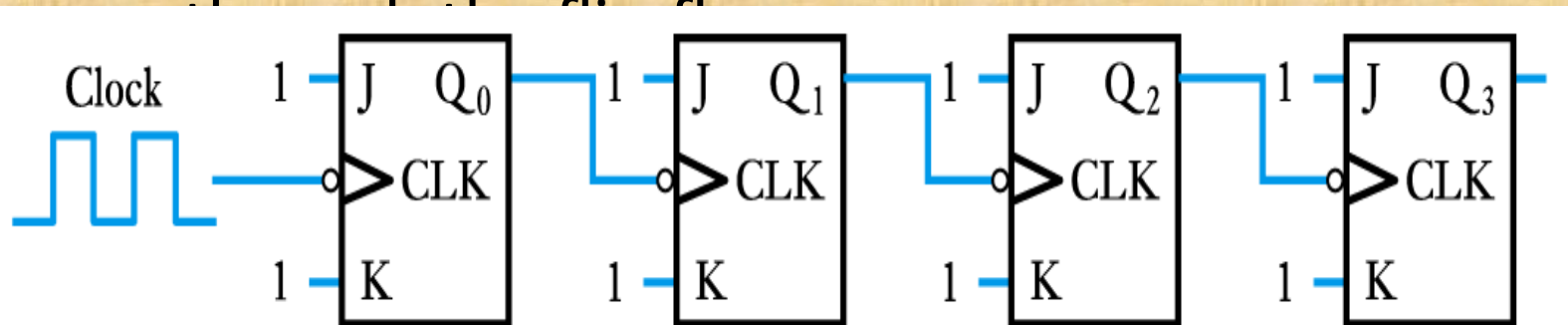
Asynchronous (Ripple) Counters

- The counter is called **ripple counter** because **the output of one flip-flop feed to the clock input of another.**
- In **asynchronous circuits** we are allowed **to play with the clock.**
- We use **trailing edge Flip-Flop**. The flip flops respond to **negative going clock edge.**
- **T Flip-Flop** should have its input connected **to 1**. **D Flip-Flop** should have its input connected **to the complement** of its output.
- Each flip flop divides the **incoming clock pulse frequency by a factor of 2.**

Classifications of Counters

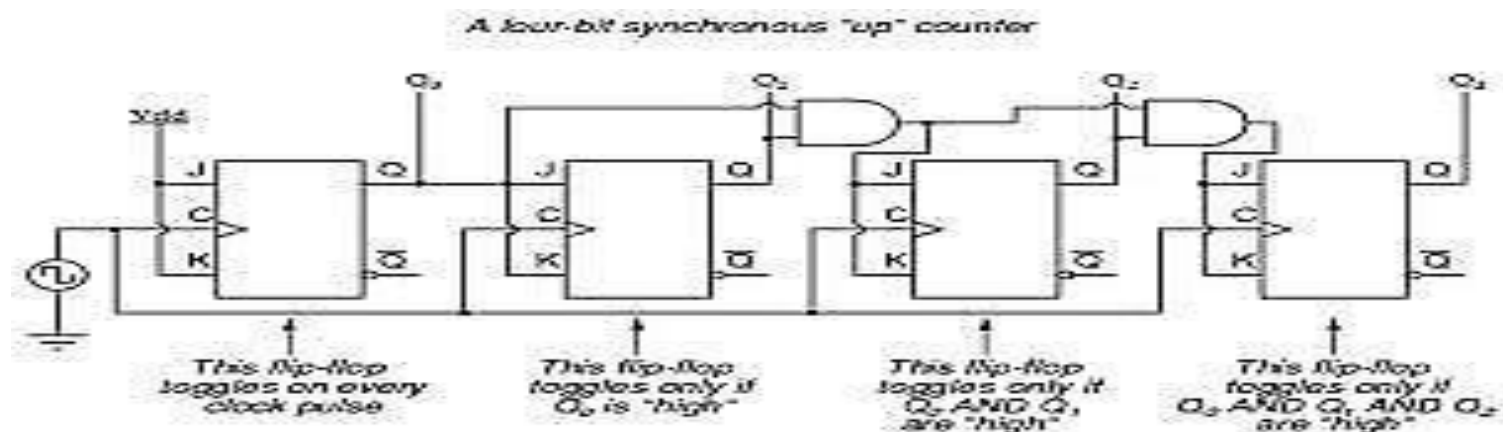
Asynchronous Counters

- Only the first flip-flop is clocked by an external clock. All subsequent flip-flops are clocked by the output of the preceding flip-flop. means output of previous flip-flop is connected to clock input of next flip flop.
- Asynchronous counters are slower than synchronous counters because of the delay in the transmission of the pulses from flip-flop to flip-flop.
- Asynchronous counters are also **called ripple-counters** because of the way the clock pulse ripples it



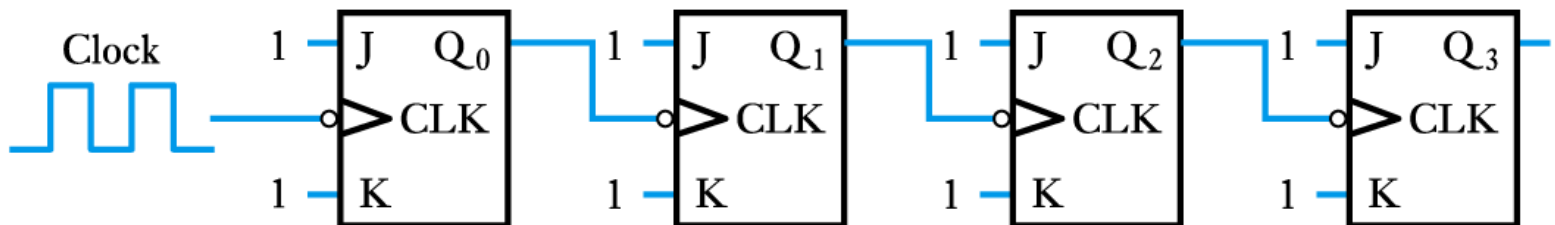
Synchronous Counters

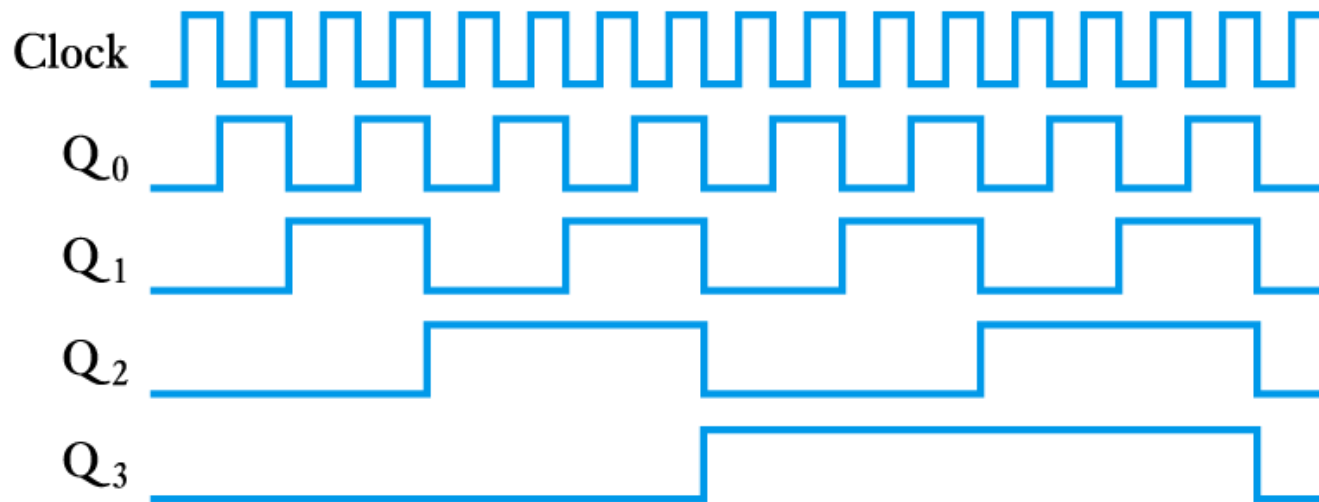
- All flip-flops are clocked simultaneously by an external clock. Means clock input of all flip flops are connected to same external clock.
- Synchronous counters are faster than asynchronous counters because of the simultaneous clocking.
- Synchronous counters are an example of *state machine* design because they have a set of states and a set of transition rules for moving between those



Asynchronous Counters

- **Asynchronous Counter/Ripple counters**
 - can be constructed using several flip flops
 - consider the following arrangement
 - with $J = K = 1$ each flip flop toggles on the falling edge of its clock input



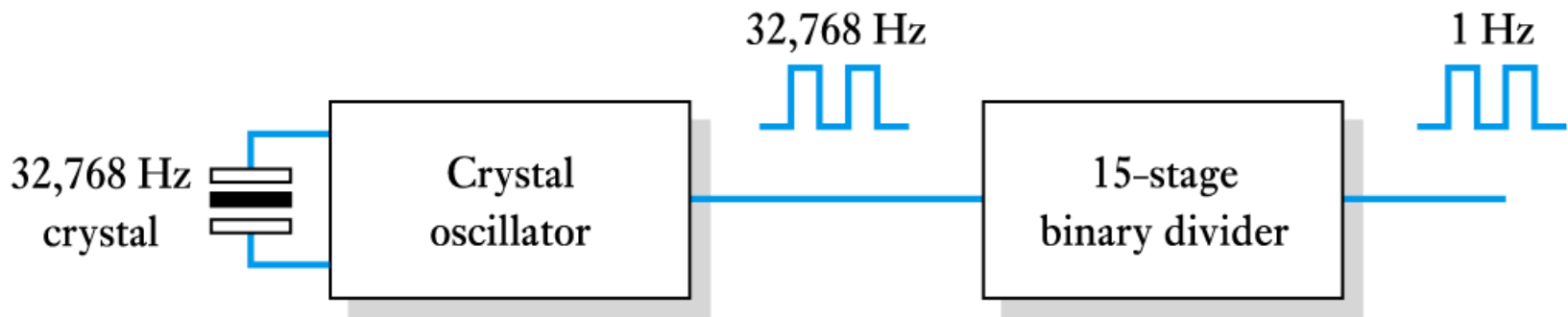


- Each stage toggles at half the frequency of the previous stage
 - acts as a frequency divider
 - divides frequency by 2^n (n is the number of stages)

- Application of a frequency divider

Clock generator for a digital watch

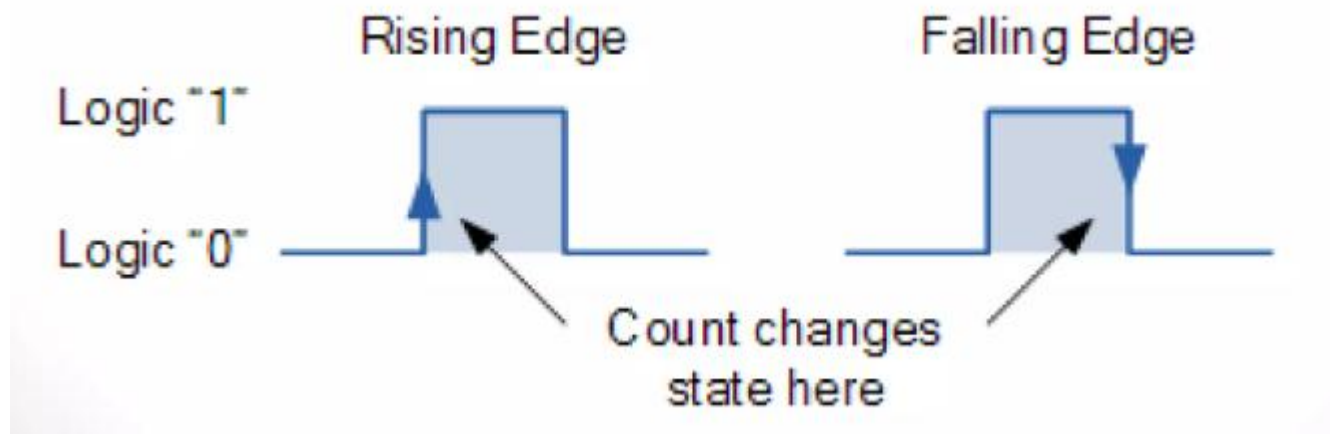
- 15-stage counter divides signal from a crystal oscillator by 32,768 to produce a 1 Hz signal to drive stepper motor or digital display





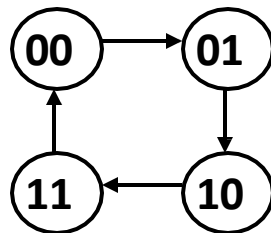
Synchronous (Parallel) Counters

- **Synchronous Counters** are so called because the clock input of all the individual flip-flops within the counter are all clocked together at the same time by the same clock signal.
- The clock signal is directly applied to all Flip flops in a parallel sequence.



Synchronous (Parallel) Counters

- **Synchronous (parallel) counters:** the flip-flops are clocked at the same time by a common clock pulse.
- We can design these counters using the sequential logic design process
- Example: 2-bit synchronous binary counter (using T flip-flops, or JK flip-flops with identical J,K inputs).



Present state		Next state		Flip-flop inputs	
A_1	A_0	A_1^+	A_0^+	TA_1	TA_0
0	0	0	1	0	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1

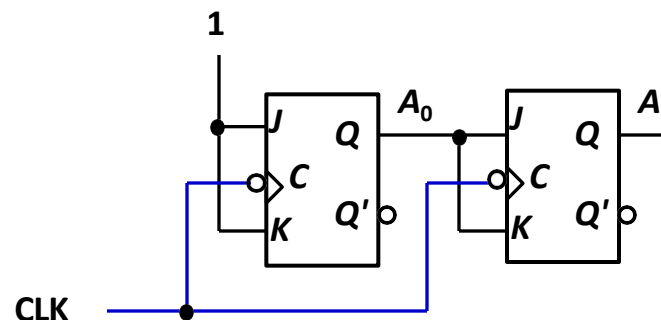
Synchronous (Parallel) Counters

- Example: 2-bit synchronous binary counter (using T flip-flops, or JK flip-flops with identical J,K inputs).

Present state		Next state		Flip-flop inputs	
A_1	A_0	A_1	A_0	TA_1	TA_0
0	0	0	1	0	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1

$$TA_1 = A_0$$

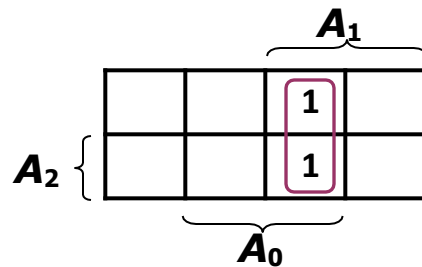
$$TA_0 = 1$$



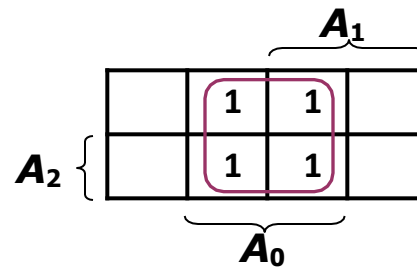
Synchronous (Parallel) Counters

- Example: 3-bit synchronous binary counter (using T flip-flops, or JK flip-flops with identical J, K inputs).

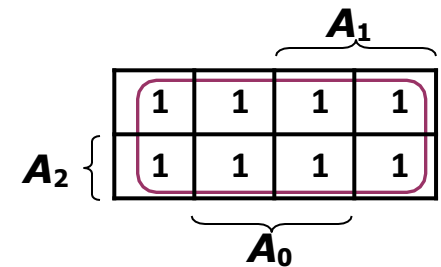
Present state			Next state			Flip-flop inputs		
A_2	A_1	A_0	A_2^+	A_1^+	A_0^+	TA_2	TA_1	TA_0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1



$$TA_2 = A_1 \cdot A_0$$

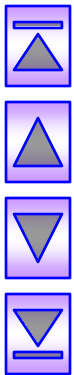


$$TA_1 = A_0$$



$$TA_0 = 1$$

Synchronous (Parallel) Counters

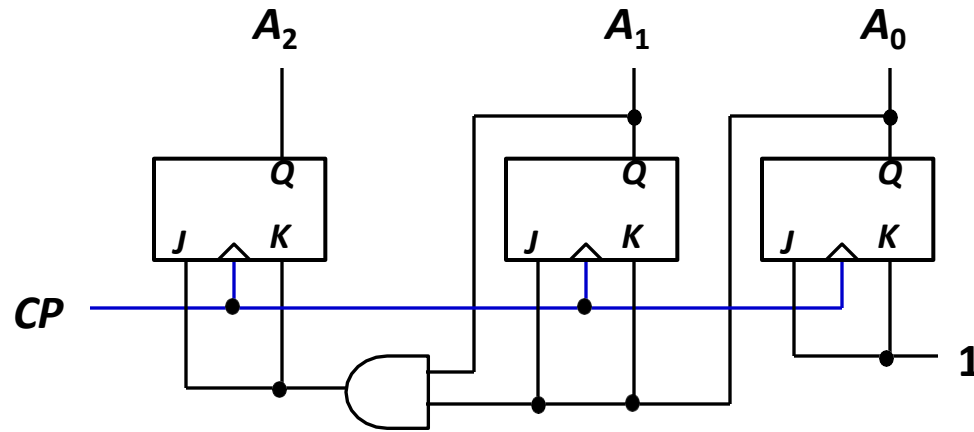


Synchronous (Parallel) Counters

- Example: 3-bit synchronous binary counter (cont'd).

$$TA_2 = A_1 \cdot A_0$$

$$TA_1 = A_0 \quad TA_0 = 1$$



Synchronous (Parallel) Counters

- Note that in a binary counter, the n^{th} bit (shown underlined) is always complemented whenever

$$\underline{0}11\dots11 \rightarrow \underline{1}00\dots00$$

$$\text{or } \underline{1}11\dots11 \rightarrow \underline{0}00\dots00$$

- Hence, X_n is complemented whenever

$$X_{n-1}X_{n-2} \dots X_1X_0 = 11\dots11.$$

- As a result, if T flip-flops are used, then

$$TX_n = X_{n-1} \cdot X_{n-2} \cdot \dots \cdot X_1 \cdot X_0$$



संयुक्त प्रौद्योगिकी

SYMBIOSIS

INSTITUTE OF TECHNOLOGY, NAGPUR

*Thank
you*

